

A neuronal least-action principle for real-time learning in cortical circuits

Walter Senn , Dominik Dold, Akos F. Kungl, Benjamin Ellenberger, Jakob Jordan, Yoshua Bengio, João Sacramento, Mihai A. Petrovici

Department of Physiology, University of Bern • Kirchoff-Institute for Physics, Heidelberg University • European Space Research and Technology Centre, European Space Agency • Insel Data Science Center, University Hospital Bern • MILA, University of Montreal • Department of Computer Science, ETH Zurich

Reviewed Preprint

Revised by authors after peer review.

About eLife's process

Reviewed preprint version 2

April 12, 2024 (this version)

Reviewed preprint version 1

August 22, 2023

Posted to preprint server

June 6, 2023

Sent for peer review

June 5, 2023

 https://en.wikipedia.org/wiki/Open_access

 Copyright information

Abstract

One of the most fundamental laws of physics is the principle of least action. Motivated by its predictive power, we introduce a neuronal least-action principle for cortical processing of sensory streams to produce appropriate behavioural outputs in real time. The principle postulates that the voltage dynamics of cortical pyramidal neurons prospectively minimize the local somato-dendritic mismatch error within individual neurons. For motor output neurons, it implies minimizing an instantaneous behavioural error. For deep network neurons, it implies a prospective firing to overcome integration delays and correct for possible output errors right in time. The neuron-specific errors are extracted in the apical dendrites of pyramidal neurons through a cortical microcircuit that tries to explain away the feedback from the periphery, and correct the trajectory on the fly. Any motor output is in a moving equilibrium with the sensory inputs and the motor feedback during the whole sensory-motor trajectory. Ongoing synaptic plasticity reduces the somato-dendritic mismatch error within each cortical neuron and performs gradient descent on the output cost at any moment in time. The neuronal least-action principle offers an axiomatic framework to derive local neuronal and synaptic dynamics for global real-time computation and learning in the brain and in physical substrates in general.

eLife assessment

This manuscript describes a potentially **important** theoretical framework to link predictive coding, error-based learning, and neuronal dynamics. The provided evidence is **solid**, but some details would benefit from additional clarification. The exposition of the manuscript is targeted for a specialist audience.

Introduction

Wigner's remark about the 'unreasonable effectiveness' of mathematics in allowing us to understand physical phenomena (Wigner, 1959 ) is famously contrasted by Gelfand's quip about its 'unreasonable ineffectiveness' in doing the same for biology (Borovik, 2021 ). Considering the component of randomness that is inherent to evolution, this may not be all that surprising.

However, while this argument holds just as well for the brain at the cellular level, ultimately brains are computing devices. At the level of computation, machine learning and neuroscience have revealed near-optimal strategies for information processing and storage, and evolution is likely to have found similar principles through trial and error (Hassabis *et al.*, 2017). Thus, we have reason to hope for the existence of fundamental principles of cortical computation that are similar to those we have found in the physical sciences. Eventually, it is important for such approaches to relate these principles back to brain phenomenology and connect function to structure and dynamics.

In physics, a fundamental measure of ‘effort’ is the action of a system, which nature seeks to ‘minimize’. Given an appropriate description of interactions between the systems constituents, the least-action principle can be used to derive the equations of motion of any physical system (Feynman *et al.*, 2011; Coopersmith, 2017). Here, we suggest that in biological information processing, a similar principle holds for prediction errors, which are of obvious relevance for cognition and behavior. Based on such errors, we formulate a neuronal least-action (NLA) principle which can be used to derive neuronal dynamics and map them to observed dendritic morphologies and cortical microcircuits. Within this framework, local synaptic plasticity at basal and apical dendrites can be derived by stochastic gradient descent on errors. The errors that are minimized refer to the errors in output neurons that are typically thought to represent motor trajectories, planned and encoded in cortical motor areas and ultimately in the spinal cord and muscles. In the context of motor control, a phenomenological ‘minimal action principle’ has previously been proposed that guides the planning and execution of movements (Feldman & Levin, 2009). Our neuronal least-action principle reformulates and formalizes the classical equilibrium point hypothesis (Latash, 2010) in a dynamical setting, linking it to optimality principles in sensory-motor control (Todorov, 2004).

Other attempts exist to link biological information processing and neural networks with the least-action principle, for instance by directly learning to reproduce a given trajectory (Amirikian & Lukashin, 1992), by minimizing the physical action for the muscle force generation by motor unit recruitment (Senn, Wyler, Clamann, Kleinle, Larkum, *et al.*, 1995), minimizing cognitive prediction errors (Alonso *et al.*, 2012), minimizing output errors with a weight-change regularization (Betti & Gori, 2016), minimizing psychomotor work (Fox & Kotelba, 2018), minimizing data transport through a network (Karkar *et al.*, 2021), minimizing the discrimination information (Summers, 2021), or minimizing the free energy (Friston, 2010; Friston *et al.*, 2022). Apart from the latter, however, these attempts remain far from the biology that seems to resist a formalization with the tool of physics – at least, when applied too strictly.

The fundamental novelty of our NLA principle is the way it deals with time. In physics, bodies interact based on where they are now, irrespective of what happens in the future. Living systems, instead, interact based on what could happen in the near future, and react early to stay alive. This difference is also mirrored in the way our NLA principle looks for an error-minimizing trajectory of brain states. We postulate that the brain trades with near-future states and seeks for a path that minimizes errors of these future states any moment in time. Looking ahead towards what will likely happen allows the network for correcting the internal trajectory of deep neurons early enough so that the delayed output moves along the desired path. The notion of looking into the future to gate a dynamical system is also central in optimal control theory (as expressed by the Bellman equation, see e.g. Todorov, 2006). Yet, starting with a neuronal action is more principled as it includes the derivation of the dynamical system itself that will be optimally controlled.

The insight into the time structure of biological information processing allows us to express a simple form of a total ‘mismatch energy’ for our cortical neuronal networks, from which we derive the dynamic neuronal and synaptic laws. In short, the mismatch energy within a single pyramidal neuron is the squared prediction error between basal dendrites and the soma, together

with the apical dendrites receiving a top-down feedback. The apical dendrites calculate a local prospective prediction error that looks ahead in time and overcomes neuronal integration delays (Fig. 1a). As a consequence, the output neurons are corrected on the fly by the prospective error processing, pushing them in real time closer to the desired path. In addition, the prospective errors are suited for gradient learning of the sensory synapses on the basal dendrites. This gradient learning is proven to reduce the error in the output neurons at any moment in time.

The NLA principle builds on and integrates various ingredients from existing work and theories. Output neurons, be they motor neurons or decision making neurons, are postulated to be ‘nudged’ towards the desired target time course by additional synaptic input to the soma or the proximal apical dendrite, as described by (Urbanczik & Senn, 2014). The cortical microcircuit with lateral ‘inhibition’ that seeks to cancel the top-down feedback in order to extract the apical error is inspired from Sacramento *et al.*, 2018 and Haider *et al.*, 2021. The energy-based approach for describing error-backpropagation for weak nudging is borrowed from the Equilibrium Propagation algorithm (Scellier & Bengio, 2017) that we generalize from a steady-state algorithm to real-time computation in cross-cortical microcircuits. Our theory covers both cases of weak and strong output nudging. For strong nudging, it likewise generalises the least-control principle (Meulemans, Zucchet, *et al.*, 2022) and the prospective configuration algorithm (Song *et al.*, 2024) from a steady-state to a dynamic real-time version, linking to optimal feedback control (Todorov & Jordan, 2002). Finally, the apical activity of our pyramidal neurons can be seen in the tradition of predictive coding (Rao & Ballard, 1999), where cortical feedback connections try to explain away lower-level activities. Yet, different from classical predictive coding, our prediction errors are integrated in the soma, and these errors are prospective in time. The errors extrapolate from current to future activities, so that their integration improves the network output in real time. The combination of an energy-based model with prospective coding in which neuronal integration delays are compensated on the fly enters also in Haider *et al.*, 2021.

The paper is organized as follows: we first define the prospective somato-dendritic mismatch error, construct out of this the mismatch energy of a network, and ‘minimise’ this energy to obtain the error-corrected, prospective voltage dynamics of the network neurons. We then show that the prospective error coding leads to an instantaneous and joint processing of low-pass filtered input signals and backpropagated errors. Applied to motor control, the instantaneous processing is interpreted as a moving equilibrium hypothesis according to which sensory inputs, network state, motor commands and muscle feedback are in a self-consistent equilibrium at any point of the movement. We then derive a local learning rule that globally minimizes the somato-dendritic mismatch errors across the network, and show how this learning can be implemented through error-extracting cortical microcircuits and dendritic predictive plasticity.

Results

Somato-dendritic mismatch errors and the Lagrangian of cortical circuits

We consider a network of neurons – identified as pyramidal cells – with firing rates $r_i(t)$ in continuous time t . The somatic voltage u_i of pyramidal neuron i is driven by the close-by basal input current, $\sum_j W_{ij}r_j$, with presynaptic rates r_j and synaptic weights W_{ij} , and an additional distal apical input e_i that will be learned to represent a prospective prediction error at any moment in time (Fig. 1a). While in classical rate-based neuron models the firing rate r_i of a neuron is a function of the somatic voltage, $\rho(u_i)$, the NLA principle implies that the effective firing rate of a cortical neuron is prospective. More concretely, the formalism derives a firing rate that linearly extrapolates from $\rho(u_i)$ into the future with the temporal derivative, $r_i = \rho(u_i) + \tau \dot{\rho}(u_i)$, where $\dot{\rho}(u_i)$ represents the temporal derivative of $\rho(u_i(t))$. There is experimental evidence for

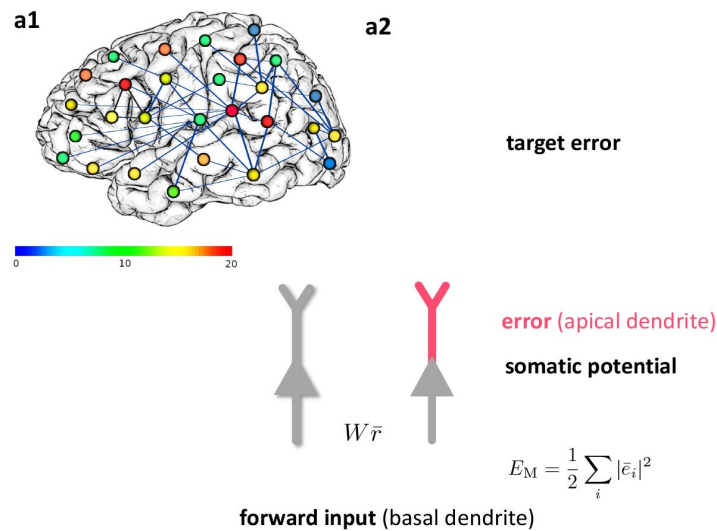


Figure 1.

Somato-dendritic mismatch energies and the neuronal least-action (NLA) principle.

(a1) Sketch of a cross-cortical network of pyramidal neurons described by NLA. **(a2)** Correspondence between elements of NLA and biological observables such as membrane voltages and synaptic weights. **(b1)** The NLA principle postulates that small variations $\delta \tilde{u}$ (dashed) of the trajectories $\tilde{u}$ (solid) leave the action invariant, $\delta A = 0$. It is formulated in the look-ahead coordinates $\tilde{u}$ (symbolized by the spyglass) in which ‘hills’ of the Lagrangian (shaded grey zones) are foreseen by the prospective voltage so that the trajectory can turn by early enough to surround them. **(b2)** In the absence of output nudging ($\beta = 0$), the trajectory $u(t)$ is solely driven by the sensory input, and prediction errors and energies vanish ($L = 0$, outer blue trajectory at bottom). When nudging the output neurons towards a target voltage ($\beta > 0$), somato-dendritic prediction errors appear, the energy increases (red dashed arrows symbolising the growing ‘volcano’) and the trajectory $u(t)$ moves out of the $L = 0$ hyperplane, riding on top of the ‘volcano’ (red trajectory). Synaptic plasticity $\dot{W}$ reduces the somato-dendritic mismatch along the trajectory by optimally ‘shoveling down the volcano’ (blue dashed arrows) while the trajectory settles in a new place on the $L = 0$ hyperplane (inner blue trajectory at bottom).

such prospective coding in cortical pyramidal neurons where the instantaneous rate r_i is in fact not only a function of the underlying voltage, but also a function of how quickly that voltage increases (see **Fig. 2a** [↗](#)).

The second central notion of the theory is the prospective error e_p , that we interpret as prospective somato-dendritic mismatch error in the individual network neurons,

$e_i = (u_i + \tau \dot{u}_i) - \sum_j W_{ij} r_j$. It is defined as a mismatch between the prospective voltage, $u_i + \tau \dot{u}_i$, and the weighted prospective input rates, $\sum_j W_{ij} r_j$. In the same way as the firing rates r_j linearly extrapolate into the future given the current voltages u_j of the presynaptic neurons j , the postsynaptic error is based on the linear extrapolation of its current voltage u_i using its temporal derivative, $u_i + \tau \dot{u}_i$. If the prospective error e_i is low-pass filtered with time constant τ , it takes the form $\bar{e}_i = u_i - \sum_j W_{ij} \bar{r}_j$, where $\bar{r}_j$ is the corresponding low-pass filtered firing rate of the presynaptic neuron j (that becomes a function of the presynaptic voltage, $\bar{r}_j = \rho(u_j)$, see Methods, Sect. A). We refer to $\bar{e}_i$ as somato-dendritic mismatch error of neuron i that, as compared to e_p , is non-prospective and instantaneous.

We next interpret the mismatch error $\bar{e}_i$ in terms of the morphology and biophysics of pyramidal neurons with basal and apical dendrites. While the error e_i is formed in the apical dendrite, this error is low-pass filtered and added to the somatic voltage u_i that is also driven by the low-pass filtered basal input $\sum_j W_{ij} \bar{r}_j$, so that $u_i = \sum_j W_{ij} \bar{r}_j + \bar{e}_i$. From the perspective of the basal dendrites, the low-pass filtered apical error $\bar{e}_i$ can be calculated as difference between the somatic voltage and the own local low-pass filtered input, $\bar{e}_i = u_i - \sum_j W_{ij} \bar{r}_j$. The somatic voltage u_i is assumed to be sampled in the basal dendrite by the backpropagating acting potentials (Urbanczik & Senn, 2014 [↗](#); Spicher *et al.*, 2017 [↗](#)). The apical error now appears as ‘somato-basal’ mismatch error, that both are summarized as somato-dendritic mismatch error. It tells the difference between ‘what a neuron does’, which is based on the somatic voltage u_i , and ‘what the basal inputs think it should do’, which is based on its own input $\sum W_{ij} \bar{r}_j$ (**Fig. 1a2** [↗](#)). The two quantities may deviate because neuron i gets additional ‘unpredicted’ apical inputs from higher-area neurons that integrates in the somatic voltage u_i . What cannot be predicted in u_i by the sensory-driven basal input remains as somato-basal (somato-dendritic) mismatch error $\bar{e}_i$.

Associated with this mismatch error is the somato-dendritic mismatch energy defined for each network neuron $i \in \mathcal{N}$ as the squared mismatch error,

$$E_i^M = \frac{1}{2} \bar{e}_i^2 = \frac{1}{2} \left(u_i - \sum_j W_{ij} \bar{r}_j \right)^2. \quad (1)$$

On a subset of output neurons of the whole network, $\mathcal{O} \subseteq \mathcal{N}$, a cost is defined as a function of the somatic voltage and some instructive reference signal such as targets or a reward. When a target trajectory $u_o^*(t)$ is available, the cost is defined at each time point as a squared target error,

$$C_o = \frac{1}{2} (\bar{e}_o^*)^2 = \frac{1}{2} (u_o^* - u_o)^2. \quad (2)$$

Much more general mismatch energies and cost functions are conceivable, encompassing e.g. conductance-based neurons or including further dynamic variables (see SI, Sect. F). The cost represents a performance measure for the entire network that produces the output voltages $u_o(t)$ in response to some input rates $r_{in}(t)$. The cost directly relates to behavioral or cognitive measures such as the ability of an animal or human to perform a particular task in real time. The target could be provided by explicit external supervision, for example target movements in time encoded by $u_o^*(t)$, it could represent an expected reward signal, or it could arise via self-supervision from other internal prediction errors.

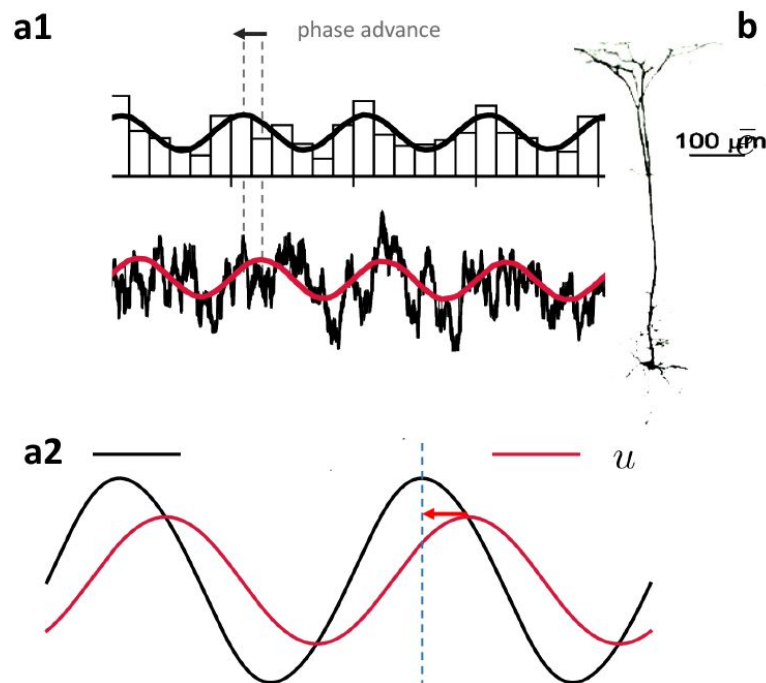


Figure 2.

Prospective coding in cortical pyramidal neurons enables instantaneous voltage-to-voltage transfer.

(a1) The instantaneous spike rate of cortical pyramidal neurons (top) in response to sinusoidally modulated noisy input current (bottom) is phase-advanced with respect to the input (adapted from Köndgen *et al.*, 2008). **(a2)** Similarly, in NLA, the instantaneous firing rate of a model neuron ($r = \rho(u) + \tau \dot{\rho}(u)$, black) is phase-advanced with respect to the underlying voltage (u , red, postulating that the low-pass filtered rate is a function of the voltage, $\bar{r} = \rho(u)$). **(b)** Dendritic input in the apical tree (here called $\bar{e}$) is instantaneously causing a somatic voltage modulation (u , modeling data from Ulrich, 2002). The low-pass filtering with τ along the dendritic shaft is compensated by a lookahead mechanism in the dendrite ($e = \bar{e} + \tau \dot{\bar{e}}$). In Ulrich (2002) a phase advance is observed even with respect to the dendritic input current, not only the dendritic voltage, although only for slow modulations (as here). **(c)** While the voltage of the first neuron (u_1) integrates the input rates r_{in} from the past (bottom black upward arrows), the output rate r_1 of that first neuron looks ahead in time, $r_1 = \rho(u_1) + \tau \dot{\rho}(u_1)$ (red dashed arrows pointing into the future). The voltage of the second neuron (u_2) integrates the prospective rates r_1 (top black upwards arrows). By doing so, it inverts the lookahead operation, resulting in an instantaneous transfer from $u_1(t)$ to $u_2(t)$ (blue arrow and circles).

We define the Lagrangian (or ‘total energy’) of the network as a sum across all mismatch energies and costs, weighted by the nudging strength β of the output neurons,

$$L = \sum_{i \in \mathcal{N}} E_i^M + \beta \sum_{o \in \mathcal{O}} C_o = \frac{1}{2} \sum_{i \in \mathcal{N}} \left(u_i - \sum_j W_{ij} \bar{r}_j \right)^2 + \frac{\beta}{2} \sum_{o \in \mathcal{O}} (u_o^* - u_o)^2. \quad (3)$$

The low-pass filtered presynaptic rates, $\bar{r}_j$, also encompass the external input neurons. While in classical energy-based approaches L is called the total energy, we call it the ‘Lagrangian’ because it will be integrated along real and virtual voltage trajectories as done in variational calculus (leading to the Euler-Lagrange equations, see below and SI, Sect. F). We ‘prospectively’ minimize L locally across a voltage trajectory, so that, as a consequence, the local synaptic plasticity for W_{ij} will globally reduce the cost along the trajectory (Theorem 1 below).

Due to the prospective coding, the Lagrangian can be minimal at any moment in time while the network dynamics evolves. This is different from the classical predictive coding (Rao & Ballard, 1999) and energy-based approaches (Scellier & Bengio, 2017; Song *et al.*, 2024), where a stimulus needs to be fixed in time while the network relaxes to a steady state, and only there the prediction error is minimized (see SI, Sect. C, comparison with Equilibrium Propagation).

The least-action principle expressed for prospective firing rates

Motivated by the prospective firing in pyramidal neurons, we postulate that cortical networks strive to look into the future to prevent instantaneous errors. Each neuron tries to move along a trajectory that minimizes its own mismatch error $\bar{e}_i$ across time (Fig. 1b1). The ‘neuronal currency’ with which each neuron ‘trades’ with others to choose its own error-minimizing trajectory is the future discounted membrane potential,

$$\tilde{u}(t) = \frac{1}{\tau} \int_t^\infty u(t') e^{-\frac{t'-t}{\tau}} dt'. \quad (4)$$

The prospective voltages $\tilde{u}$ are the ‘canonical coordinates’ entering the NLA principle, and in these prospective coordinates the overall network searches for a ‘least-action trajectory’. Since from $\tilde{u}$ we can recover the instantaneous voltage via $u = \tilde{u} - \tau \dot{\tilde{u}}$ (see SI, Sect. B), we can replace u in the Lagrangian and obtain L as a function of our new prospective coordinates $\tilde{u}$ and the ‘velocities’ $\dot{\tilde{u}}$, i.e. $L = L[\tilde{u}, \dot{\tilde{u}}]$, where bold fonts represent vectors. Inspired by the least-action principle from physics, we define the neuronal action A as a time-integral of the Lagrangian,

$$A = \int_{t_1}^{t_2} L[\tilde{u}(t), \dot{\tilde{u}}(t)] dt. \quad (5)$$

The NLA principle postulates that the trajectory $\tilde{u}(t)$ keeps the action A stationary with respect to small variations $\delta \tilde{u}$ (Fig. 1b1). In other words, nature chooses a trajectory such that, when deviating a little bit from it, say by $\delta \tilde{u}$, the value of A will not change (or at most up to second order in the variation), formally $\delta A = 0$. The motivation to search for a trajectory that keeps the action stationary is borrowed from physics. The motivation to search for a stationary trajectory by varying the near-future voltages $\tilde{u}$, instead of u , is assigned to the evolutionary pressure in biology to ‘think ahead of time’. To not react too late, internal delays involved in the integration of external feedback need to be considered and eventually need to be overcome. In fact, only for the ‘prospective coordinates’ defined by looking ahead into the future, even when only virtually, will a real-time learning from feedback errors become possible (as expressed by our Theorems below).

The equations of motion that keep the action stationary with respect to these prospective coordinates are known to satisfy the Euler-Lagrange equations

$$\frac{\partial L}{\partial \tilde{u}_i} - \frac{d}{dt} \frac{\partial L}{\partial \dot{\tilde{u}}_i} = 0. \quad (6)$$

Applying these equations to our Lagrangian yields a prospective version of the classical leaky integrator voltage dynamics, with rates $\mathbf{r}$ and errors $\mathbf{e}$ that are looking into the future (Methods, Sects A, B),

$$\tau \dot{\mathbf{u}} = -\mathbf{u} + \mathbf{W} \mathbf{r} + \mathbf{e}, \quad (7a)$$

$$\bar{\mathbf{e}} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^*. \quad (7b)$$

The ‘ $\cdot$ ’ denotes the component-wise product, and the weight matrix splits into weights from input neurons and weights from network neurons, $\mathbf{W} = (\mathbf{W}_{\text{in}}, \mathbf{W}_{\text{net}})$. While for output neurons a target error can be defined, $\bar{\mathbf{e}}_o^* = u_o^* - u_o$, for non-output neurons i no target exist and we hence set $\bar{\mathbf{e}}_i^* = 0$. In a control theoretic framework, the neuronal dynamics (Eq. 7a) represents the state trajectory, and the adjoint error dynamics (Eq. 7b) represents the integrated costate trajectory (Todorov, 2006).

From the point of view of theoretical physics, where the laws of motion derived from the least-action principle contain an acceleration term (as in Newton’s law of motion, like $m\ddot{x} = -x + F$ for a harmonic oscillator), one may wonder why no second-order time derivative appears in the NLA dynamics. As an intuitive example, consider driving into a bend. Looking ahead in time helps us to reduce the lateral acceleration by braking early enough, as opposed to braking only when the lateral acceleration is already present. This intuition is captured by minimizing the neuronal action A with respect to the discounted future voltages $\tilde{u}_i$ instead of the instantaneous voltages u_i . Keeping up an internal equilibrium in the presence of a changing environment requires to look ahead and compensate early for the predicted perturbations. Technically, the acceleration disappears because the Euler-Lagrange operator (Eq. 6) turns into a lookahead-gradient operator, $\frac{\partial}{\partial \tilde{u}_i} - \frac{d}{dt} \frac{\partial}{\partial \dot{\tilde{u}}_i} = \left(1 + \frac{d}{dt}\right) \frac{\partial}{\partial u_i}$, since the $\ddot{u}_i$ is absorbed via $\dot{\tilde{u}}_i - \tau \ddot{\tilde{u}}_i = \dot{u}_i$ (see Methods, Sect. A, and SI, Sect. F for the link to the least-action principle in physics).

Mathematically, the voltage dynamics in Eq. 7a specifies an implicit differential equation since $\dot{\mathbf{u}}(t)$ also appears on the right-hand side. This is because the prospective rates $\mathbf{r} = \rho(\mathbf{u}) + \tau \dot{\rho}(\mathbf{u})$ include $\dot{\mathbf{u}}$ through $\dot{\rho}(\mathbf{u}) = \rho'(\mathbf{u}) \cdot \dot{\mathbf{u}}$. Likewise, the prospective errors $\mathbf{e} = \bar{\mathbf{e}} + \tau \dot{\bar{\mathbf{e}}}$, with $\bar{\mathbf{e}}$ given in Eq. 7b and plugged into Eq. 7a, imply $\dot{\mathbf{u}}$ through $\dot{\bar{\mathbf{e}}}(\mathbf{u}) = \bar{\mathbf{e}}'(\mathbf{u}) \cdot \dot{\mathbf{u}}$. Nevertheless, the voltage dynamics can be stably run by replacing $\dot{\mathbf{u}}(t)$ on the right-hand side of Eq. 7a with the temporal derivative $\dot{\mathbf{u}}(t - dt)$ from the previous time step (technically, the Hessian $(\mathbf{1} - \mathbf{W} \rho' - \bar{\mathbf{e}}')$ is required to be strictly positive definite, see Methods Sect. F and SI, Sect. C). This ensures that the voltage dynamics of Eq. 7 can be implemented in cortical neurons with a prospective firing and a prospective dendritic error (see Fig. 2).

The error expression in Eq. 7b is reminiscent of error backpropagation (Rumelhart *et al.*, 1986) and can in fact be related (Methods, Sect. C). Formally, the errors are backpropagated via transposed network matrix, $\mathbf{W}_{\text{net}}^T$, modulated by $\bar{\mathbf{r}}_i'$, the derivative of $\bar{\mathbf{r}}_i = \rho(u_i)$ with respect to the underlying voltage. While the transpose can be constructed with various local methods (see Akrouit *et al.*, 2019; Max *et al.*, 2022) in our simulations we mainly adhere to the phenomenon

of feedback alignment (Lillicrap, Cownden, *et al.*, 2016) and consider fixed and randomized feedback weights $\mathbf{B}$ (unless stated differently). Recent control theoretical work is exploiting the same prospective coding technique as expressed in Eq. 7 to tackle general time-varying optimization problems (see Simonetto *et al.*, 2020 for a review and the SI, Sect. C for the detailed connection).

Prospective coding in neurons and instantaneous propagation

The prospective rates and errors entering via $\mathbf{r}$ and $\mathbf{e}$ in the NLA (Eq. 7) are consistent with the prospective coding observed in cortical pyramidal neurons *in vitro* (Köndgen *et al.*, 2008). Upon sinusoidal current injection into the soma, the somatic firing rate is advanced with respect to its voltage (Fig. 2a), effectively compensating for the delay caused by the current integration. Likewise, sinusoidal current injection in the apical tree causes a lag-less voltage response in the soma (Fig. 2b, Ulrich, 2002). While the rates and errors in general can be reconstructed from their low-pass filterings via $\mathbf{r} = \bar{\mathbf{r}} + \tau \dot{\bar{\mathbf{r}}}$ and $\mathbf{e} = \bar{\mathbf{e}} + \tau \dot{\bar{\mathbf{e}}}$, they become prospective in time because $\bar{\mathbf{r}}$ and $\bar{\mathbf{e}}$ are themselves instantaneous functions of the voltage $\mathbf{u}$, and hence $\mathbf{r}$ and $\mathbf{e}$ depend on $\dot{\mathbf{u}}$. The derivative of the membrane potential implicitly also appears in the firing mechanism of Hodgkin-Huxley-type conductances, with a quick depolarization leading to a stronger sodium influx due to the dynamics of the gating variables (Hodgkin & Huxley, 1952). This advances the action potential as compared to a firing that would only depend on $\mathbf{u}$, not $\dot{\mathbf{u}}$, giving an intuition of how such a prospective coding may arise. A similar prospective coding has been observed for retinal ganglion cells (Palmer *et al.*, 2015) and cerebellar Purkinje cells (Ostojic *et al.*, 2015), making a link from the visual input to the motor control.

To understand the instantaneous propagation through the network, we low-pass filter the dynamic equation $\mathbf{u} + \tau \dot{\mathbf{u}} = \mathbf{W}\mathbf{r} + \mathbf{e}$ (obtained by rearranging Eq. 7a), with $\bar{\mathbf{e}}$ given by Eq. 7b, to obtain the somatic voltage $\mathbf{u} = \mathbf{W}\bar{\mathbf{r}}(\mathbf{u}) + \bar{\mathbf{e}}(\mathbf{u})$. At any point in time, the voltage is in a moving equilibrium between forward and backpropagating inputs. Independently of the network architecture, whether recurrent or not, the output is an instantaneous function of the low-pass filtered input and a putative correction towards the target, $\mathbf{u}_o(t) = \mathbf{F}_W(\bar{\mathbf{r}}_{\text{in}}(t), \bar{\mathbf{e}}_o^*(t))$, see Fig. 2c and Methods, Sect. D. The mapping again expresses an instantaneous propagation of voltages throughout the network in response to both, the low-pass filtered input $\bar{\mathbf{r}}_{\text{in}}$ and feedback error $\bar{\mathbf{e}}_o^*$. This instantaneity is independent of the network size, and in a feed-forward network is independent of its depths (see also Haider *et al.*, 2021, where the instantaneity is on the rates, not the voltages). In the absence of the look-ahead activity, each additional layer would slow down the network relaxation time.

Notice that an algorithmic implementation of the time-continuous dynamics of a N -layer feedforward network would still need N calculation steps until information from layer 1 reaches layer N . However, this does not imply that an analog implementation of the prospective dynamics will encounter delays. To see why, consider a finite step-change $\Delta \mathbf{u}_1$ in the voltage of layer 1. In the absence of the look-ahead, $\Delta \mathbf{u}_1$ were mapped within the infinitesimal time interval dt to an infinitesimal change $d\mathbf{u}_2$ in the voltages of layer 2. But with a prospective firing rate, $\mathbf{r}_1 = \rho(\mathbf{u}_1) + \tau \rho'(\mathbf{u}_1) \dot{\mathbf{u}}_1$, a step-change $\Delta \mathbf{u}_1$ translates to a delta-function in $\mathbf{r}_1$, this in turn to a step-change in the low-pass filtered rates $\Delta \bar{\mathbf{r}}_1$, and therefore within dt to a step-change $\Delta \mathbf{u}_2$ in the voltages $\mathbf{u}_2$ of the postsynaptic neurons (Fig. 2c). Iterating this argument, a step-change $\Delta \mathbf{u}_1$ propagates ‘instantaneously’ through N layers within the ‘infinitesimal’ time interval $N dt$ to a step-change $\Delta \mathbf{u}_N$ in the last layer. When run in a biophysical device in continuous time that exactly implements the dynamical equations (Eq. 7), the implementation becomes an instantaneous computation (since $dt \rightarrow 0$). Yet, in a biophysical device, information has to be moved across space. This typically introduces further propagation delays that may not be captured in our formalism where low-pass filtering and prospective coding cancel each other exactly. Nevertheless, analog

computation in continuous time, as formalized here, offers an idea to ‘instantaneously’ realize an otherwise time consuming numerical recipe run on time-discrete computing systems that operate with a finite clock cycle.

Prospective control and the moving equilibrium hypothesis

Crucially, at the level of the voltage dynamics (Eq. 7a) the correction is based on the prospective error $\mathbf{e}$. This links our framework to optimal control theory and motor control where delays are also taken into account, so that a movement can be corrected early enough (Wolpert & Ghahramani, 2000; Todorov & Jordan, 2002; Todorov, 2004). The link between energy-based models and optimal control was recently drawn for strong nudging ($\beta \rightarrow \infty$) to learn individual equilibrium states (Meulemans, Zucchet, et al., 2022). Our prospective error $\mathbf{e}(t)$ appears as a ‘controller’ that, when looking at the output neurons, pushes the voltage trajectories towards the target trajectories. Depending on the nudging strength β , the control is tighter or weaker. For infinitely large β , the voltages of the output neurons are clamped to the time-dependent target voltages, $\mathbf{u}_o = \mathbf{u}_o^*$ (implying $\mathbf{e}_o^* = 0$), while their errors, $\bar{\mathbf{e}}_o = \mathbf{u}_o - (\mathbf{W}\bar{\mathbf{r}})_o$, instantaneously correct all network neurons. For small β , the output voltages are only weakly controlled, and they are dominated by the forward input, $\mathbf{u}_o \approx (\mathbf{W}\bar{\mathbf{r}})_o$.

To show how the NLA principle with the prospective coding globally maps to cortico-spinal circuits we consider the example of motor control. In the context of motor control, our network mapping $\mathbf{u}_o = \mathbf{F}_W(\bar{\mathbf{r}}_{in}, \bar{\mathbf{e}}_o^*)$ can be seen as a forward internal model that quickly calculates an estimate of the future muscle length $\mathbf{u}_o$ based on some motor plans, sensory inputs, and the current proprioceptive feedback (Fig. 3a). Forward models help to overcome delays in the execution of the motor plan by predicting the outcome, so that the intended motor plans and commands can be corrected on the fly (Kawato, 1999; Wolpert & Ghahramani, 2000).

The observation that muscle spindles prospectively encode the muscle length and velocity (Dimitriou & Edin, 2010) suggests that the prospective coding in the internal forward model mirrors the prospective coding in the effective forward pathway. This forward pathway leads from the motor plan to the spindle feedback, integrating also cerebellar and brainstem feedback (Kawato, 1999). Based on the motor plans, the intended spindle lengths and the effective muscle innervation are communicated via descending pathway to activate the γ - and α -motoneurons, respectively (Li et al., 2015). The mapping from the intended arm trajectory to the intended spindle lengths via γ -innervation is mainly determined by the joint geometry. The mapping from the intended arm trajectory to the force generating α -innervation, however, needs to also take account of the internal and external forces, and this is engaging our network $\mathbf{W}$.

When we prepare an arm movement, spindles in antagonistic muscles pairs that measure the muscle length are tightened or relaxed before the movement starts (Papaioannou & Dimitriou, 2021). According to the classical equilibrium-point hypothesis (Feldman & Levin, 2009; Latash, 2010), top-down input adjusts the activation threshold of the spindles through (γ)-innervation from the spinal cord so that slight deviations from the equilibrium position can be signaled (Fig. 3a). We postulate that this γ -innervation acts also during the movement, setting an instantaneous target $\mathbf{u}_o^*(t)$ for the spindle lengths. The effective lengths of the muscle spindles is $\mathbf{u}_o$, and the spindles are prospectively signaling back the deviation from the target through the I_a -afferents (Dimitriou & Edin, 2010; Dimitriou, 2022). The low-pass filtered I_a -afferents may be approximated by a threshold-nonlinearity, $\bar{I}_a = \beta[\mathbf{u}_o - \mathbf{u}_o^*]^+$, with β being interpreted as spindle gain (Latash, 2018). Combining the feedback from agonistic and antagonistic muscle pairs allows for extracting the scaled target error $\beta\bar{\mathbf{e}}_o = \beta(\mathbf{u}_o^* - \mathbf{u}_o)$. Taking account of the prospective feedback, we postulate the *moving equilibrium hypothesis* according to which the instructional inputs, $\bar{\mathbf{r}}_{in}$, the spindle feedback, $\beta\bar{\mathbf{e}}_o^*$, and the muscle lengths, $\mathbf{u}_o$, are at any

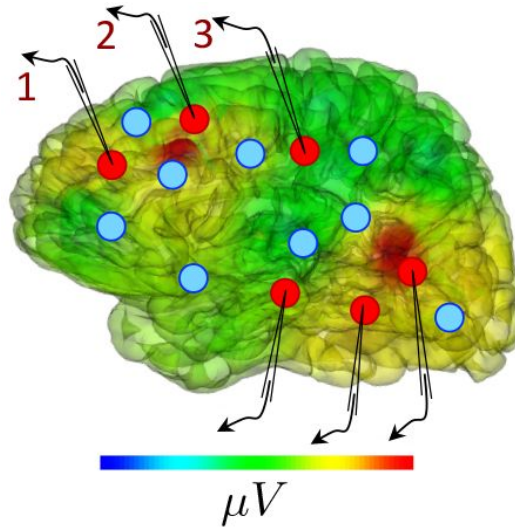


Figure 3.

Moving equilibrium hypothesis for motor control and real-time learning of cortical activity.

(a) A voluntary movement trajectory can be specified by the target length of the muscles in time, $\mathbf{u}_o^*$, encoded through the γ -innervation of muscle spindles, and the deviation of the effective muscle lengths from the target, $\mathbf{u}_o - \mathbf{u}_o^* = -\bar{\mathbf{e}}_o^*$. The I_{σ} -afferents emerging from the spindles prospectively encode the error, so that their low-pass filtering is roughly proportional to the length deviation, truncated at zero (red). The moving equilibrium hypothesis states that the low-pass filtered input $\bar{\mathbf{r}}_{\text{in}}$, composed of the movement plan $\bar{\mathbf{r}}_{\text{in}}^{\text{plan}}$ and the sensory input (here encoding the state of the plant e.g. through visual and proprioceptive input, $\bar{\mathbf{r}}_{\text{in}}^{\text{vis}}$ and $\bar{\mathbf{r}}_{\text{in}}^{\text{prop}}$), together with the low-pass filtered error feedback from the spindles, $\bar{\mathbf{e}}_o^*$, instantaneously generate the muscle lengths, $\mathbf{u}_o = F_W(\bar{\mathbf{r}}_{\text{in}}, \bar{\mathbf{e}}_o^*)$, and are thus at any point in time in an instantaneous equilibrium (defined by Eq. 7). (b1) Intracortical iEEG activity recorded from 56 deep electrodes and projected to the brain surface. Red nodes symbolize the 56 iEEG recording sites modeled alternately as input or output neurons, and blue nodes symbolize the 40 'hidden' neurons for which no data is available, but used to reproduce the iEEG activity. (b2) Corresponding NLA network. During training, the voltages of the output neurons were nudged by the iEEG targets (black input arrows, but for all red output neurons). During testing, nudging was removed for 14 out of these 56 neurons (here, represented by neurons 1, 2, 3). (c1) Voltage traces for the 3 example neurons in a2, before (blue) and after (red) training, overlaid with their iEEG target traces (grey). (c2) Total cost, integrated over a window of 8 s of the 56 output nodes during training with sequences of the same duration. The cost for the test sequences was evaluated on a 8 s window not used during training.

point of the movement in a dynamic equilibrium. The moving equilibrium hypothesis extends the classical equilibrium-point hypothesis from the spatial to the temporal domain (for a formal definition of a moving equilibrium see Methods, Sect. F).

Prediction errors are also reduced when motor units within a muscle are recruited according to the size principle (Senn, Wyler, Clamann, Kleinle, Lüscher, *et al.*, 1997 [\[1\]](#)), which itself was interpreted in terms of the physical least-action principle (Senn, Wyler, Clamann, Kleinle, Larkum, *et al.*, 1995 [\[2\]](#)). With regard to the interpretation of the prospective feedback error e_o^* as spindle activity, it is worth noticing that in humans the spindle activity is not only ahead of the muscle activation (Dimitriou & Edin, 2010 [\[3\]](#)), but also shares the property of a motor error (Dimitriou, 2016 [\[4\]](#)). The experiments show that during the learning of a gated hand movement, spindle activity is initially stronger when making movement errors, and it returns back to baseline with the success of learning. This observation is consistent with the NLA principle, telling that the proprioceptive prediction errors are minimized through the movement learning. We next address how the synaptic strengths $\mathbf{W}$ involved in producing the muscle length can be optimally adapted to capture this learning.

Local plasticity at basal synapses minimizes the global cost in real time

The general learning paradigm starts with input time series $r_{in(t),i}$ and target time series $u_o^*(t)$, while assuming that the target series are an instantaneous function of the low-pass filtered input series, $u_o^*(t) = \mathbf{F}^*(\bar{\mathbf{r}}_{in}(t))$. The low-pass filtering in the individual inputs could be with respect to any time constant $\tau_{in,i}^*$ (that may also be learned, see SI, Sect. B). Yet, for simplicity we assume the same time constant τ for low-pass filtering the rates of the network neurons and input neurons. The goal of learning is to adapt the synaptic strengths $\mathbf{W}$ in the student network so that this moves towards the target mapping, $\mathbf{F}_W \rightarrow \mathbf{F}^*$. The local synaptic plasticity will also reduce the global cost C defined on the output neurons o in terms of the deviation of the voltage from the target, $u_o^* - u_o$ (Eq. 2 [\[5\]](#)).

The problem of changing synaptic weights to correct the behavior of downstream neurons, potentially multiple synapses away, is typically referred to as the credit assignment problem and is notoriously challenging in physical or biological substrates operating in continuous time. A core aspect of the NLA principle is how it relates the global cost C to the Lagrangian L and eventually to somato-dendritic prediction errors $\bar{\mathbf{e}}$ that can be reduced through local synaptic plasticity $\dot{\mathbf{W}}$. We define this synaptic plasticity as partial derivative of the Lagrangian with respect to the weights, $\dot{\mathbf{W}} \propto -\frac{\partial L}{\partial \mathbf{W}} = \bar{\mathbf{e}} \bar{\mathbf{r}}^T$. Since the somato-dendritic mismatch error is $\bar{\mathbf{e}} = \mathbf{u} - \mathbf{W} \bar{\mathbf{r}}$, this leads to the local learning rule of the form ‘postsynaptic error times low-pass filtered presynaptic rate’,

$$\dot{\mathbf{W}} = \eta (\mathbf{u} - \mathbf{W} \bar{\mathbf{r}}) \bar{\mathbf{r}}^T. \quad (8)$$

The plasticity rule runs simultaneously to the neuronal dynamics in the presence of a given nudging strength β that tells how strongly the voltage of an output neurons is pushed towards the target, $u_o \rightarrow u_o^*$. The learning rule is local in space since $\mathbf{W} \bar{\mathbf{r}}$ is represented as voltage of the basal dendrites, and the somatic voltage $\mathbf{u}$ may be read out at the synaptic site on the basal dendrite from the backpropagating action potentials that sample $\mathbf{u}$ at a given time (Urbaniczik & Senn, 2014 [\[6\]](#)). The basal voltage $\mathbf{W} \bar{\mathbf{r}}$ becomes the dendritic prediction of the somatic activity $\mathbf{u}$, interpreting Eq. 8 [\[5\]](#) as ‘dendritic predictive plasticity’.

We have derived the neuronal dynamics as a path that keeps the action stationary. Without external teaching signal, the errors vanish and the voltage trajectory wriggles on the bottom of the energy landscape ($L = 0$, Fig. 1b2 [\[5\]](#)). If the external nudging is turned on, $\beta > 0$, errors emerge and

hills grow out of the landscape. The trajectory still tries to locally minimize the action, but it is lifted upwards on the hills ($L > 0$, **Fig. 1b2**). Synaptic plasticity reshapes the landscape so that, while keeping β fixed, the errors are reduced and the landscape again flattens. The transformed trajectory settles anew in another place (inside the ‘volcano’ in **Fig. 1b2**). Formally, the local plasticity rule (**Eq. 8**) is shown to perform gradient descent on the Lagrangian and hence on the action. In the energy landscape picture, plasticity ‘shovels off’ energy along the voltage path so that this is lowered most efficiently. The error that is back-propagated through the network tells at any point on the voltage trajectory how much to ‘dig’ in each direction, i.e. how to adapt the basal input in each neuron in order to optimally lower the local error.

The following theorem tells that synaptic plasticity $\dot{\mathbf{W}}$ pushes the network mapping $\mathbf{u}_o = \mathbf{F}_W(\bar{\mathbf{r}}_{\text{in}})$ towards the target mapping $\mathbf{u}_o^* = \mathbf{F}^*(\bar{\mathbf{r}}_{\text{in}})$ at any moment in time. The convergence of the mapping is a consequence of the fact the plasticity reduces the Lagrangian $L = E^M + \beta C$ along its gradient.

Theorem 1

(real-time Dendritic Error Propagation, rt-DeEP)

Consider an arbitrary network $\mathbf{W}$ with voltage and error dynamics following **Eq. 7**. Then the local plasticity rule $\dot{\mathbf{W}} \propto \bar{\mathbf{e}} \bar{\mathbf{r}}^T$ (**Eq. 8**), acting at each moment along the voltage trajectories, is gradient descent

1. on the Lagrangian L for any nudging strength $\beta \geq 0$, i.e. $\bar{\mathbf{e}} \bar{\mathbf{r}}^T = -\frac{dL}{d\mathbf{W}}$, with

$$\lim_{\beta \rightarrow \infty} \bar{\mathbf{e}} \bar{\mathbf{r}}^T = -\frac{dE^M}{d\mathbf{W}} \propto \dot{\mathbf{W}}.$$

2. on the cost C for small nudging, $\beta \rightarrow 0$, while up-scaling the error to $\frac{1}{\beta} \bar{\mathbf{e}}$, i.e.

$$\lim_{\beta \rightarrow 0} \frac{1}{\beta} \bar{\mathbf{e}} \bar{\mathbf{r}}^T = -\frac{dC}{d\mathbf{W}} \propto \dot{\mathbf{W}}.$$

The gradient statements hold at any point in time (long enough after initialization), even if the input trajectories $\mathbf{r}_{\text{in}}(t)$ contain delta-functions and the target trajectories $\mathbf{u}_o^*(t)$ contain step-functions.

Loosely speaking, the NLA enables the network to localize in space and time an otherwise global problem: what is good for a single neuron (the local plasticity) becomes good for the entire network (the gradient on the global cost). Learning is possible any point in time along the trajectory because the NLA inferred a prospective voltage dynamics expressed in prospective firing rates r_i and prospective errors e_i of the network neurons. In the limit of strong nudging ($\beta \rightarrow \infty$), the learning rule performs gradient descent on the mismatch energies E_i^M in the individual neurons. If the network architecture is powerful enough so that after learning all the mismatch energies vanish, $E_i^M = 0$, then the cost will also vanish, $C = \frac{1}{2} \|\mathbf{u}_o^* - \mathbf{u}_o\|^2 = 0$. This is because for the output neurons the mismatch error includes the target error (**Eq. 7b**). In the limit of weak nudging ($\beta \rightarrow 0$), the learning rule performs gradient descent on C , and with this also finds a local minimum of the mismatch energies.

In the case of weak nudging and a single steady-state equilibrium, the NLA algorithm reduces to the Equilibrium Propagation algorithm (Scellier & Bengio, 2017) that minimizes the cost C for a constant input and a constant target. In the case of strong nudging and a single steady-state equilibrium, the NLA principle reduces to the Least-Control Principle (Meulemans, Zucchet, et al., 2022) that minimizes the mismatch energy E^M for a constant input and a constant target, with the apical prediction error becoming the prediction error from standard predictive coding (Rao &

Ballard, 1999 [↗](#)). While in the Least-Control Principle the inputs and outputs are clamped to fixed values, the output errors are backpropagated and the network equilibrates in a steady state where the corrected network activities reproduce the clamped output activities. This state is called the ‘prospective configuration’ in Song *et al.*, 2024 [↗](#) because neurons deep in the network are informed about the distal target already during the inference, and are correspondingly adapted to be consistent with this distal target. In the NLA principle, after an initial transient, the network always remains in the moving equilibrium due to the prospective coding. While inputs and targets dynamically change, the network moves along a continuous sequence of prospective configurations.

In the motor control example, the theorem tells that a given target motor trajectory $\mathbf{u}_o^*(t)$ is learned to be reproduced with the forward model $\mathbf{u}_o(t) = \mathbf{F}_W(\bar{\mathbf{r}}_{\text{in}}(t))$, by applying the dendritic predictive plasticity for the network neurons (Eq. 8 [↗](#)). We next exemplify the theory by looking into the brain, reproducing cortical activity, and showing how a multi-layer cortical network can learn a sensory-motor mapping while staying in a moving equilibrium throughout the training.

Reproducing intracortical EEG recordings and recognizing handwritten digits

As illustration we consider a recurrently connected network that learns to represent intracortical electroencephalogram (iEEG) data from epileptic patients (Fig. 3b [↗](#)). For each electrode, we assign a neuron within this network to represent the activity of the cell cluster recorded in the corresponding iEEG signal via its membrane potential. During learning, a randomly selected subset of electrode neurons are nudged towards the target activity from recorded data while learned to be reproduced by the other neurons. After learning, we can present only a subset of electrode neurons with previously unseen recordings and observe how the activity of the other neurons closely matches the recordings of their respective electrodes (Fig. 3c [↗](#)). The network derived from NLA is thus able to learn complex correlations between signals evolving in real time by embedding them in a recurrent connectivity structure.

As an example of sensory-motor processing in the NLA framework, we next consider a well-studied image recognition task, here reformulated in a challenging time-continuous setting, and interpreted as motor task where 1 out of 10 fingers has to be bent upon seeing a corresponding visual stimulus (see Fig. 3a [↗](#)). In the context of our moving equilibrium hypothesis, we postulate that during the learning phase, but not the testing phase, an auditory signal identifies the correct finger and sets the target spindle lengths of 10 finger flexors, $\mathbf{u}_o^*(t)$. The target spindle length encodes the desired contraction of a flexor muscle in the correct finger upon the visual input $\mathbf{r}_{\text{in}}(t)$, and a corresponding relaxation for the 9 incorrect fingers.

We train a hierarchical three-layer network on images of handwritten digits (MNIST, LeCun, 1998 [↗](#)), with image presentation times between 0.5τ ($= 5$ ms) and 20τ ($= 200$ ms, with $\tau = 10$ the membrane time constant). Fig. 4a-c1 [↗](#) depict the most challenging scenario with the shortest presentation time. Synaptic plasticity is continuously active, despite the network never reaching a temporal steady state (Fig. 4b1 [↗](#)). Due to the lookahead firing rates in the NLA, the mismatch errors $\bar{e}_i(t)$ represent the correct gradient and propagate without lag throughout the network. As a consequence, our mismatch errors are almost equal to the errors obtained from classical error backpropagation applied at each time step to the purely forward network (i.e. the network that suppresses the error-correction $\bar{e}$ of the voltage and instead considers the ‘classical’ voltage $\mathbf{u}_l = \mathbf{W}_l \rho(\mathbf{u}_{l-1})$ only, see blue dots in Fig. 4b2 [↗](#)). The network eventually learned to implement the mapping $\mathbf{u}_o = \mathbf{F}_W(\bar{\mathbf{r}}_{\text{in}}) \approx \mathbf{u}_o^*$ with a performance comparable to error-backpropagation at each dt , despite the short presentation time of only 5 ms (Fig. 4c1 [↗](#)). The approximation is due to the fact that the NLA learns an instantaneous mapping from the low-pass filtered input rates $\bar{\mathbf{r}}_{\text{in}}$

to the output voltage u_o , while the mapping from the original input rates r_{in} to the voltages u_1 of the first-layer neurons (and hence also to the output voltages u_o) is delayed by τ_{in} . Since in the simulations the target voltages u_o^* were switched instantaneously with r_{in} (and not with $\bar{r}_{in}$), however, a mismatch error between u_o and u_o^* remains for stimulus presentation times shorter than τ_{in} (Fig. 4c2). The Latent Equilibrium (Haider *et al.*, 2021) avoids these temporal limitations by implementing an instantaneous mapping on the rates instead on the voltages (Methods, Sect. F).

The instantaneous voltage propagation reduces an essential constraint of previous models of bioplausible error backpropagation (e.g., Scellier & Bengio (2017), Whittington & Bogacz (2017), and Sacramento *et al.* (2018), with reviews Richards *et al.* (2019), Whittington & Bogacz (2019), and Lillicrap, Santoro, *et al.* (2020)): without lookahead firing rates, networks need much longer to correctly propagate errors across layers, with each layer roughly adding another membrane time constant of 10 ms, and thus cannot cope with realistic input presentation times. In fact, in networks without lookahead output, learning is only successful if plasticity is switched off while the network dynamics did not reach a stationary state during a stimulus presentation interval (Fig. 4c2). As a comparison, neuronal response latency to flashing stimuli in the primary visual cortex (V1) of rats in a synchronized state are in the order of 50 ms (Wang *et al.*, 2014). The latency shortens to roughly 40 ms if in a desynchronized state (Wang *et al.*, 2014).

In humans, cortical responses to expected visual stimuli (rectangulars) are observed 70-90 ms ahead of the responses observed in the absence of expectations (which have a latency of roughly 150 ms, see Blom *et al.*, 2020). A remaining latency in the presence of prospective coding is consistent with a remaining response delay of τ_{in} in the presented NLA principle. The minimal cortical response latency of roughly 50 ms to an expected rectangular (Blom *et al.*, 2020) compares with the roughly 80 ms cortical response time when humans have to decide whether a flashed image contains an animal or not (Thorpe *et al.*, 1996). The ongoing higher cortical processing during animal detection does not seem to substantially slow down cortical responses to sensory stimuli. This supports a ‘sensory equilibrium hypothesis’, the sensory correlate of our ‘moving equilibrium hypothesis’ for motor control, that enables us to interact with a dynamic visual environment in almost real time (as also suggested in Blom *et al.*, 2020). Notice that, from a technical perspective, making the time constants of individual cortical neurons arbitrarily short leads to network instabilities and is unlikely the option chosen by the brain (see SI Sect. C, comparison with Equilibrium Propagation).

Implementation in cortical microcircuits

So far, we did not specify how errors e appearing in the differential equation for the voltage (Eq. 7a) are transmitted across the network in a biologically plausible manner. Building on Sacramento *et al.*, 2018, we propose a cortical microcircuit to enable this error transport, with all neuron dynamics evolving according to the NLA principle. Although the idea applies for arbitrarily connected networks, we use the simpler case of functionally feedforward networks to illustrate the flow of information in these microcircuits (Fig. 5a).

For such an architecture, pyramidal neurons in area l (that is a ‘layer’ of the feedforward network) are accompanied by a pool of interneurons in the same layer (area). The dendrites of the interneurons integrate in time (with time constant τ) lateral input from pyramidal neurons of the same layer (r_l) through plastic weights W_l^{IP} . Additionally, interneurons receive ‘top-down nudging’ from pyramidal neurons in the next layer through randomly initialized and fixed backprojecting synapses B_l^{IP} targeting the somatic region, and interneuron nudging strength β^l . The notion of ‘top-down’ originates from the functionally feed-forward architecture leading from sensory to ‘higher cortical areas’. In the context of motor control, the highest ‘area’ is last stage controlling the muscle lengths, being at the same time the first stage for the proprioceptive input (Fig. 3a).

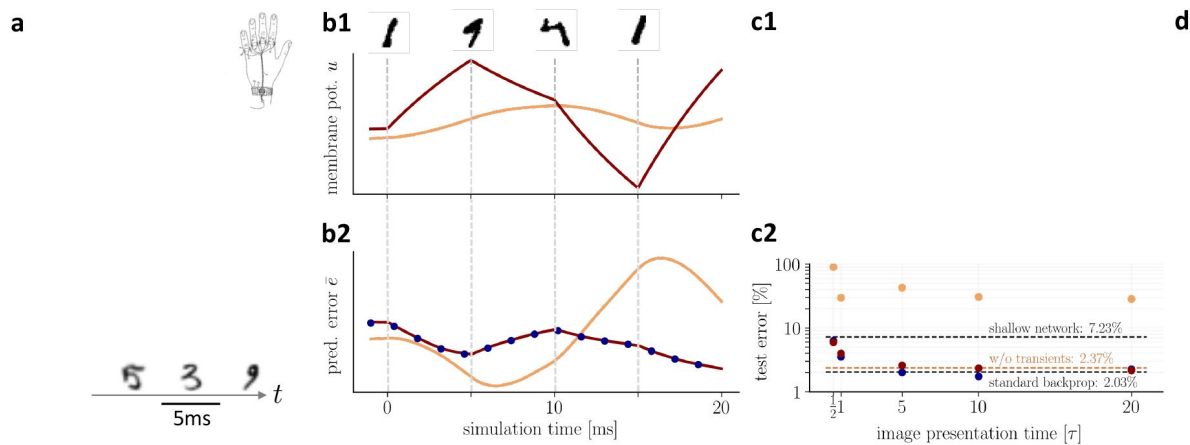


Figure 4.

On-the-fly learning of finger responses to visual input with real-time Dendritic Error Propagation (rt-DeEP).

(a) Functionally feedforward network with handwritten digits as visual input ($r_{in}^{(2)}(t)$ in Fig. 3a), here from the MNIST data set, 5 ms presentation time per image), backprojections enabling credit assignment, and activity of the 10 output neurons interpreted as commands for the 10 fingers (forward architecture: $784 \times 500 \times 10$ neurons). **(b)** Example voltage trace (b1) and local error (b2) of a hidden neuron in NLA (red) compared to an equivalent network without lookahead rates (orange). Note that neither network achieves a steady state due to the extremely short input presentation times. Errors calculated via exact backpropagation, i.e. by using the error backpropagation algorithm on a purely feedforward NLA network at every simulation time step (with output errors scaled by β), shown for comparison (blue dots). **(c)** Comparison of network models during and after learning. Color scheme as in (b). **(c1)** The test error under NLA evolves during learning on par with classical error backpropagation performed each Euler dt based on the feedforward activities. In contrast, networks without lookahead rates are incapable of learning such rapidly changing stimuli. **(c2)** With increasing presentation time, the performance under NLA further improves, while networks without lookahead rates stagnate at high error rates. This is caused by transient, but long-lasting misrepresentation of errors following stimulus switches: when plasticity is turned off during transients and is only active in the steady state, comparably good performance can be achieved (dashed orange). **(d)** Receptive fields of 6 hidden-layer neurons after training, demonstrating that even for very brief image presentation times (5ms), the combined neuronal and synaptic dynamics are capable of learning useful feature extractors such as edge filters.

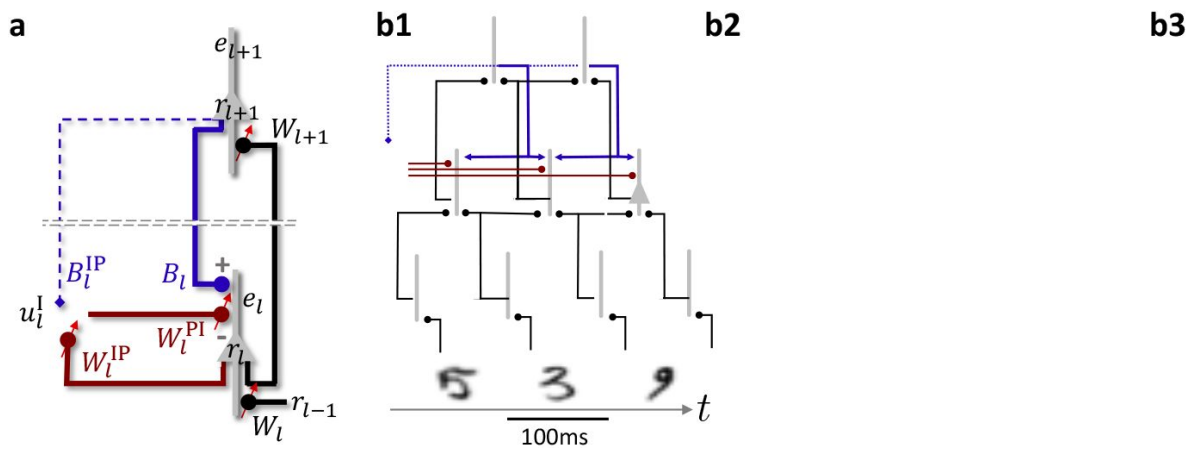


Figure 5.

Hierarchical plastic microcircuits implement real-time Dendritic Error Learning (rt-DeEL).

(a) Microcircuit with ‘top-down’ input (originating from peripheral motor activity, blue line) that is explained away by the lateral input via interneurons (dark red), with the remaining activity representing the error $\bar{e}_l$. Plastic connections are denoted with a small red arrow and nudging with a dashed line. (b1) Simulated network with 784-300-10 pyramidal-neurons and a population of 40 interneurons in the hidden layer used for the MNIST learning task where the handwritten digits have to be associated to the 10 fingers. (b2) Test errors for rt-DeEL with joint tabula rasa learning of the forward and lateral weights of the microcircuit. A similar performance is reached as with classical error backpropagation. For comparability, we also show the performance of a shallow network (dashed line). (b3) Angle derived from the Frobenius norm between the lateral pathway $W_l^{IP} W_l^{PI}$ and the feedback pathway $B_l W_{l+1}$. During training, both pathways align to allow correct credit assignment throughout the network. Indices are dropped in the axis label for readability.

According to the biophysics of the interneuron, the somatic membrane potential becomes a convex combination of the two types of afferent input (Urbanczik & Senn, 2014 [↗](#)),

$$\mathbf{u}_l^I = (1 - \beta^I) \mathbf{W}_l^{IP} \bar{\mathbf{r}}_l + \beta^I \mathbf{B}_l^I \mathbf{u}_{l+1}. \quad (9)$$

In the biological implementation, the feedback input is mediated by the low-pass filtered firing rates $\bar{\mathbf{r}}_{l+1} = \rho(\mathbf{u}_{l+1})$, not by $\mathbf{u}_{l+1}$ as expressed in the above equation. Yet, we argue that for a threshold-linear ρ the ‘top-down nudging’ by the rate $\bar{\mathbf{r}}_{l+1}$ is effectively reduced to a nudging by the voltage $\mathbf{u}_{l+1}$. This is because errors are only backpropagated when the slope of the transfer function is positive, $\rho'_{l+1} > 0$, and hence when the upper-layer voltage is in the linear regime. For more general transfer functions, we argue that short-term synaptic depression may invert the low-pass filtered presynaptic rate back to the presynaptic membrane potential, $\bar{\mathbf{r}}_{l+1} \rightarrow \mathbf{u}_{l+1}$, provided that the recovery time constant τ matches the membrane time constant (see end of Results and SI, Sect. A).

Apical dendrites of pyramidal neurons in each layer receive top-down input from the pyramidal population in the upper layer through synaptic weights $\mathbf{B}_l$. These top-down weights could be learned to predict the lower-layer activity (Rao & Ballard, 1999 [↗](#)) or to become the transposed of the forward weight matrix ($\mathbf{B}_l = \mathbf{W}_{l+1}^T$, Max *et al.*, 2022), but for simplicity we randomly initialized them and keep them fixed (Lillicrap, Santoro, *et al.*, 2020). Beside the top-down projections the apical dendrites also receive lateral input via an interneuron population in the same layer through synaptic weights $-\mathbf{W}_l^{PI}$ that are plastic and will be learned to obtain suitable dendritic errors. The ‘-’ sign is suggestive for these interneurons to subtract away the top-down input entering through $\mathbf{B}_l$ (while the weights can still be positive or negative). Assuming again a conversion of rates to voltages, also for the inhibitory neurons that may operate in a linear regime, the overall apical voltage becomes

$$\bar{e}_l^A = \mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{PI} \mathbf{u}_l^I. \quad (10)$$

What cannot be explained away from the top-down input $\mathbf{B}_l \mathbf{u}_{l+1}$ by the lateral feedback, $\mathbf{W}_l^{PI} \mathbf{u}_l^I$, remains as dendritic prediction error $\bar{e}_l^A$ in the apical tree (Fig. 5a [↗](#)). If the top-down and lateral feedback weights are learned as outlined next, these apical prediction errors take the role of the backpropagated errors in the classical backprop algorithm.

To adjust the interneuron circuit in each layer (‘area’), the synaptic strengths from pyramidal-to-interneurons, $\mathbf{W}_l^{IP}$, are learned to minimize the interneuron mismatch energy,

$E_l^{IP} = \frac{1}{2} \|\mathbf{u}_l^I - \mathbf{W}_l^{IP} \bar{\mathbf{r}}_l\|^2$. The interneurons, while being driven by the lateral inputs $\mathbf{W}_l^{PI} \bar{\mathbf{r}}_l$, learn to reproduce the upper-layer activity that also nudges the interneuron voltage. Learning is accomplished if the upper-layer activity, in the absence of an additional error on the upper layer, is fully reproduced in the interneurons by the lateral input.

Once the interneurons learned to represent the ‘error-free’ upper-layer activity, they can be used to explain away the top-down activities that also project to the apical trees. The synaptic strengths from the inter-to-pyramidal neurons, $\mathbf{W}_l^{PI}$, are learned to minimize the apical mismatch energy, $E_l^{PI} = \frac{1}{2} \|\bar{e}_l^A\|^2 = \frac{1}{2} \|\mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{PI} \mathbf{u}_l^I\|^2$. While in the absence of an upper-layer error, the top-down activity $\mathbf{B}_l \mathbf{u}_{l+1}$ can be fully cancelled by the interneuron activity $\mathbf{W}_l^{PI} \mathbf{u}_l^I$, a neuron-specific error will remain in the apical dendrites of the lower-level pyramidal neurons if there was an error endowed in the upper-layer neurons. Gradient descent learning on these two energies results in the learning rules for the P-to-I and I-to-P synapses,

$$\dot{\mathbf{W}}_l^{IP} = \eta^{IP} (\mathbf{u}_l^I - \mathbf{W}_l^{IP} \bar{\mathbf{r}}_l) \bar{\mathbf{r}}_l^T \quad \text{and} \quad \dot{\mathbf{W}}_l^{PI} = \eta^{PI} (\mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{PI} \mathbf{u}_l^I) \mathbf{u}_l^{IT}. \quad (11)$$

The following theorem on dendritic error learning tells that the plasticity in the lateral feedback loop leads to an appropriate error representation in the apical dendrites of pyramidal neurons.

Theorem 2

(real-time Dendritic Error Learning, rt-DeEL)

Consider a cortical microcircuit composed of pyramidal and interneurons, as illustrated in [Fig. 5a](#), with more interneurons in layer ('cortical area') l than pyramidal neurons in layer $l+1$, and with adaptable pyramidal-to-inhibitory weights $\mathbf{W}_l^{IP}$ within the same layer (that are nudged through top-down weights $\mathbf{B}_l^{IP}$, see Methods, Sect. E). Then, for suitable top-down nudging, learning rates, and initial conditions, the inhibitory-to-pyramidal synapses $\mathbf{W}_l^{PI}$ within each layer l ([Eq. 11](#)) evolve such that the lateral feedback circuit aligns with the bottom-up-top-down feedback circuit,

$$\mathbf{W}_l^{PI} \mathbf{W}_l^{IP} = \mathbf{B}_l \mathbf{W}_{l+1}. \quad (12)$$

After this horizontal-to-vertical circuit alignment, the apical voltages $\bar{e}_l^A = \mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{PI} \mathbf{u}_l^I$ of the layer- l pyramidal neurons ([Eq. 10](#)) represent the 'B-backpropagated' errors $\bar{e}_l^A = \mathbf{B}_l \bar{e}_{l+1}$. When modulated by the postsynaptic rate derivatives, $\bar{r}_l' = \rho'(\bar{u}_l)$, the apical voltages yield the appropriate error signals

$$\bar{e}_l = \mathbf{u}_l - \mathbf{W}_l \bar{\mathbf{r}}_{l-1} = \bar{\mathbf{r}}_l' \cdot \bar{e}_l^A = \bar{\mathbf{r}}_l' \cdot \mathbf{B}_l \bar{e}_{l+1} \quad (13)$$

for learning the forward weights $\mathbf{W}_l$ according to $\dot{\mathbf{W}}_l \propto \bar{\mathbf{e}}_l \bar{\mathbf{r}}_{l-1}^T$ ([Eq. 8](#)).

The backprojecting weights can also be learned by a local real-time learning rule to become transpose of the forward weights, $\mathbf{B}_l = \mathbf{W}_{l+1}^T$ (Max *et al.*, 2022). In this case, the error signals $\bar{\mathbf{e}}_l$ learned in the apical dendrites according to the above Theorem ([Eq. 13](#)) represent the gradient errors $\bar{\mathbf{e}}$ appearing in the real-time dendritic error propagation (rt-DeEP, Theorem 1). There, the errors $\bar{\mathbf{e}}$ drive the gradient plasticity of the general weight matrix $\mathbf{W}$, split up here into the forward weights $\mathbf{W}_l$ to a layer l (for $l = 1, \dots, N$).

Simultaneously learning apical errors and basal signals

Microcircuits following these neuronal and synaptic dynamics are able to learn the classification of hand-written digits from the MNIST dataset while learning the apical signal representation ([Fig. 5b1-2](#)). In this case, feedforward weights $\mathbf{W}_l$ and lateral weights $\mathbf{W}_l^{PI}$ and $\mathbf{W}_l^{IP}$ are all adapted simultaneously. Including the $\dot{\mathbf{W}}_l^{IP}$ -plasticity (by turning on the interneuron nudging from the upper layer, $\beta^I > 0$ in [Eq. 9](#)), greatly speeds up the learning.

With and without $\dot{\mathbf{W}}_l^{IP}$ -plasticity, the lateral feedback via interneurons (with effective weight $\mathbf{W}_l^{IP} \mathbf{W}_l^{PI}$) learns to align with the forward-backward feedback via upper layer pyramidal neurons (with effective weight $\mathbf{B}_l \mathbf{W}_{l+1}$, [Fig. 5b3](#)). The microcircuit extracts the gradient-based errors ([Eq. 13](#)), while the forward weights use these errors to reduce these errors to first minimize the neuron-specific mismatch errors, and eventually the output cost.

Since the apical voltage $\bar{e}_l^A$ appears as a postsynaptic factor in the plasticity rule for the interneurons ($\dot{\mathbf{W}}_l^{PI}$), this I-to-P plasticity can be interpreted as Hebbian plasticity of inhibitory neurons, consistent with earlier suggestions (Vogels *et al.*, 2012; Bannon *et al.*, 2020). The

plasticity $\dot{W}_l^{\text{IP}}$ of the P-to-I synapses, in the same way as the plasticity for the forward synapses $\dot{W}_l$, can be interpreted as learning from the dendritic prediction of somatic activity (Urbanczik & Senn, 2014 [↗](#)).

Crucially, choosing a large enough interneuron population, the simultaneous learning of the lateral microcircuit and the forward network can be accomplished without fine-tuning of parameters. As an instance in case, all weights shared the same learning rate. Such stability bolsters the biophysical plausibility of our NLA framework and improves over the previous, more heuristic approach (Sacramento *et al.*, 2018 [↗](#); Mesnard *et al.*, 2019 [↗](#)). The stability may be related to the nested gradient descent learning according to which somatic and apical mismatch errors in pyramidal neurons, and somatic mismatch errors in inhibitory neurons are minimized.

Finally, since errors are defined at the level of membrane voltages (Eq. 11 [↗](#)), synapses need a mechanism by which they can recover the presynaptic voltage errors from their afferent firing rates. While for threshold-linear transfer functions the backpropagated voltage errors translate into rate errors (SI, Sect. A), more general neuronal nonlinearities must be matched by corresponding synaptic nonlinearities. Pfister *et al.* (2010) [↗](#) have illustrated how spiking neurons can leverage short-term synaptic depression to estimate the membrane potential of their presynaptic partners. Here, we assume a similar mechanism in the context of our rate-based neurons. The monotonically increasing neuronal activation function, $\bar{r}_{l+1} = \rho(u_{l+1})$, can be approximately compensated by a vesicle release probability that monotonically decreases with the low-pass filtered presynaptic rate $\bar{r}_{l+1}$ (see SI, Sect. A and Fig. 6 [↗](#) therein). If properly matched, this leads to a linear relationship between the presynaptic membrane potential u_{l+1} and the postsynaptic voltage contribution.

Discussion

We introduced a least-action principle to neuroscience for deriving the basic laws of the voltage and synaptic dynamics in networks of cortical neurons. The approach is inspired by the corresponding principle in physics where basic laws of motions are derived across the various scales. While in physics the action is defined as time-integral of the kinetic minus potential energy, we define the action as time-integral of instantaneous somato-dendritic mismatch errors across network neurons plus a behavioural error. The ‘kinetics’ of a voltage trajectory only arises because we postulate that the action along a trajectory is minimized with respect to future voltages, not the instantaneous voltage, as would be done in physics. The postulate implies a prospective voltage dynamics that looks ahead in time, together with prospective local errors, in order to minimize the action and hence the somato-dendritic mismatch errors. The prospective errors nudge the firing of pyramidal neurons deep in the brain, so that motor neurons improve the output of the network right in time. A putative behavioural error, encoded in the motor feedback, propagates back through the network and produces prospective corrections of the pyramidal neuron activities that effectively manifest in instantaneous corrections of the motor trajectory. Through this prospective coding, the sensory stream, the deep network activity, and the motor feedback are in sync at any moment in time. We formulated the dynamic synchronization as ‘moving equilibrium hypothesis’, referring to the classical equilibrium point hypothesis for motor control (Feldman & Levin, 2009 [↗](#); Latash, 2010 [↗](#)). More generally, the brain activity formed by the prospective firing of cortical pyramidal neurons is in a moving equilibrium while converting sensory input streams into motor outputs, consistent with prospective sensory processing in the human cortex (Blom *et al.*, 2020 [↗](#)).

Because the neuronal dynamics derived from the global NLA principle is in a moving equilibrium, the prospective dendritic errors that globally correct the output trajectory are also suited to instruct local synaptic plasticity in the dendrites. In fact, working down the apical errors by

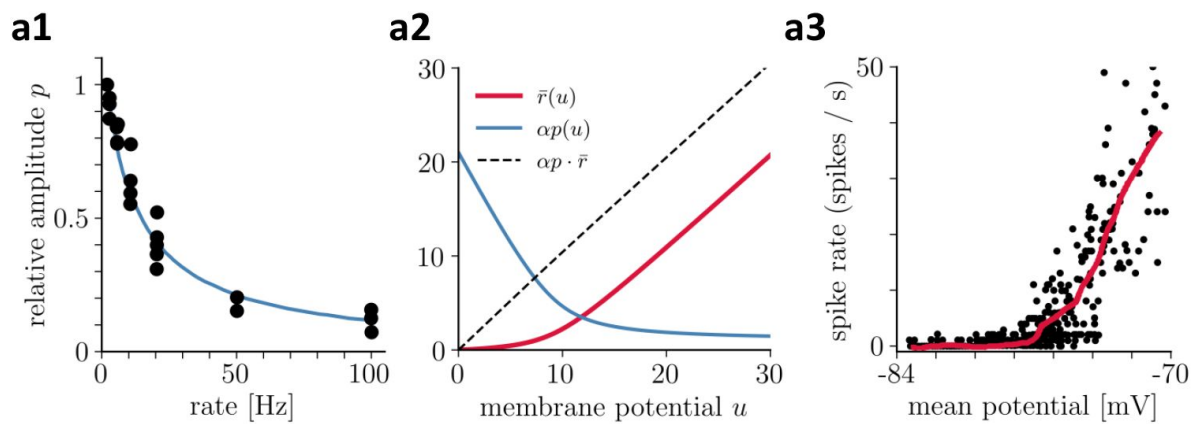


Figure 6.

Recovering presynaptic potentials through short term depression.

(a1) Relative voltage response of a depressing cortical synapse (recreated from Abbott *et al.*, 1997), identified as synaptic release probability p . **(a2)** The product of the low-pass filtered presynaptic firing rate $\bar{r}(u)$ times the synaptic release probability $p(\bar{r})$ is proportional to the presynaptic membrane potential, $p(\bar{r}) \bar{r} \propto u$. **(a3)** Average in vivo firing rate of a neuron in the visual cortex as a function of the somatic membrane potential (recreated from Anderson *et al.*, 2000), which can be qualitatively identified as the stationary rate $\bar{r}(u)$ derived in Eq. 43.

adapting the sensory driven synapses on the basal dendrites reduces the global output errors in real-time. The apical errors are extracted from the top-down feedback via lateral ‘inhibition’ that tries to cancel the top-down signal. This top-down feedback includes activity from a putative erroneous motor output that was not foreseen by the local inhibition and thus survives as a local apical error. Given the prospective coding of the pyramidal neurons, the dendritic errors are also prospective and thus able to induce the correct error-minimizing plasticity online, while stimuli and targets continuously change.

The NLA principle as a bottom-up theory from neurons to behaviour

To show that the NLA principle offers a viable program for a formalization of neuroscience following the example of physics, we exemplified its ramifications into dendritic computation, cortical microcircuits, synaptic plasticity, motor control, and sensory-based decision making. The crucial point of our axiomatization is that it connects the local neuronal errors to the global behavioural errors right in the formulation of the principle, eventually leading to local gradient-based plasticity rules. Because the formulation builds upon computations that can be realized in single neurons and dendrites to produce a behavioural output, the NLA principle can be seen as a bottom-up theory of behaviour. It is articulated in terms of apical and basal dendrites, somatic firing, network connectivity and behavioural outputs that jointly minimize their errors. This contrasts the related free energy principle, for instance, that leads to a top-down theory of behaviour by starting with the statistical, but more universal, notion of a free energy. It postulates that any self-organizing system, that is at a statistical equilibrium with its environment, must minimize its free energy (Friston, 2010 [↗](#); Friston *et al.*, 2022 [↗](#)), and from there works down its way to neurons and dendrites (Bastos *et al.*, 2012 [↗](#); Kiebel & Friston, 2011 [↗](#)).

Starting with a single Lagrangian function that specifies the form of the somato-dendritic prediction errors leaves some freedom for the interpretation and the implementation of the emerging dynamical equations for the voltages. We interpret errors to be represented in the apical dendrites of pyramidal neurons while sensory input targets the basal dendrites, but other dendritic configurations are conceivable (Mikulasch, Rudelt, Wibral, *et al.*, 2023) that apply also to non-pyramidal neurons. We have chosen a specific interneuron circuitry to extract our apical errors, but other microcircuits or error representations might also be considered (Keller & Mrosovsky, 2018 [↗](#)). On the other hand, the derived gradient-based synaptic plasticity is tightly linked to the specific form of the somato-dendritic prediction errors expressed in the Lagrangian and its interpretation, making specific predictions for synaptic plasticity (as outlined below). The ‘external’ feedback entering through the cost function offers additional freedom to model behavioural interactions. We considered an explicit time course of a target voltage in motor neurons, for instance imposed by the feedback from muscle spindles that are themselves innervated by a prospective top-down signal to control muscle lengths (Papaioannou & Dimitriou, 2021 [↗](#); Dimitriou, 2022 [↗](#)). But the cost may also link to reinforcement learning and express a delayed reward feedback delivered upon a behavioural decision of an agent acting in a changing environment (Friedrich, Urbanczik, *et al.*, 2011 [↗](#); Friedrich & Senn, 2012 [↗](#)).

A fundamental difficulty arises when the neuronal implementation of the Euler-Lagrange equations requires an additional microcircuit with its own dynamics. This is the case for the suggested microcircuit extracting the local errors. Formally, the representation of the apical feedback errors first needs to be learned before the errors can teach the feedforward synapses on the basal dendrites. We showed that this error learning can itself be formulated as minimizing an apical mismatch energy. What the lateral feedback through interneurons cannot explain away from the top-down feedback remains as apical prediction error. Ideally, while the network synapses targetting the basal tree are performing gradient descent on the global cost, the microcircuit synapses involved in the lateral feedback are performing gradient descent on local error functions, both at any moment in time. The simulations show that this intertwined system

can in fact learn simultaneously with a common learning rate that is properly tuned. The cortical model network of inter- and pyramidal neurons learned to classify handwritten digits on the fly, with 10 digit samples presented per second. Yet, the overall learning is more robust if the error learning in the apical dendrites operates in phases without output teaching but with corresponding sensory activity, as may arise during sleep (see e.g. [Deporrois et al., 2022](#); [Deporrois et al., 2023](#)).

The NLA principle integrates classical theories for cortical processing and learning

The prospective variational principle introduced with the NLA allows for integrating previous ideas on formalizing the processing and learning in cortical networks. Four such classical lines of theories come together. (i) The first line refers to the use of an *energy function* to jointly infer the neuronal dynamics and synaptic plasticity, originally formulated for discrete-time networks ([Hopfield, 1982](#); [Ackley et al., 1985](#)), and recently extended to continuous-time networks ([Scellier & Bengio, 2017](#)). (ii) The second line refers to understanding *error-backpropagation* in the brain ([Rumelhart et al., 1986](#); [Xie & Seung, 2003](#); [Whittington & Bogacz, 2017](#); [Whittington & Bogacz, 2019](#); [Lillicrap, Santoro, et al., 2020](#)). (iii) The third line refers to *dendritic computation* and the use of dendritic compartmentalization for various functions such as nonlinear processing ([Schiess et al., 2016](#); [Poirazi & Papoutsis, 2020](#)) and deep learning ([Guerquiev et al., 2017](#); [Sacramento et al., 2018](#); [Haider et al., 2021](#)). (iv) The fourth line refers to *predictive coding* ([Rao & Ballard, 1999](#)) and active inference ([Pezzulo et al., 2022](#)) to improve the sensory representation and motor output, respectively.

1. With regard to *energy functions*, the NLA principle adds a variational approach to characterize continuous-time neuronal trajectories and plasticity. Variational approaches are studied in the context of optimal control theory where a cost integral is minimized across time, constrained to some network dynamics ([Todorov & Jordan, 2002](#); [Meulemans, Farinha, et al., 2021](#)). The NLA represents a unifying notion that allows to infer both, the network dynamics and its optimal control from a single Lagrangian. The error we derive represents prospective control variables that are applied to the voltages of each network neurons so that they push the output neurons towards their target trajectory. The full expression power of this control theoretic framework has yet to be proven when it is extended to genuine temporal processing that includes longer time constants, for instance inherent in a slow threshold adaptation ([Bellec et al., 2020](#)). The NLA principle can also treat the case of strong feedback studied so far in relaxation networks only ([Meulemans, Zucchet, et al., 2022](#); [Song et al., 2024](#)). Our rt-DeEP Theorem makes a statement for real-time gradient descent learning while the network is in a moving equilibrium, linking to motor learning in the presence of perturbing force fields ([Herzfeld et al., 2014](#)) or perturbing visual inputs ([Dimitriou, 2016](#)).
2. With regard to *error-backpropagation*, the NLA principle infers a local error that originates from the error in the output neurons. In the current version, the NLA relies on feedback alignment ([Lillicrap, Cownden, et al., 2016](#)) to obtain a local apical errors without postulating plasticity in the top-down synapses. Other works have explored learning the feedback weights ([Akrouit et al., 2019](#); [Kunin et al., 2020](#)), notably within a phase-less real-time learning framework as considered here ([Max et al., 2022](#)). It would be promising to combine these ideas to obtain a fully plastic microcircuit that adjusts from scratch to various real-time learning tasks.

3. With regard to *dendritic computation*, the NLA principle extends the idea of dendritic error representation (Sacramento *et al.*, 2018 [↗](#); Mesnard *et al.*, 2019 [↗](#)) by the prospective coding of both errors and firing rates (Fig. 2 [↗](#)). As a consequence, the various dendritic delays are compensated and synaptic plasticity can operate at any moment, without need to wait for network relaxations (Scellier & Bengio, 2017 [↗](#); Song *et al.*, 2024 [↗](#)). In the present framework, the input rates are low-pass filtered by variable input time constants (τ_{in}) before the induced voltages are instantaneously processed in the network. While this offers the possibilities for a temporal processing of the inputs, step-changes in the input rates cannot instantaneously propagate (as made possible in Haider *et al.*, 2021 [↗](#)). The dendritic error representation has also been applied to spike-based learning in recurrent networks (Mikulasch, Rudelt & Priesemann, 2021 [↗](#)).
4. With regard to the principle of *predictive coding* (Rao & Ballard, 1999 [↗](#); Pezzulo *et al.*, 2022 [↗](#)), the NLA offers a form of predictive error-coding in the apical tree of sensory pyramidal neurons that shapes the sensory representation. Both the predictive coding and the NLA principle imply an error-based adaptation of the lower and higher cortical representations. However, while predictive coding is based on propagating errors, the NLA principle directly propagates the error-corrected representations. Based on the backpropagated activities, it extracts in each area the prospective errors to reshape the local representations and induce synaptic plasticity. Otherwise, the freedom in defining the cost function in the NLA, and the inclusion of active inference in predictive coding (Pezzulo *et al.*, 2022 [↗](#)), makes the two frameworks equally powerful. Both frameworks can explain the learning from motor and sensory prediction errors. In fact, our example of motor control minimizes the proprioceptive error formed by the target muscle lengths minus the effective muscle lengths. Active inference likewise minimizes the mismatch between an internal motor target and the proprioceptive inputs caused by the own actions. In turn, the cost function in the NLA principle may also capture the sensory prediction errors, where the ground truth lies in the external inputs that are learned to be matched by the sensory expectation. Hence, while being functionally equivalent, the neuronal interpretations are different: predictive coding focuses on the propagation of errors using the sensory- and motor-representations as local auxiliary quantities, and the NLA principle directly integrates errors in ‘auxiliary dendrites’ to propagate sensory or motor representations.

The NLA integrates and predicts features of synapses, dendrites and circuits

Motivated by the predictive power of the least-action principle in physics, we ask about experimental confirmation and predictions of the NLA principle. Given its axiomatic approach, it appears astonishing to find various preliminary matches at the dendritic, somatic, interneuron, synaptic and even behavioural level. Some of these are:

1. the prospective coding of pyramidal neuron firing (Köndgen *et al.*, 2008);
2. the prospective processing of apical signals while propagating to the soma (Ulrich, 2002 [↗](#));
3. the basal synaptic plasticity on pyramidal neurons and synaptic plasticity on interneurons, driven by the postsynaptic activity that is ‘unexplained’ by the distal dendritic voltage (Urbanczik & Senn, 2014 [↗](#)) – while partly consistent with spike-timing dependent plasticity (Sjöström *et al.*, 2001 [↗](#); Spicher *et al.*, 2017 [↗](#)), the postulated dendritic voltage-dependence of the plasticity rules still awaits an experimental inspection;
4. the Hebbian homeostatic plasticity of interneurons targeting the apical dendritic tree of pyramidal neurons (Bannon *et al.*, 2020 [↗](#));
5. the short-term synaptic depression at top-down synapses targetting inhibitory neurons and apical dendrites (akin to Abbott *et al.*, 1997 [↗](#), but with a faster recovery time constant) that invert the presynaptic activation function (see also Pfister *et al.*, 2010 [↗](#));

6. the modulation of the apical contribution to the somatic voltage by the slope of the somatic activation function (for instance by downregulating apical NMDA receptors with increasing rate of backpropagating action potentials, Theis *et al.*, 2018 [DOI](#)); and
7. the involvement of muscle spindles in the prospective encoding of motor errors during motor learning, with the γ -innervation setting the target length of the spindles (Dimitriou, 2016 [DOI](#); Papaioannou & Dimitriou, 2021 [DOI](#); Vargas *et al.*, 2023 [DOI](#)).

More experimental and theoretical work is required to substantiate these links and test specific predictions, such as the apical error representation in cortical pyramidal neurons.

Overall, our approach adapts the least-action principle from physics to be applied to neuroscience, and couples it with a normative perspective on the prospective processing of neurons and synapses in global cortical networks and local microcircuits. Given its physical underpinnings, the approach may inspire the rebuilding of computational principles of cortical neurons and circuits in neuromorphic hardware (Bartolozzi *et al.*, 2022 [DOI](#)). A step in this direction, building on the instantaneous computational capabilities with slowly integrating neurons, has been done in Haider *et al.*, 2021. Given its aspiration for a theoretical framework in neurobiology, a next challenge would be to generalize the NLA principle to spiking neurons (Gerstner & Kistler, 2002 [DOI](#); Brendel *et al.*, 2020 [DOI](#)) with their potential for hardware implementation (Zenke & Ganguli, 2018 [DOI](#); Gölitz *et al.*, 2021 [DOI](#); Cramer *et al.*, 2022 [DOI](#)), to include attentional mechanisms in terms of dendritic gain modulation (Larkum *et al.*, 2004 [DOI](#)) with a putative link to self-attention in artificial intelligence (Vaswani *et al.*, 2017 [DOI](#)), to add second-order errors to cope with certainties (Granier *et al.*, 2023 [DOI](#)), and to incorporate longer temporal processing as, for instance, offered by neuronal adaptation processes (La Camera *et al.*, 2006 [DOI](#)) or realistically modelled dendrites (Chavlis & Poirazi, 2021 [DOI](#)).

Methods

A. Euler-Lagrange equations as inverse low-pass filters

The theory is based on the lookahead of neuronal quantities. In general, the lookahead of a trajectory $x(t)$ is defined via lookahead operator applied to x ,

$$\left(1 + \tau \frac{d}{dt}\right)x = \bar{x} + \tau \dot{\bar{x}}. \quad (14)$$

The lookahead operator is the inverse of the low-pass filter operator denoted by a bar,

$$\bar{x}(t) = \frac{1}{\tau} \int_{-\infty}^t x(t') e^{-\frac{t-t'}{\tau}} dt'. \quad (15)$$

This low-pass filtering can also be characterized by the differential equation $\tau \dot{\bar{x}}(t) = -\bar{x}(t) + x(t)$, see SI, Sect.B. Hence, applying the low-pass filtering to x and then the lookahead operator $\left(1 + \tau \frac{d}{dt}\right)$ to $\bar{x}(t)$, and using the Leibnitz rule for differentiating an integral, we calculate $\left(1 + \tau \frac{d}{dt}\right)\bar{x}(t) = x(t)$. In turn, applying first the lookahead, and then the low-pass filtering, also yields the original trace back, $\overline{\left(1 + \tau \frac{d}{dt}\right)x} = \bar{\bar{x}} + \tau \dot{\bar{\bar{x}}} = x$.

We consider an arbitrary network architecture with network neurons that are recurrently connected and that receive external input through an overall weight matrix $\mathbf{W} = (\mathbf{W}_{\text{in}}, \mathbf{W}_{\text{net}})$, aggregated column-wise. The instantaneous presynaptic firing rates are $\mathbf{r} = (\mathbf{r}_{\text{in}}, \mathbf{r}_{\text{net}})^T$, interpreted as a single column vector. A subset of network neurons are output neurons, $\mathcal{O} \subseteq \mathcal{N}$, for which target voltages $\mathbf{u}^*$ may be imposed. Rates and voltages may change in time t . Network neurons are

assigned a voltage $\mathbf{u}$, generating the low-pass filtered rate $\bar{\mathbf{r}}_{\text{net}} = \rho(\mathbf{u})$, and a low-pass filtered error $\bar{\mathbf{e}} = \mathbf{u} - \mathbf{W}\bar{\mathbf{r}}$. We further define output errors $\bar{\mathbf{e}}_o^* = \mathbf{u}_o^* - \mathbf{u}_o$ for $o \in \mathcal{O}$, and $\bar{\mathbf{e}}_i^* = 0$ for non-output neurons $i \in \mathcal{N} \setminus \mathcal{O}$. With this, the Lagrangian from Eq. 3 takes the form

$$L = \frac{1}{2} \|\bar{\mathbf{e}}\|^2 + \frac{\beta}{2} \|\bar{\mathbf{e}}^*\|^2. \quad (16)$$

We next use that $\mathbf{u} = \tilde{\mathbf{u}} - \tau \dot{\tilde{\mathbf{u}}}$, with the $\sim$ operator defined in Eq. 4, to write out the Lagrangian L in the canonical coordinates $(\tilde{\mathbf{u}}, \dot{\tilde{\mathbf{u}}})$ as (see also Eq. 3)

$$L = \frac{1}{2} \sum_{i \in \mathcal{N}} \left[\dot{\tilde{u}}_i - \tau \ddot{\tilde{u}}_i - \sum_j W_{ij} \rho(\tilde{u}_j - \tau \dot{\tilde{u}}_j) \right]^2 + \frac{\beta}{2} \sum_{o \in \mathcal{O}} \left[u_o^* - (\tilde{u}_o - \tau \dot{\tilde{u}}_o) \right]^2. \quad (17)$$

The neuronal dynamics is derived from requiring a stationary action (see Eq. 5), which is generally solved by the Euler-Lagrange equations $\frac{\partial L}{\partial \tilde{u}_i} - \frac{d}{dt} \frac{\partial L}{\partial \dot{\tilde{u}}_i} = 0$ (see Eq. 6). Because $\tilde{\mathbf{u}}$ only arises in L in the compound $\tilde{\mathbf{u}} - \tau \dot{\tilde{\mathbf{u}}}$, the derivative of L with respect to $\tilde{\mathbf{u}}$ is identical to the derivative with respect to $\tau \dot{\tilde{\mathbf{u}}}$,

$$\frac{\partial L}{\partial \dot{\tilde{u}}_i} = -\tau \frac{\partial L}{\partial \tilde{u}_i}. \quad (18)$$

Using the lookahead operator Eq. 14, the Euler-Lagrange equations can then be rewritten as

$$\frac{\partial L}{\partial \tilde{u}_i} + \tau \frac{d}{dt} \frac{\partial L}{\partial \tilde{u}_i} = \left(1 + \tau \frac{d}{dt} \right) \frac{\partial L}{\partial \tilde{u}_i} = 0. \quad (19)$$

Since $L(\tilde{\mathbf{u}}, \dot{\tilde{\mathbf{u}}}) = L(\mathbf{u})$ and $\mathbf{u} = \tilde{\mathbf{u}} - \tau \dot{\tilde{\mathbf{u}}}$, the derivative of L with respect to $\tilde{\mathbf{u}}$ is the same as the derivative of L with respect to $\frac{\partial L}{\partial \tilde{u}_i} = \frac{\partial L}{\partial u_i}$. Plugging this into Eq. 19, the Euler-Lagrange equations become a function of $\mathbf{u}$ and $\dot{\mathbf{u}}$,

$$\left(1 + \tau \frac{d}{dt} \right) \frac{\partial L}{\partial u_i} = 0. \quad (20)$$

Notice that, if we would have directly calculated $\frac{\partial L}{\partial \tilde{u}_i} - \frac{d}{dt} \frac{\partial L}{\partial \dot{\tilde{u}}_i} = 0$, the second-order time derivative $\ddot{\tilde{u}}_i$ of the discounted future voltage would be absorbed in a first-order time derivative of the voltage. The reason is that $\dot{\tilde{u}}_i - \tau \ddot{\tilde{u}}_i = \dot{u}_i$, and $\ddot{\tilde{u}}_i$ only arises in this combination because the Lagrangian $L = L(\mathbf{u})$ is only a function of $\mathbf{u}$ and not of $\tilde{\mathbf{u}}$. Hence, the acceleration term $\ddot{\tilde{u}}_i$ disappears, while a voltage derivative $\dot{u}_i$ appears.

The solution of this differential equation Eq. 20 is $\frac{\partial L}{\partial u_i} = c_i e^{-\frac{t-t_0}{\tau}}$, and hence any trajectory $(\tilde{u}_i, \dot{\tilde{u}}_i)$ which satisfy the Euler-Lagrange equations will hence cause $\frac{\partial L}{\partial u_i}$ to converge to zero with a characteristic time scale of τ . Since we require that the initialisation is at $t_0 = -\infty$, we conclude that $\frac{\partial L}{\partial u_i} = 0$, as required in the rt-DeEP Theorem.

B. Deriving the network dynamics from the Euler-Lagrange equations

We now derive the equations of motion from the Euler-Lagrange equations. Noticing that $\mathbf{u}$ enters in $\bar{\mathbf{e}} = \mathbf{u} - \mathbf{W}\bar{\mathbf{r}}$ twice, directly and through $\bar{\mathbf{r}}_{\text{net}} = \rho(\mathbf{u})$, and once in the output error $\bar{\mathbf{e}}^*$, we calculate from Eq. 16, using $\bar{\mathbf{r}}(\mathbf{u}) = (\bar{\mathbf{r}}_{\text{in}}, \rho(\mathbf{u}))^T$ and $\mathbf{W} = (\mathbf{W}_{\text{in}}, \mathbf{W}_{\text{net}})$,

$$\frac{\partial L}{\partial \mathbf{u}} = \bar{\mathbf{e}} - \bar{\boldsymbol{\epsilon}} - \beta \bar{\mathbf{e}}^*, \quad \text{with } \bar{\boldsymbol{\epsilon}} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}}. \quad (21)$$

Remember that for non-output neurons i no target exist, and for those we set $\bar{\mathbf{e}}_i^* = 0$. Next, we apply the lookahead operator to this expression, as required by the Euler-Lagrange equations Eq. 19. In general $(1 + \tau \frac{d}{dt})\bar{\mathbf{x}} = \bar{\mathbf{x}} + \tau \dot{\bar{\mathbf{x}}} = \mathbf{x}$, and we set for $\bar{\mathbf{x}}$ the expression on the right-hand side of Eq. 21, $\bar{\mathbf{x}} = \bar{\mathbf{e}} - \bar{\boldsymbol{\epsilon}} - \beta \bar{\mathbf{e}}^*$, which at the same time is $\bar{\mathbf{x}} = \frac{\partial L}{\partial \mathbf{u}}$. Hence, the Euler-Lagrange equations in the form of Eq. 20, $(1 + \tau \frac{d}{dt})\bar{\mathbf{x}} = 0$, translate into

$$(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = 0 \iff \mathbf{e} - \boldsymbol{\epsilon} - \beta \mathbf{e}^* = 0 \iff \tau \dot{\mathbf{u}} = -\mathbf{u} + \mathbf{W}\mathbf{r} + \mathbf{e}. \quad (22)$$

To move from the middle to the last equality we replaced $\mathbf{e}$ by $\mathbf{e} = (1 + \tau \frac{d}{dt})\bar{\mathbf{e}} = \mathbf{u} + \tau \dot{\mathbf{u}} - \mathbf{W}\mathbf{r}$. In the last equality we interpret $\mathbf{e}$ as the sum of the two errors, $\mathbf{e} = \boldsymbol{\epsilon} + \beta \mathbf{e}^*$, again using the middle equality. This proves Eq. 7.

Notice that the differential equation $\tau \dot{\mathbf{u}} = \dots$ in Eq. 22 represents an implicit ordinary differential equation as on the right-hand side not only $\mathbf{u}$, but also $\dot{\mathbf{u}}$ appears (in $\mathbf{r}$ and $\mathbf{e}$). The uniqueness of the solution $\mathbf{u}(t)$ for a given initial condition is only guaranteed if it can be converted into an explicit ordinary differential equation (see Sect. C).

In taking the temporal derivative we assumed small learning rates such that terms including $\dot{W}_{ij}$ can be neglected. The derived dynamics for the membrane potential of a neuron u_i in Eq. 22 show the usual leaky behavior of biological neurons. However, both presynaptic rates $\bar{r}_i$ and prediction errors $\bar{e}_i$ enter the equation of motion with lookaheads, i.e., they are advanced ($r_i = \bar{r}_i + \tau \dot{\bar{r}}_i$ and $e_i = \bar{e}_i + \tau \dot{\bar{e}}_i$), cancelling the low-pass filtering. Since $\dot{r}_i = \rho'(u_i) \dot{u}_i$, the rate and error, r_i and e_i , can also be seen as nonlinear extrapolations from the voltage and its derivative into the future.

The instantaneous transmission of information throughout the network at the level of the voltages can now be seen by low-pass filtering Eq. 22 with initialization far back in the past,

$$\mathbf{u} = \overline{\mathbf{u} + \tau \dot{\mathbf{u}}} = \overline{\mathbf{W}\mathbf{r} + \mathbf{e}} = \mathbf{W}\bar{\mathbf{r}}(\mathbf{u}) + \bar{\mathbf{e}}, \quad (23)$$

with column vector $\bar{\mathbf{r}}(\mathbf{u}) = (\bar{\mathbf{r}}_{\text{in}}, \rho(\mathbf{u}))^T$ and $\bar{\mathbf{e}} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^*$. Hence, solving the voltage dynamics for $\mathbf{u}$ (Eq. 7a), with apical voltage $\mathbf{e} = \bar{\mathbf{e}} + \tau \dot{\bar{\mathbf{e}}}$ derived from Eq. 7b, yields the somatic voltage $\mathbf{u}$ satisfying the self-consistency equation (Eq. 23) at any time. In other words, $\mathbf{u}$ and $\bar{\mathbf{e}}$ ‘propagate instantaneously’.

C. Deriving the error backpropagation formula

For clarity, we derive the error backpropagation algorithm for layered networks here. These can be seen as a special case of a general network with membrane potentials $\mathbf{u}$ and all-to-all weight matrix $\mathbf{W}$ (as introduced in appendix I), where the membrane potentials decompose into layerwise membrane potential vectors $\mathbf{u}_l$ and the weight matrix into according block diagonal matrices $\mathbf{W}_l$ (with $\mathbf{W}_l$ being the weights that project into layer l).

Assuming a network with N layers, by low-pass filtering the equations of motion we get

$$\mathbf{u}_l = \mathbf{W}_l \bar{\mathbf{r}}_{l-1} + \bar{\mathbf{e}}_l, \quad (24)$$

for all $l \in 1, \dots, N$, with the output error $\bar{\mathbf{e}}_o$ of the general recurrent network becoming the error in the last layer, that itself is the target error, $\bar{\mathbf{e}}_o = \bar{\mathbf{e}}_N = \beta \bar{\mathbf{e}}^* = \beta (\mathbf{u}_N^* - \mathbf{u}_N)$. The error $\bar{\mathbf{e}} = \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^*$, that we obtain from the general dynamics with $\bar{\mathbf{e}} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}}$, see Eq. 21 and Eq. 22, translates to an iterative formula for the error at the current layer l given the error at the downstream layer $l+1$, inherited from the drive $\bar{\mathbf{r}}_l = \rho(\mathbf{u}_l)$ of that downstream layer via $\mathbf{W}_{l+1}$,

$$\bar{\mathbf{e}}_l = \bar{\mathbf{r}}_l' \cdot \mathbf{W}_{l+1}^T \bar{\mathbf{e}}_{l+1} \quad \text{for } l < N. \quad (25)$$

and $\bar{\mathbf{e}}_N = \beta \bar{\mathbf{e}}^*$ for the output layer. The learning rule that reduces $\bar{\mathbf{e}}_l$ by gradient descent is proportional to this error and the presynaptic rate, as stated by Theorem 1, is

$$\dot{\mathbf{W}}_l \propto (\mathbf{u}_l - \mathbf{W}_l \bar{\mathbf{r}}_{l-1}) \bar{\mathbf{r}}_{l-1}^T = \bar{\mathbf{e}}_l \bar{\mathbf{r}}_{l-1}^T, \quad (26)$$

for $l = 1 \dots N$. Eq. 25 and Eq. 26 together take the form of the error backpropagation algorithm, where an output error is iteratively propagated through the network and used to adjust the weights in order to reduce the output cost C . From this, it is easy to see that without output nudging (i.e., $\beta = 0$), the output error vanishes and consequently all other prediction errors vanish as well, $\bar{\mathbf{e}}_l = \mathbf{u}_l - \mathbf{W}_l \bar{\mathbf{r}}_l = 0$ for all $l \leq N$. This also means that in the absence of nudging, no weight updates are performed by the plasticity rule.

The learning rule for arbitrary connectivities is obtained in the same way by dropping the layerwise notation. In this case, low-pass filtering the equations of motion yields $\mathbf{u} = \mathbf{W} \bar{\mathbf{r}} + \bar{\mathbf{e}}$, as calculated in Eq. 23, and the low-pass filtered error $\bar{\mathbf{e}} = \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^* = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^*$, as inferred from Eqs 21 and 22. Hence, the plasticity rule in general reads

$$\dot{\mathbf{W}} \propto (\mathbf{u} - \mathbf{W} \bar{\mathbf{r}}) \bar{\mathbf{r}}^T = \bar{\mathbf{e}} \bar{\mathbf{r}}^T, \quad \text{with } \bar{\mathbf{e}} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^*. \quad (27)$$

D. Proving Theorem 1 (rt-DeEP)

The implicit assumption in Theorem 1 is that $\dot{\mathbf{u}}$ exists in the distributional sense for $t > -\infty$, which is the case for delta-functions in $\mathbf{r}_{\text{in}}$ and step-functions in $\mathbf{u}^*$. Both parts (i) and (ii) of the Theorem are based on the requirement of stationary action $\delta A = 0$, and hence on $\mathbf{u}$ satisfying the Euler-Lagrange equations in the form of Eq. 22, $(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = 0$. From the solution $\frac{\partial L}{\partial u_i} = c e^{-\frac{t-t_0}{\tau}}$ we conclude that for initialization at $t_0 = -\infty$ we have $\frac{\partial L}{\partial \mathbf{u}} = 0$ for all t . It is the latter stronger condition that we require in the proof. With this, the main ingredient of the proof

follows is the mathematical argument of Scellier & Bengio, 2017 [\[1\]](#), according to which the total and partial derivative of L with respect to $\mathbf{W}$ are identical, and this in our case is true for any time t ,

$$\frac{dL}{d\mathbf{W}} = \frac{\partial L}{\partial \mathbf{u}} \frac{d\mathbf{u}}{d\mathbf{W}} + \frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}}, \quad (28)$$

For convenience we considered $\frac{\partial L}{\partial \mathbf{u}}$ to be a column vector, deviating from the standard notations (see tutorial end of sec:Integration). Analogously to Eq. 28 [\[1\]](#), we infer $\frac{dL}{d\beta} = \frac{\partial L}{\partial \beta}$. Reading Eq. 28 [\[1\]](#) from the right to the left, we conclude that the learning rule $\dot{\mathbf{W}} \propto -\frac{\partial L}{\partial \mathbf{W}} = \bar{\mathbf{e}} \bar{\mathbf{r}}^T$ for all $\beta > 0$ is gradient descent on L , i.e. $\dot{\mathbf{W}} \propto -\frac{dL}{d\mathbf{W}}$. This total derivative of L can be analyzed for large and small β .

1. We show that in the limit of large β , $\dot{\mathbf{W}}$ becomes gradient descent on the mismatch energy $E^M = \frac{1}{2} \|\bar{\mathbf{e}}\|^2$. For this we first show that there is a solution of the self-consistency equation $\mathbf{u} = \mathbf{F}(\mathbf{u}) = \mathbf{W}\bar{\mathbf{r}} + \bar{\mathbf{r}}'_{\text{net}} \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^*$ that is uniformly bounded for all t and β . For this we assume that the transfer function $\rho(u)$ is non-negative, monotonically increasing and bounded, that its derivative $\rho'(u)$ is bounded too, and that the input rates $\mathbf{r}_{\text{in}}$ and the target potentials $\mathbf{u}_o^*$ are also uniformly bounded. To show that under these conditions we always find an uniformly bounded solution $\mathbf{u}(t)$, we first consider the case where the output voltages are clamped to the target, $\mathbf{u}_o = \mathbf{u}_o^*$ such that $\bar{\mathbf{e}}^* = 0$. For simplicity we assume that $\rho'(u) = 0$ for $|u| \geq c_0$. For voltages $\mathbf{u}$ with $\mathbf{u}_i \leq c_0$ the recurrent input current $\mathbf{W}\bar{\mathbf{r}}$ is bounded, say $|(\mathbf{W}\bar{\mathbf{r}})_j| \leq c_1$ for some $c_1 > c_0$. When including the error term $\bar{\mathbf{r}}'_{\text{net}} \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}}$, the total current still remains uniformly bounded, say $|\mathbf{F}(\mathbf{u})_j| \leq c_2$ for all $\mathbf{u}$ with $\mathbf{u}_i \leq c_0$. Because for larger voltages $\mathbf{u}_i > c_0$ the error term vanishes due to a vanishing derivative $\rho'(\mathbf{u}_i) = 0$, the mapping $\mathbf{F}(\mathbf{u})$ maps the c_2 -box $\mathbf{u}$ (for which $|\mathbf{u}_i| \leq c_2$) onto itself. Brouwer's fixed point theorem then tells us that there is a fixed point $\mathbf{u} = \mathbf{F}(\mathbf{u})$ within the c_2 -box. The theorem requires the continuity of $\mathbf{F}$, and this is assured if the neuronal transfer function $\bar{r} = \rho(u)$ is continuous.

We next relax the voltages of the output neurons from their clamped stage, $\mathbf{u}_o = \mathbf{u}_o^*$. Remember that these voltages satisfy $\mathbf{u}_o = (\mathbf{W}\bar{\mathbf{r}} + \bar{\mathbf{r}}'_{\text{net}} \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}^*)_o = \mathbf{F}(\mathbf{u})_o$ at any time t . We determine the correction term $\beta \bar{\mathbf{e}}_o^*$ such that in the limit $\beta \rightarrow \infty$ we get $\mathbf{u}_o = \mathbf{F}(\mathbf{u})_o = \mathbf{u}_o^*$. The correction remains finite, and in the limit must be equal to $\lim_{\beta \rightarrow \infty} \beta \bar{\mathbf{e}}_o^* = \mathbf{u}_o^* - (\mathbf{W}\bar{\mathbf{r}} + \bar{\mathbf{r}}'_{\text{net}} \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}})_o$. For arbitrary large nudging strength β , the output voltage $\mathbf{u}_o$ deviates arbitrary little from the target voltage, $\mathbf{u}_o = \mathbf{u}_o^* + o(1/\beta)$, with target error $\bar{\mathbf{e}}_o^* = \frac{1}{\beta} (\mathbf{u} - \mathbf{W}\bar{\mathbf{r}} - \bar{\mathbf{r}}'_{\text{net}} \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}})_o$ shrinking like c_2/β . Likewise, also for non-output neurons i , the self-consistency solution $\mathbf{u}_i = \mathbf{F}(\mathbf{u})_i$ deviates arbitrarily little from the solution of the clamped state. To ensure the smooth drift of the fixed point while $1/\beta$ deviates from 0 we require that the Jacobian of $\mathbf{F}$ at the fixed point is invertible.

Because the output $\bar{\mathbf{e}}_o^*$ shrinks with $1/\beta$, the cost shrinks quadratically with increasing nudging strength, $C = \frac{1}{2} \|\bar{\mathbf{e}}^*\|^2 = o(\frac{1}{\beta^2})$, and hence the cost term $\frac{\beta}{2} \|\bar{\mathbf{e}}^*\|^2$ that enters in $L = E^M + \frac{\beta}{2} \|\bar{\mathbf{e}}^*\|^2$ vanishes in the limit $\beta \rightarrow \infty$. In this large β limit, where $\bar{\mathbf{e}}_o^* = 0$ and hence the outputs are clamped, $\mathbf{u}_o = \mathbf{u}_o^*$, the Lagrangian reduces to the mismatch energy, $L = E^M$. Along the least-action trajectories we therefore get $\dot{\mathbf{W}} \propto -\frac{\partial L}{\partial \mathbf{W}} = -\frac{dL}{d\mathbf{W}} = -\frac{dE^M}{d\mathbf{W}}$. The first equality uses Eq. 28, and the second uses $L = E^M$ just derived for $\beta = \infty$. This is statement (i) of Theorem 1. In the case of successful learning, $E^M = 0$, we also conclude that the cost vanishes, $C = 0$. This is the case because $E^M = 0$ implies $E^M = 0$ for all output neurons o . Since $E_o^M = \frac{1}{2} \bar{\mathbf{e}}_o^2 = \frac{1}{2} (\bar{\mathbf{r}}'_{\text{net}} \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}} + \beta \bar{\mathbf{e}}_o^*)^2$, we conclude that $\bar{\mathbf{e}}_o = 0$, and if the output neurons do not feed back to the network (which we can assume without loss of generality), we conclude that $\bar{\mathbf{e}}_o^* = 0$.

2. To consider the case of small β , we use that the cost C can be expressed as $C = \frac{\partial L}{\partial \beta}$. This is a direct consequence of how C enters in $L = \frac{1}{2} \|\bar{\mathbf{e}}\|^2 + \frac{\beta}{2} C$, see Eq. 16 and Scellier & Bengio, 2017. We now put this together with Eq. 28 and the finding that $\frac{\partial L}{\partial \beta} = \frac{dL}{d\beta}$.

Since for the Lipschitz continuous function L in $\mathbf{u}$, $\mathbf{W}$ and β (L is even smooth in these arguments), the total derivatives interchange (which is a consequence of the Moore-Osgood theorem applied to the limits of the difference quotients), we then get at any t ,

$$\frac{dC}{d\mathbf{W}} = \frac{d}{d\mathbf{W}} \frac{\partial L}{\partial \beta} = \frac{d}{d\mathbf{W}} \frac{dL}{d\beta} = \frac{d}{d\beta} \frac{dL}{d\mathbf{W}} = \frac{d}{d\beta} \frac{\partial L}{\partial \mathbf{W}} = -\frac{d}{d\beta} \bar{\mathbf{e}} \bar{\mathbf{r}}^T. \quad (29)$$

The last expression is calculated from the specific form of the Lagrangian (Eq. 17), using that by definition $\bar{\mathbf{e}} = \mathbf{u} - \mathbf{W} \bar{\mathbf{r}}$. Finally, in the absence of output nudging, $\beta = 0$, we can assume vanishing errors, $\bar{\mathbf{e}} = 0$, as they solve the self-consistency equation, $\bar{\mathbf{e}} = \bar{\mathbf{r}}'_{\text{net}} \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}}$ for all t , see Eq. 27. For these solutions we have $\bar{\mathbf{e}} \bar{\mathbf{r}}^T|_{\beta=0} = 0$. Writing out the total derivative of the function $g(\beta) = \bar{\mathbf{e}} \bar{\mathbf{r}}^T$ with respect to β at $\beta = 0$ as limit of the difference quotient,

$\left. \frac{dg(\beta)}{d\beta} \right|_{\beta=0} = \lim_{\beta \rightarrow 0} \frac{1}{\beta} (g(\beta) - g(0)) = \lim_{\beta \rightarrow 0} \frac{1}{\beta} g(\beta)$, using that $g(0) = 0$, we calculate at any t ,

$$\left. \frac{d \bar{\mathbf{e}} \bar{\mathbf{r}}^T}{d\beta} \right|_{\beta=0} = \lim_{\beta \rightarrow 0} \frac{1}{\beta} (\bar{\mathbf{e}} \bar{\mathbf{r}}^T - \bar{\mathbf{e}} \bar{\mathbf{r}}^T|_{\beta=0}) = \lim_{\beta \rightarrow 0} \frac{1}{\beta} \bar{\mathbf{e}} \bar{\mathbf{r}}^T. \quad (30)$$

Here we assume that $\bar{\mathbf{e}} \bar{\mathbf{r}}^T$ is evaluated at $\beta > 0$ (that itself approaches 0), while $\bar{\mathbf{e}} \bar{\mathbf{r}}^T|_{\beta=0}$ is evaluated at $\beta = 0$. Combining Eq. 29 and 30 yields the cost gradient at any t ,

$$-\frac{dC}{d\mathbf{W}} = \lim_{\beta \rightarrow 0} \frac{1}{\beta} \bar{\mathbf{e}} \bar{\mathbf{r}}^T. \quad (31)$$

This justifies the gradient learning rule $\dot{\mathbf{W}}$ in Eq. 27. Learning is stochastic gradient descent on the expected cost, where stochasticity enters in the randomization of the stimulus and target sequences $\mathbf{r}_{\text{in}}(t)$ and $\mathbf{u}^*(t)$. For the regularity statement, see ‘From implicit to explicit differential equations’ in the sec:Integration. Notice that this proof works for a very general form of the Lagrangian L , until the specific expression for $\frac{\partial L}{\partial \mathbf{W}}$. For a proof in terms of partial derivatives only, see SI, Sect. I, where also a primer on partial and total derivatives is found (SI, Sect. H).

Instantaneous gradient descent on $C(\mathbf{u}_o^*, \bar{\mathbf{r}}_{\text{in}})$

The cost $C = \frac{1}{2} \|\mathbf{u}_o^* - \mathbf{u}_o\|^2$ at each time t is a function of the voltage $\mathbf{u}_o$ of the output neurons and the corresponding targets. In a feedforward network, due to the instantaneity of the voltage propagation (Eq. 23), $\mathbf{u}_o$ is in the absence of output nudging ($\beta = 0$) an instantaneous function of the voltage at the first layer, $\mathbf{u}_1(t) = \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}}(t) + \mathbf{u}_1(t_0) e^{-\frac{t-t_0}{\tau}}$. For initialisation at $t = -\infty$, the second term vanishes for all t and hence $\mathbf{u}_1(t) = \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}}(t)$. The output voltage $\mathbf{u}_o(t)$ therefore becomes a function F_W of the low-pass filtered input rate $\mathbf{r}_{\text{in}}(t)$ that captures the instantaneous network mapping, $\mathbf{u}_o(t) = F_W(\bar{\mathbf{r}}_{\text{in}}(t))$, and with this the cost also becomes an instantaneous function of $\bar{\mathbf{r}}_{\text{in}}$ and $\mathbf{u}_o^*$, namely

$$C(t) = \frac{1}{2} \|\mathbf{u}_o^*(t) - \mathbf{u}_o(t)\|^2 = \frac{1}{2} \|\mathbf{u}_o^*(t) - F_W(\bar{\mathbf{r}}_{\text{in}}(t))\|^2.$$

For a general network, again assuming $t_0 = -\infty$, the voltage is determined by the vanishing gradient $\frac{\partial L}{\partial \mathbf{u}} = \mathbf{f}(\mathbf{u}, t) = \mathbf{u} - \mathbf{W}\bar{\mathbf{r}}(\mathbf{u}) - \bar{\mathbf{e}}(\mathbf{u}) = 0$ with $\bar{\mathbf{e}} = \bar{\mathbf{e}} - \beta\bar{\mathbf{e}}^*$, see Eq. 21. For the inclusive treatment of the initial transient see SI, Sects C and D. Remember that $\bar{\mathbf{r}} = (\bar{\mathbf{r}}_{\text{in}}, \bar{\mathbf{r}}_{\text{net}}(\mathbf{u}))^T$ and $\bar{\mathbf{e}}^* = \mathbf{u}_o^* - \mathbf{u}_o$. For a given $\bar{\mathbf{r}}_{\text{in}}$ and $\mathbf{u}_o^*$ at time t , the equation $\mathbf{f}(\mathbf{u}, t) = 0$ can be locally solved for $\mathbf{u}$ if the Hessian $\mathbf{H} = \frac{\partial^2 L}{\partial \mathbf{u}^2} = \frac{\partial \mathbf{f}}{\partial \mathbf{u}} = \mathbf{1} - \mathbf{W}_{\text{net}}\rho' - \bar{\mathbf{e}}'$ is invertible, $\mathbf{u} = \mathbf{F}(\bar{\mathbf{r}}_{\text{in}}, \mathbf{u}_o^*)$. This mapping can be restricted to the output voltages $\mathbf{u}_o$ on the left-hand side, while replacing $\mathbf{u}_o^* = \mathbf{u}_o + \bar{\mathbf{e}}_o^*$ in the argument on the right-hand side (even if this again introduces $\mathbf{u}_o$ there). With this we obtain the instantaneous mapping $\mathbf{u}_o(t) = \mathbf{F}_W(\bar{\mathbf{r}}_{\text{in}}(t), \bar{\mathbf{e}}_o^*(t))$ from the low-pass filtered input and the output error to the output itself. Notice that for functional feedforward network, the network weight matrix $\mathbf{W}_{\text{net}}$ is lower triangular, and for small enough β the Hessian $\mathbf{H}$ is therefore always positive definite (see also Methods, Sect. F).

E. Proving Theorem 2 (rt-DeEL)

Here we restrict ourselves to layered network architectures. To prove Theorem 2 first assume that interneurons receive no nudging ($\beta^I = 0$) and only the lateral interneuron-to-pyramidal weights $\mathbf{W}_l^{\text{PI}}$ are plastic. This is already sufficient to prove the rt-DeEL theorem. Yet, simulations showed that learning the lateral pyramidal-to-interneuron weights $\mathbf{W}_l^{\text{IP}}$ via top-down nudging, so that the interneuron activity mimics the upper layer pyramidal neuron activity, helps in learning a correct error representation. We consider this case of learning $\mathbf{W}_l^{\text{IP}}$ later.

If the microcircuits is ought to correctly implement error backpropagation, all local prediction errors $\bar{\mathbf{e}}_l$ must vanish in the absence of output nudging ($\beta = 0$) as there is no target error. Consequently, any remaining errors in the network are caused by a misalignment of the lateral microcircuit. We show how learning the interneuron-to-pyramidal weights $\mathbf{W}_l^{\text{PI}}$ corrects for such misalignments.

To define the gradient descent plasticity of the weights $\mathbf{W}_l^{\text{PI}}$ from the interneurons to the pyramidal neurons, we consider the apical error formed by the difference of top-down input and interneuron input, $\bar{\mathbf{e}}_l^A = \mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{\text{PI}} \mathbf{u}_l^I$, and define the apical mismatch energy as

$E_l^{\text{PI}} = \frac{1}{2} \|\bar{\mathbf{e}}_l^A\|^2$. Gradient descent along this energy with respect to $\mathbf{W}_l^{\text{PI}}$ yields

$$\dot{\mathbf{W}}_l^{\text{PI}} = \eta^{\text{PI}} \bar{\mathbf{e}}_l^A \mathbf{u}_l^{I^T} = \eta^{\text{PI}} (\mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{\text{PI}} \mathbf{u}_l^I) \mathbf{u}_l^{I^T}, \quad (32)$$

evaluated online while presenting input patterns from the data distribution to the network. We assume that the apical contribution to the somatic voltage is further modulated by the somatic spike rate, $\bar{\mathbf{r}}_l' \cdot \bar{\mathbf{e}}_l^A$. After successful learning, the top-down input $\mathbf{B}_l \mathbf{u}_{l+1}$ is fully subtracted away by the lateral input in the apical compartment, and we have

$$\mathbf{B}_l \mathbf{u}_{l+1} = \mathbf{W}_l^{\text{PI}} \mathbf{u}_l^I. \quad (33)$$

Once this condition is reached, the network achieves a state where, over the activity space spanned by the data, top-down prediction errors throughout the network vanish,

$$\bar{\mathbf{e}}_l = \bar{\mathbf{r}}_l' \cdot \bar{\mathbf{e}}_l^A = \bar{\mathbf{r}}_l' \cdot (\mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{\text{PI}} \mathbf{u}_l^I) = 0. \quad (34)$$

We show that this top-down prediction error, after the successful learning of the microcircuit, shares the properties of error-backpropagation for a suitable backprojection weights $\mathbf{B}$.

Due to the vanishing prediction errors, pyramidal cells only receive bottom-up input $\mathbf{u}_{l+1} = \mathbf{W}_{l+1} \bar{\mathbf{r}}_l$. Using this expression as well as the expression for interneuron membrane potentials without top-down nudging ($\beta^l = 0$ in Eq. 9), $\mathbf{u}_l^I = \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l$, and plugging both into Eq. 33, we get

$$\mathbf{B}_l \mathbf{W}_{l+1} \bar{\mathbf{r}}_l = \mathbf{W}_l^{\text{PI}} \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l. \quad (35)$$

Assuming that $\mathbf{W}_l^{\text{IP}}$ has full rank, and the low-pass filtered rates $\bar{\mathbf{r}}_l$ span the full n_l dimensions of layer l when sampled across the data set, we conclude that

$$\mathbf{B}_l \mathbf{W}_{l+1} = \mathbf{W}_l^{\text{PI}} \mathbf{W}_l^{\text{IP}}. \quad (36)$$

In other words, the loop via upper layer and back is learned to be matched by a lateral loop through the interneurons. Eq. 36 imposes a restriction on the minimal number of interneurons n_l^I at layer l . In fact, the matrix product $\mathbf{B}_l \mathbf{W}_{l+1}$ maps a n_{l+1} -dimensional space onto itself via n_{l+1} -dimensional space. The maximal rank of this matrix product is limited by the smallest dimension, i.e. $\text{rank}(\mathbf{B}_l \mathbf{W}_{l+1}) \leq \min(n_l, n_{l+1})$. Analogously, $\text{rank}(\mathbf{W}_l^{\text{PI}} \mathbf{W}_l^{\text{IP}}) \leq \min(n_l, n_l^I)$. But since the two ranks are the same according to Eq. 36, we conclude that in general $n_l^I \geq \min(n_l, n_{l+1})$ must hold, i.e. there should be at least as many interneurons at layer l as the lowest number of pyramidal neurons at either layer l or $l+1$. Note that by choosing $n_l^I = n_{l+1}$ as in (Sacramento *et al.*, 2018) (or $n_l^I > n_{l+1}$ as in this work), the conditions is fulfilled.

With $\mathbf{u}_l^I = \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l$ and Eq. 36, the top-down prediction error from Eq. 34, in the presence of output nudging ($\beta > 0$), can be written in the backpropagation form

$$\bar{\mathbf{e}}_l = \bar{\mathbf{r}}_l' \cdot (\mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{\text{PI}} \mathbf{u}_l^I) = \bar{\mathbf{r}}_l' \cdot (\mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{\text{PI}} \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l) \quad (37a)$$

$$= \bar{\mathbf{r}}_l' \cdot (\mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{B}_l \mathbf{W}_{l+1} \bar{\mathbf{r}}_l) = \bar{\mathbf{r}}_l' \cdot \mathbf{B}_l (\mathbf{u}_{l+1} - \mathbf{W}_{l+1} \bar{\mathbf{r}}_l) \quad (37b)$$

$$= \bar{\mathbf{r}}_l' \cdot \mathbf{B}_l \bar{\mathbf{e}}_{l+1} = \bar{\mathbf{r}}_l' \cdot \mathbf{B}_l \bar{\mathbf{r}}_{l+1}' \cdot \bar{\mathbf{e}}_{l+1}^A. \quad (37c)$$

Finally, the simulations showed that learning the lateral weights in the microcircuit greatly benefits from also adapting the pyramidal-to-interneuron weights $\mathbf{W}^{\text{IP}}$ by gradient descent on $E^{\text{IP}} = \frac{1}{2} \sum_l \|\mathbf{u}_l^I - \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l\|^2$, using top-down nudging of the inhibitory neurons ($\beta^l > 0$),

$$\dot{\mathbf{W}}_l^{\text{IP}} = \eta^{\text{IP}} (\mathbf{u}_l^I - \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l) \bar{\mathbf{r}}_l^T. \quad (38)$$

After learning we have $\mathbf{u}_l^I = \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l$, and plugging in $\mathbf{u}_l^I = (1 - \beta^l) \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l + \beta^l \mathbf{B}_l^{\text{IP}} \mathbf{u}_{l+1}$ (Eq. 9), we obtain $\mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l = \mathbf{B}_l^{\text{IP}} \mathbf{u}_{l+1}$. Since $\mathbf{u}_{l+1} = \mathbf{W}_{l+1} \bar{\mathbf{r}}_l$, we conclude as before,

$$\mathbf{W}_l^{\text{IP}} = \mathbf{B}_l^{\text{IP}} \mathbf{W}_{l+1}. \quad (39)$$

The top-down weights $\mathbf{B}_l^{\text{IP}}$ that nudge the lower-layer interneurons has randomized entries and may be considered as full rank. If there are less pyramidal neurons in the upper layer than interneurons in the lower layer, $\mathbf{B}_l^{\text{IP}}$ selects a subspace in the interneuron space of dimension $n_{l+1} < n_l^I$. This seems to simplify the learning of the interneuron-to-pyramidal cell connections $\mathbf{W}^{\text{PI}}$. In fact, this learning now has only to match the n_{l+1} -dimensional interneuron subspace embedded in n_l^I dimensions to an equal (n_{l+1} -)dimensional pyramidal cell subspace embedded in n_l dimensions.

Learning of the interneuron-to-pyramidal cell connections works with the interneuron nudging as before, and combining Eqs 36 with 39 yields the ‘loop consistency’

$$B_l W_{l+1} = W_l^{\text{PI}} B_l^{\text{IP}} W_{l+1} . \quad (40)$$

The learning of the microcircuit was described in the absence of output nudging. Conceptually, this is not a problem as one could introduce a pre-learning phase where the lateral connections are first correctly aligned before learning of the feedforward weights begins. In simulations we find that both the lateral connections as well as the forward connections can be trained simultaneously, without the need for such a pre-learning phase. We conjecture that this is due to the fact that our plasticity rules are gradient descent on the energy functions L , E^{PI} and E^{IP} respectively.

F. From implicit to explicit differential equations

The voltage dynamics is solved by a forward-Euler scheme $\mathbf{u}(t+dt) = \mathbf{u}(t) + \dot{\mathbf{u}}(t) dt$. The derivative $\dot{\mathbf{u}}(t)$ is calculated either through (i) the implicit differential equation (Eq. 7) yielding $\tau \dot{\mathbf{u}}(t) = \mathbf{h}(\mathbf{u}(t), \dot{\mathbf{u}}(t-dt))$, or (ii) by isolating $\dot{\mathbf{u}}(t)$ and solving for the explicit differential equation $\tau \dot{\mathbf{u}}(t) = \mathbf{g}(\mathbf{u}(t))$, as explained in SI, Sect. C (after Eq. 51).

1. The implicit differential equation, $\tau \dot{\mathbf{u}}(t) = -\mathbf{u}(t) + \mathbf{W} \mathbf{r}(t) + \mathbf{e}(t)$, see Eq. 22, is iteratively solved by assigning $\mathbf{r}(t) = \rho(\mathbf{u}(t)) + \rho'(\mathbf{u}(t)) \cdot \dot{\mathbf{u}}(t-dt)$ and calculating the error $\mathbf{e}(t) = \bar{\mathbf{e}}(t) + \tau \dot{\bar{\mathbf{e}}}(t)$ with

$$\bar{\mathbf{e}}(\mathbf{u}) = \rho'(\mathbf{u}) \cdot \mathbf{W}_{\text{net}}^T (\mathbf{u} - \mathbf{W}_{\text{net}} \rho(\mathbf{u}) - \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}}) + \beta \bar{\mathbf{e}}^* \text{ and } \dot{\bar{\mathbf{e}}}(t) = \bar{\mathbf{e}}'(\mathbf{u}(t)) \cdot \dot{\mathbf{u}}(t-dt).$$

This iteration exponentially converges to a fixed point $\dot{\mathbf{u}}(t)$ on a time scale $\frac{dt}{1-k}$, where $1-k > 0$ is the smallest Eigenvalue of the Hessian $\mathbf{H} = \frac{\partial^2 L}{\partial \mathbf{u}^2} = \mathbf{1} - \mathbf{W}_{\text{net}} \rho' - \bar{\mathbf{e}}'$, see SI, Sect. C.

2. The explicit differential equation is obtained by eliminating the $\dot{\mathbf{u}}$ from the right-hand side of the implicit differential equation. Since $\dot{\mathbf{u}}$ enters linearly we get

$$\tau \mathbf{H} \dot{\mathbf{u}} = -\mathbf{f} - \tau \frac{\partial \mathbf{f}}{\partial t} \text{ with } \mathbf{f}(\mathbf{u}, t) = \frac{\partial L}{\partial \mathbf{u}} = \mathbf{u} - \mathbf{W} \bar{\mathbf{r}} - \bar{\mathbf{e}} - \beta \bar{\mathbf{e}}^*. \text{ The explicit form is obtained by matrix inversion, } \dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, t) = -\frac{1}{\tau} \mathbf{H}^{-1} \left(\mathbf{f} + \tau \frac{\partial \mathbf{f}}{\partial t} \right), \text{ as the}$$

Hessian is invertible if it is strictly positive definite (which is typically the case, see SI, Sect. C, after Eq. 48). The external input and the target enter through

$$\frac{\partial \mathbf{f}}{\partial t} = \mathbf{W}_{\text{in}} \dot{\bar{\mathbf{r}}}_{\text{in}} + \beta \dot{\mathbf{u}}_{\text{o}}^*, \text{ where the derivative of the target voltage is only added for the output neurons } \mathbf{o}. \text{ This explicit differential equation is shown to be contractive in the sense that for each input trajectory } \mathbf{r}_{\text{in}}(t) \text{ and target trajectory } \mathbf{u}^*(t), \text{ the voltage trajectory } \mathbf{u}(t) \text{ is locally attracting for neighbouring trajectories. This local attracting trajectory is the vanishing-gradient trajectory } \mathbf{f}(\mathbf{u}, t) = 0, \text{ and the gradient remains 0 even if the input contains delta-functions, see SI Sect. D.}$$

Moving and latent equilibria: a formal definition

We showed that the motor output ($\mathbf{u}_{\text{o}}$), together with the low-pass filtered sensory input ($\bar{\mathbf{r}}_{\text{in}}$) and the motor feedback ($\bar{\mathbf{e}}_{\text{o}}^*$) is in a moving equilibrium, $\mathbf{u}_{\text{o}} = \mathbf{F}_{\text{W}}(\bar{\mathbf{r}}_{\text{in}}, \bar{\mathbf{e}}_{\text{o}}^*)$, see Fig. 3a. In general, a dynamical system in $\mathbf{u}$ that is given in an implicit form $\mathbf{G}(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{u}, \dot{\mathbf{u}}) = 0$ with external inputs $(\mathbf{x}, \dot{\mathbf{x}})$ is said to be in a moving equilibrium if the variable $\mathbf{u}$ is an instantaneous

function of the input $\mathbf{x}$ at any point in time, $\mathbf{u} = \mathbf{F}(\mathbf{x})$. The fact that the implicit differential equation $\mathbf{G} = 0$ represents a dynamical system in $\mathbf{u}$ implies that, in principle, it has a representation in the explicit form $\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, \mathbf{x}, \dot{\mathbf{x}})$, guaranteed by an invertible Jacobian $\frac{\partial \mathbf{G}}{\partial \mathbf{u}}$.

Our example is obtained from $\mathbf{G} = (1 + \tau \frac{d}{dt})\mathbf{f}$ with $\mathbf{f}(\mathbf{u}, \mathbf{x}) = \frac{\partial L}{\partial \mathbf{u}}$ and $\mathbf{x} = (\bar{\mathbf{r}}_{\text{in}}, \bar{\mathbf{e}}_o^*)$, leading to $\mathbf{x} + \tau \cdot \dot{\mathbf{x}} = (\mathbf{r}_{\text{in}}, \mathbf{e}_o^*)$. Given the parametrization of $\mathbf{G}$ by the weights, we get the parametrized function $\mathbf{u} = \mathbf{F}_W(\mathbf{x})$, and this is restricted to the output components $\mathbf{u}$ of $\mathbf{u}$. The condition on the Jacobian translates to $\frac{\partial \mathbf{G}}{\partial \mathbf{u}} = \frac{\partial \mathbf{f}}{\partial \mathbf{u}} = \frac{\partial^2 L}{\partial \mathbf{u}^2}$ being invertible. Crucially, the description of the dynamics in the biological or physical substrate is not given in its explicit form $\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, \mathbf{x}, \dot{\mathbf{x}})$. However, it is given in an implicit form expressed as $\dot{\mathbf{u}} = \mathbf{h}(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{u}, \dot{\mathbf{u}})$, where $\dot{\mathbf{u}}$ still appears on the right-hand side. This ‘hybrid’ form is directly solved either in real time by the biophysical substrate itself, or by the forward-Euler scheme on clocked hardware, see (i) above. Notice that moving equilibria $\mathbf{u} = \mathbf{F}_W(\mathbf{x})$ with $\mathbf{x} = (\bar{\mathbf{r}}_{\text{in}}, \bar{\mathbf{e}}_o^*)$ are able to capture complex temporal processing of the instantaneous input $\mathbf{r}_{\text{in}}$. In fact, the low-pass filtering $\bar{\mathbf{r}}_{\text{in}}$ can be obtained on various time scales through different τ_{in} ’s, and $\mathbf{F}_W$ for a general network $\mathbf{W}$ can be arbitrary complex. The task is to adapt $\mathbf{W}$ such that the ‘hybrid’ dynamical system eventually implements the target mapping $\mathbf{u}_o^* = \mathbf{F}^*(\mathbf{x})$.

The Latent Equilibrium (Haider *et al.*, 2021) can be analogously formalized as a dynamical system in $\mathbf{u}$, implicitly given by $\mathbf{G}(\mathbf{x}, \mathbf{u}, \dot{\mathbf{u}}) = 0$, and having a solution of the form

$\mathbf{u} + \tau \cdot \dot{\mathbf{u}} = \mathbf{F}(\mathbf{x})$. Abbreviating again $\mathbf{f}(\mathbf{u}, \mathbf{x}) = \frac{\partial L}{\partial \mathbf{u}}$ with the same Lagrangian $L = \frac{1}{2} \|\mathbf{u} - \mathbf{W}\rho(\mathbf{u})\|^2 + \frac{\beta}{2} C$ as in the present NLA, the Latent Equilibrium is obtained for $\mathbf{G}(\mathbf{x}, \mathbf{u}, \dot{\mathbf{u}}) = \mathbf{f}(\mathbf{u} + \tau \cdot \dot{\mathbf{u}}, \mathbf{x})$. The solution implies that the rate $\mathbf{r} = \rho(\mathbf{u} + \tau \cdot \dot{\mathbf{u}}) = \rho(\mathbf{F}(\mathbf{x}))$ is an instantaneous function of $\mathbf{x} = (\mathbf{r}_{\text{in}}, \mathbf{e}_o^*)$, here without low-pass filtering. As for moving equilibria, the crucial point is that the biophysical substrate implements a hybrid form of the dynamical system, now $\dot{\mathbf{u}} = \mathbf{h}(\mathbf{x}, \mathbf{u}, \dot{\mathbf{u}})$, that is implicitly solved by the analog substrate, and also allows for a solution in clocked hardware. For an extended stability analysis of the moving and latent equilibria see SI, Sect. D.

G. Simulation details

Solving the explicit differential equation seems to be more robust when the learning rate for $\dot{\mathbf{W}}$ gets larger. The explicit form is also less sensitive to large Euler steps dt , see SI Sect. C. By this reason, the ordinary differential equations (ODE) were solved in the explicit form when including plasticity $\dot{\mathbf{W}}$. The algorithms are summarized as follows, once without interneurons (Algo 1), and once with interneurons (Algo 2):

Algorithm 1

with projection neurons only, for Figs 3 & 4 (using the explicit ODE, i.e. Step 12 instead of 11)

```

1: current state:  $u(t), W(t)$ 
2: # consider full vectors and matrices (padded with 0's for feedforward networks)
3: # drop time argument (t) for convenience
4:  $\bar{r}_{\text{net}} \leftarrow \rho(u)$ ,  $\bar{r} \leftarrow (\bar{r}_{\text{in}}, \bar{r}_{\text{net}})^T$ ,  $W \leftarrow (W_{\text{in}}, W_{\text{net}})$ 
5: calculate weight derivatives
6:  $\dot{W} \leftarrow \eta(u - W\bar{r})\bar{r}^T$ 
7: calculate low-pass-filtered errors
8:  $\bar{e}_o^* \leftarrow u_o^* - u_o$ ,  $\bar{e}_i^* = 0$  for non-output neurons
9:  $\bar{e} \leftarrow \bar{r}_{\text{net}}^T \cdot W_{\text{net}}^T(u - W\bar{r}) + \beta\bar{e}^*$ 
10: calculate temporal voltage derivatives either implicitly (11) or explicitly (12)
11: Implicit:  $\tau\dot{u} \leftarrow -u + W(\bar{r} + \tau\dot{\bar{r}}) + (\bar{e} + \tau\dot{\bar{e}})$ 
12: Explicit:  $f \leftarrow u - W\bar{r} - \bar{e}$ ,  $H \leftarrow \frac{\partial f}{\partial u}$ ,  $\dot{u} \leftarrow$  solve  $\tau H(u) \dot{u} = -f - \tau \frac{\partial f}{\partial t}$  via Cholesky decomposition
13: update voltage and weights
14:  $u \leftarrow u + \dot{u} \cdot dt$ ,  $W \leftarrow W + \dot{W} \cdot dt$ 

```

Algorithm 2

including plastic interneurons, for Fig. 5 [↗](#) (using the explicit ODE, i.e. Step 13 instead of 12)

```

1: current state:  $u(t), W(t), u^I(t), W^{\text{PI}}(t), W^{\text{IP}}(t)$ 
2: # consider full vectors and matrices and drop time argument as in Algorithm 1
3:  $\bar{r} \leftarrow (\bar{r}_{\text{in}}, \rho(u))^T$ 
4: calculate weight derivatives
5:  $\dot{W} \leftarrow \eta(u - W\bar{r})\bar{r}^T$ 
6:  $\dot{W}^{\text{PI}} \leftarrow \eta^{\text{PI}}(Bu - W^{\text{PI}}u^I)u^{IT}$ 
7:  $\dot{W}^{\text{IP}} \leftarrow \eta^{\text{IP}}(u^I - W^{\text{IP}}\bar{r})\bar{r}^T$ 
8: calculate low-pass filtered errors
9:  $\bar{e}_o^* \leftarrow u_o^* - u_o$  ( $\bar{e}_i^* = 0$  for non-output neurons  $i$ )
10:  $\bar{e} \leftarrow Bu - W^{\text{PI}}u^I + \beta\bar{e}^*$  ( $B_{o,:} = W_{o,:}^{\text{PI}} = 0$  for output neurons  $o$ )
11: calculate temporal voltage derivatives either implicitly (12) or explicitly (13)
12: Implicit:  $\tau\dot{u} \leftarrow -u + W(\bar{r} + \tau\dot{\bar{r}}) + (\bar{e} + \tau\dot{\bar{e}})$ 
13: Explicit:  $f \leftarrow u - W\bar{r} - \bar{e}$ ,  $H \leftarrow \frac{\partial f}{\partial u}$ ,  $\dot{u} \leftarrow$  solve  $\tau H(u) \dot{u} = -f - \tau \frac{\partial f}{\partial t}$  via Cholesky decomposition
14: update network state
15: for  $X \in \{u, W, W^{\text{PI}}, W^{\text{IP}}\}$  do
16:    $X \leftarrow X + \dot{X} dt$ 
17:  $u^I \leftarrow (1 - \beta^I)W^{\text{IP}}\bar{r} + \beta^I B^{\text{IP}}u$ 

```

Details for Fig. 3b [↗](#)

Color coded snapshot of cortical local field potentials (LFPs) in a human brain from 56 deep iEEG electrodes at various locations, converted with the sigmoidal voltage-to-rate function

$\bar{r}(u) = \frac{1}{1+e^{-u}}$ and plotted onto a standard Talairach Brain (Talairach & Tournoux, 1988 [↗](#)). The

iEEG data is from a patient with pharmacoresistant epilepsy and electrodes implanted during presurgical evaluation, extracted from the data release of Burrello *et al.*, 2019 [↗](#). The locations of the electrodes are chosen in accordance with plausibility, as the original positions of the electrodes were omitted due to ethical standards to prevent patient identification.

Details for Fig. 3c [↗](#)

Simulations of the voltage dynamics (Eq. 7a [↗](#)) and weight dynamics (Eq. 8 [↗](#)), with learning rate $\eta = 10^{-3}$, step size $dt = 1$ ms for the forward Euler integration, membrane time constant $\tau = 10$ ms and logistic activation function. Weights were initialized randomly from a normal distribution $\mathcal{N}(0, 0.1^2)$ with a cut-off at ± 0.3 . The number of neurons in the network $\mathcal{N}$ was $n = 96$, among them 56 output neurons $\mathcal{O} \subset \mathcal{N}$ that were simultaneously nudged, and 40 hidden neurons. During training, all output neurons were nudged simultaneously (with $\beta = 0.1$), whereas during testing, only 42 out of 56 neurons were nudged, the remaining 14 left to reproduce the traces. Data points of the iEEG signal were sampled with a frequency of 512Hz. For simplicity, we therefore assumed that successive data points are separated by 2ms, and up-sampled the signal via simple interpolation to 1ms resolution as required by our integration scheme. Furthermore, the raw values were

normalized by dividing them by a factor of 200 to ensure that they are approximately in a range of $\pm 1 - 2$. Training and testing was done on two separate 8s traces of the iEEG recording. Same data as in [Fig. 3b1](#).

Details for Fig. 4

Simulation of the neuronal and synaptic dynamics as given by [Eq. 7a](#), [Eq. 7b](#) and [Eq. 8](#). For 5 ms, 10 ms and 50 ms presentation time, we used an integration step size of $dt = 0.05$ ms, $dt = 0.1$ ms and $dt = 0.5$ ms, respectively (and $dt = 1$ ms otherwise). As an activation function, we used the step-linear function (hard sigmoidal) with $\bar{r}(u) = 0$ for $u \leq 0$, $\bar{r}(u) = 1$ for $u \geq 1$ and $\bar{r}(u) = u$ in between. The learning rate was initially set to $\eta = 10^{-3}$ and then reduced to $\eta = 10^{-4}$ after 22 000 s. The nudging strength was $\beta = 0.1$ and the membrane time constant $\tau = 10$ ms. In these simulations (and only for these) we assumed that at each presynaptic layer $l = 0, 1, \dots, n - 1$ there is a first neuron indexed by 0 that fires with constant rate $\bar{r}_{l,0} = 1$, effectively allowing the postsynaptic neurons $\bar{r}_{l+1}$ to learn a bias through the first column of the weight matrix $\mathbf{W}_{l+1}$. Weights were initialized randomly from a normal distribution $\mathcal{N}(0, 0.01^2)$ with a cut-off at ± 0.03 . For an algorithmic conversion see the scheme below. In [Fig. 4c1](#), ‘rt-DeEP w/o lookahead’ is based on the dynamics $\tau \dot{u} = -u + \mathbf{W} \bar{r} + \bar{e}$. For ‘ $\dot{u}$ w/o error + backprop’, we use $\tau \dot{u} = -u + \mathbf{W} r$ as the forward model (so without error terms on the membrane potential, but a prospective r), and calculate weight updates using error backpropagation. In [Fig. 4c2](#), we provide three controls: the test error for (i) a standard shallow artificial neural network trained on MNIST (black dashed line), (ii) rt-DeEP without prospective coding (as in [Fig. 4c1](#)), but in c2 with plasticity only turned on when the network is completely stationary, i.e., after waiting for several 100ms, such that synaptic weights are not changed during transients (orange dashed line, denoted by ‘w/o transients’), and (iii) an equivalent artificial neural network, $u_l = \mathbf{W}_l \bar{r}_{l-1}$, trained using error backpropagation (black dashed line, ‘standard backprop’).

Details for Fig. 5

Simulation of neuronal and synaptic dynamics with plastic microcircuit, i.e., the pyramidal-to-interneuron and lateral weights of the microcircuit learned during training.

For the results shown in [Fig. 5c2](#), the following parameters were used. As an activation function, we used a hard sigmoid function and the membrane time constant was set to $\tau = 10$ ms. Image presentation time is 100 ms. Forward, pyramidal-to-interneuron and interneuron-to-pyramidal weights were initialized randomly from a normal distribution $\mathcal{N}(0, 0.01^2)$ with a cut-off at ± 0.03 . All learning rates were chosen equal $\eta = 10^{-3}$ and were subsequently reduced to $\eta = 10^{-4}$ after 22000s training time. The nudging parameters were set to $\beta = 0.1$ and $\beta^I = \frac{0.1}{1.1}$. The feedback connections $\mathbf{B}_l$ and the nudging matrices $\mathbf{B}_l^{\text{IP}}$ were initialized randomly from a normal distribution $\mathcal{N}(0, 0.01^2)$ with a cut-off at ± 0.15 . The used integration step size was $dt = 0.25$ ms. All weights were trained simultaneously. For an algorithmic conversion see the scheme below. The interneuron membrane potential was calculated by [Eq. 9](#) with a linear transfer function.

Acknowledgements

We would like to thank Federico Benitez, Jonathan Binas, Paul Haider, Kevin Max, Alexander Mathis, Alexander Meulemans and Jean-Pascal Pfister for helpful discussions, Kaspar Schindler for providing the human intracortical EEG data, and the late Karlheinz Meier for his dedication and support throughout the early stages of the project. WS thanks Nicolas Zucchet for mathematical discussions and hints to the literature on time-varying optimal control. This work has received funding from the European Union 7th Framework Programme under grant agreement 604102

(HBP), the Horizon 2020 Framework Programme under grant agreements 720270, 785907 and 945539 (HBP) and the Swiss National Science Foundation (SNSF, Sinergia grant CRSII5180316; João Sacramento is supported by SNSF Ambizione grant PZ00P3_186027). We further express our particular gratitude towards the Manfred Stärk Foundation for their continued support. We acknowledge the use of Fenix Infrastructure resources, which are partially funded from the European Union's Horizon 2020 research and innovation programme through the ICEI project under the grant agreement No. 800858.

Author contributions

WS, MAP and DD designed the conceptual approach. WS, DD, MAP, AFK, JJ, JS and YB contributed to different aspects of the framework and model. WS, DD, MAP and AFK derived different components of the theory. DD, BE and AFK performed the simulations. The manuscript went through multiple drafts, written by MAP, DD, JJ, BE and WS. The two published versions were written by WS.

Supplementary information

A. Extracting the presynaptic voltage error

Threshold-linear transfer functions

There is an important special case where the presynaptic voltage error can directly be extracted from presynaptic firing rates, without need to invert the transfer function via synaptic depression as shown below. This is the case when voltage errors in the upper layers are small, and the voltage-to-rate transfer function has derivatives $\rho' = 0$ or 1 , so that $\bar{r}'_{l+1} = (\bar{r}'_{l+1})^2$. The condition is satisfied, for instance, for a doubly threshold-linear function (a ‘doubly rectified linear unit’, ReLu) defined by $\rho(\bar{u}) = 0$ for $\bar{u} < 0$ and $\rho(\bar{u}) = \bar{u}$ for $0 \leq \bar{u} \leq r_{\max}$, while $\rho(\bar{u}) = r_{\max}$ for larger voltages. In this case we calculate

$$\bar{e}_{l+1} = \bar{r}'_{l+1} \cdot \bar{e}_{l+1}^A = (\bar{r}'_{l+1})^2 \cdot \bar{e}_{l+1}^A = \bar{r}'_{l+1} \cdot \bar{e}_{l+1} = \bar{r}'_{l+1} \cdot (\mathbf{u}_{l+1} - \mathbf{W}_{l+1} \bar{\mathbf{r}}_l) \quad (41a)$$

$$\approx \rho(\mathbf{u}_{l+1}) - \rho(\mathbf{W}_{l+1} \bar{\mathbf{r}}_l) = \bar{r}_{l+1} - \rho(\mathbf{W}_{l+1} \bar{\mathbf{r}}_l). \quad (41b)$$

The approximation uses the Taylor expansion in $\mathbf{u}_{l+1}$ and assumes that $\bar{e}_{l+1}$ is small. The crucial point of [Eq. 41](#) is that the mismatch error defined on the voltage, $\bar{e}_{l+1} = \mathbf{u}_{l+1} - \mathbf{W}_{l+1} \bar{\mathbf{r}}_l$, can be factorized into a product of the postsynaptic rate derivative, $\bar{r}'_{l+1}$, and the apical error, and hence it can be expressed as an error defined on the rate. Restricted to the segment $0 \leq \bar{u}_{l+1} \leq r_{\max}$ where the transfer function is linear and errors do not vanish, the same microcircuit delivers the feedback $\mathbf{B}_l \bar{\mathbf{r}}_{l+1}$ to the apical tree through the top-down projections, and $-\mathbf{B}_l \rho(\mathbf{W}_{l+1} \bar{\mathbf{r}}_l)$ through the lateral connections from the interneurons. While the plasticity rules for $\mathbf{W}_l^{\text{PI}}$ and $\mathbf{W}_l^{\text{IP}}$ stay the same, the top-down nudging of the interneurons, see [Eq. 9](#), can then be formulated based on the rate instead of the upper layer voltage, $\mathbf{u}_l^{\text{I}} = (\bar{1} - \beta^{\text{I}}) \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l + \beta^{\text{I}} \mathbf{B}_l^{\text{IP}} \bar{\mathbf{r}}_{l+1}$, with transfer function of the interneuron again the (doubly) threshold-linear $\rho(\mathbf{u}_l^{\text{I}})$. Since voltages and rates are identical in the segment $0 \leq \bar{u}_{l+1} \leq r_{\max}$ for each component, [Eqs 36](#) with [39](#) can still be inferred.

Rate-to-voltage inversion by short-term synaptic depression

We wish to readout the voltage error also for other nonlinear transfer functions than clipped ReLU's. To do so, we take inspiration from the classical short-term synaptic depression model (Tsodyks & Markram, 1997; Abbott *et al.*, 1997; Varela *et al.*, 1997), see also Fig. 6a1 (that deviates from the release-independent synaptic depression, see Campagnola *et al.*, 2022). We consider a dynamic vesicle release probability that is proportional to the pool size of available vesicles, $v(\bar{r})$, and this pool size is postulated to depend on past presynaptic rates,

$$p(\text{release} | \bar{r}) \propto v(\bar{r}) = 1 + \frac{a}{1 + d\bar{r}}, \quad (42)$$

where $\bar{r}$ is the low-pass filtered presynaptic rate, a and d are constants. The proportionality factor is $\frac{1}{1+a}$, making a probability out of the vesicle pools size. The effective synaptic strength B of a 'backprojecting top-down' connection is the product of the absolute synaptic strength B_o and the vesicle pool size v , i.e. $B = B_o v(\bar{r})$. The contribution to the postsynaptic current of the synapse is Wr , and the contribution to the postsynaptic voltage is $W\bar{r}$.

We search for an activation function $\bar{r}(u)$ such that the postsynaptic voltage contribution is the scaled presynaptic potential, $B\bar{r}(u) = B_o u$. Plugging in the above expression for B yields $B\bar{r}(u) = B_o v(\bar{r}) \bar{r}(u) = B_o u$, and dividing B_o out, $v(\bar{r}) \bar{r}(u) = u$, see Fig. 6a2. With the expression for $v(\bar{r})$ in Eq. 42 we obtain a quadratic equation in $\bar{r}$ that is solved by the non-negative and monotonically increasing function

$$\bar{r} = \frac{1}{2} \left(u - \theta + \sqrt{(u - \theta)^2 + 4u/d} \right) \geq 0, \text{ for } u \geq 0, \quad (43)$$

with 'smooth' threshold $\theta = (1 + a)/d$ and asymptote $\bar{r} \approx u - \theta$. This gives us a transfer function $\bar{r}(u)$ that qualitatively matches those observed for pyramidal neurons recorded in the steady state (Anderson *et al.*, 2000; Rauch *et al.*, 2003), see Fig. 6a3.

The approach generalizes to other pairs of strictly monotonic neuronal activation and depression functions $\{\bar{r}(u), v(\bar{r})\}$, as long as u is not driven below some minimal value, here $u_{\text{rest}} = 0$, corresponding to $\bar{r} = 0$. The last requirement can, for instance, be accomplished by offsetting the activation function into a regime that guarantees that u stays positive.

In our simulations for Fig. 5, we did not explicitly implement a dynamic vesicle pool, i.e. the right-hand side of $v(\bar{r}) \bar{r} = u$, but instead directly used the recovered membrane potentials u .

B. Looking back and forward in time with derivatives

Since dealing with extrapolations into the future is a crucial notion of the paper we present here some of the calculations. The discounted future voltage was introduced in Eq. 4 as

$$\tilde{u}(t) = \frac{1}{\tau} \int_t^\infty u(t') e^{-\frac{t'-t}{\tau}} dt'.$$

To show that $\tilde{u}$ satisfies $u = \tilde{u} - \tau \dot{\tilde{u}}$, we need to apply the Leibniz integral rule in calculating the derivative $\dot{\tilde{u}}$. This leads to

$$\frac{d\tilde{u}}{dt}(t) = -\frac{1}{\tau} u(t) + \frac{1}{\tau} \int_t^\infty u(t') \frac{1}{\tau} e^{-\frac{t'-t}{\tau}} dt'.$$

Multiplying this equation by τ and using the definition of $\tilde{u}$ yields $\tau \dot{u}(t) = -u(t) + \tilde{u}(t)$, or $u = \tilde{u} - \tau \dot{\tilde{u}}$. By applying the Leibniz integral rule one also shows that $\bar{x}$, defined in [Eq. 15](#), [↗](#),

$$\bar{x}(t) = \frac{1}{\tau} \int_{-\infty}^t x(t') e^{-\frac{t-t'}{\tau}} dt',$$

solves $\tau \dot{\bar{x}}(t) = -\bar{x}(t) + x(t)$. This differential equation can be written as $(1 + \tau \frac{d}{dt})\bar{x} = x$, with lookahead operator $(1 + \tau \frac{d}{dt})$ defined in [Eq. 14](#). To show that $\overline{(1 + \tau \frac{d}{dt})x} = \bar{x} + \tau \bar{\dot{x}} = x$, one applies partial integration to $\bar{\dot{x}}(t)$. Note that the equality $\bar{x} + \tau \bar{\dot{x}} = \bar{x} + \tau \dot{\bar{x}} = x$ only holds if we integrate from $-\infty$, and hence if the initialization of the trajectory is far back in the past as compared to the time constant τ .

Uniqueness of the rate function

In the main text we concluded from the postulate $r = \rho(u) + \tau \dot{\rho}(u)$ and the general relation $r = \bar{r} + \tau \dot{\bar{r}}$ that $\bar{r}(t) = \rho(u(t))$. This conclusion is a consequence of the uniqueness of a solution of an ordinary differential equation for a given initial condition (that may include delta-functions on the right-hand side, see e.g. [Nedeljkov & Oberguggenberger, 2012](#)). In fact, we may consider both variables ρ and $\bar{r}$ as solutions of the differential equation $\tau \dot{x} = -x + r$, with x either ρ or $\bar{r}$. Because the solution is unique, we conclude that $\rho = \bar{r}$.

Learning the input time constants

In our applications we assumed that the input rates in the original mapping $\mathbf{u}_o^*(t) = \mathbf{F}^*(\bar{\mathbf{r}}_{\text{in}}(t))$ are low-pass filtered by a common time constant $\tau_{\text{in}(t),i}^* = \tau$ that is also shared as membrane time constant of the neurons. But in general, we may consider a subclass of network neurons (with voltage vector $\mathbf{u}_1$) that receive exclusively input rates $\mathbf{r}_{\text{in}}$ and integrate these rates with corresponding time constants τ_{in} . The prospective rates of these neurons, $\mathbf{r}_1 = \rho(\mathbf{u}_1) + \tau \dot{\rho}(\mathbf{u}_1)$, are then produced by the same voltage time constant τ of the downstream neurons. The downstream voltage integration then cancels the presynaptic look-ahead in the presynaptic firing rate, leading again to an instantaneous voltage propagation across the entire network.

The general setting of learning to map time series is that input time series, $\mathbf{r}_{\text{in}}(t)$ are low-pass filtered with given time constants τ_{in}^* , and the target output time series $\mathbf{u}_o^*(t)$ are a function of these low-pass filtered inputs, $\mathbf{u}_o^*(t) = \mathbf{F}_o^*(\bar{\mathbf{r}}_{\text{in}}^{\tau_{\text{in}}^*}(t))$. In the student network, the input time constants appear as parameters that can be learned by gradient descent to match the output of the teacher network, just as the synaptic weights are learned.

We ask for a learning rule so that the student network can extract the input time constants τ_{in}^* used by the teacher network. For this, we assume that only the neurons $\mathbf{u}_1$ obtain the external input and hence carry the time constant τ_{in} . Because $\bar{r}_{\text{in}}^{\tau_{\text{in}}} = r_{\text{in}} - \tau_{\text{in}} \cdot \dot{\bar{r}}_{\text{in}}^{\tau_{\text{in}}}$, the gradient rule for the input time constant is

$$\dot{\tau}_{\text{in}} = -\frac{\partial L}{\partial \tau_{\text{in}}} = -\frac{\partial L}{\partial \bar{\tau}_{\text{in}}^{\tau_{\text{in}}}} \frac{\partial \bar{\tau}_{\text{in}}^{\tau_{\text{in}}}}{\partial \tau_{\text{in}}} = -\mathbf{W}_{\text{in}}^{\text{T}} (\mathbf{u}_1 - \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}}^{\tau_{\text{in}}})^{\text{T}} \cdot \dot{\bar{\mathbf{r}}}_{\text{in}}^{\tau_{\text{in}}} . \quad (44)$$

This local learning rule also globally reduces the Lagrangian, and in the limits of $\beta \rightarrow \infty$ it is gradient descent on the mismatch energy, while in the limit $\beta \rightarrow 0$ it is gradient descent on the cost. The proof works as in Theorem 1. To learn a more complex mapping of time series that includes

more complex temporal processing beyond a function of merely $\mathbf{u}_o^*(t) = \mathbf{F}_o^*(\bar{\mathbf{r}}_{\text{in}}^*(t))$, additional variables need to be introduced that form memories (see SI, Sect. F, ‘Generalizations’).

C. From the implicit to the explicit differential equation: more details

Global convergence to a moving equilibrium (i.e. to $\frac{\partial L}{\partial \mathbf{u}} = 0$)

The original Euler-Lagrange equation is $(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = 0$, Eq. 20, with $\frac{\partial L}{\partial \mathbf{u}} = \mathbf{f}(\mathbf{u}, t) = \mathbf{u} - \mathbf{W}\bar{\mathbf{r}} - \bar{\mathbf{e}} - \beta\bar{\mathbf{e}}^*$. Remember the definition of $\bar{\mathbf{e}} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}}$ given in Eq. 21. While $\bar{\mathbf{e}} = \mathbf{u} - \mathbf{W}\bar{\mathbf{r}}$ is an error that may not vanish, the ‘corrected error’ $\mathbf{f} = \frac{\partial L}{\partial \mathbf{u}}$ is shown to vanish exponentially quick, leading to the moving equilibrium with $\mathbf{u} = \mathbf{W}\bar{\mathbf{r}} - \bar{\mathbf{e}} - \beta\bar{\mathbf{e}}^*$. In fact, $(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = \mathbf{f} + \tau \dot{\mathbf{f}} = 0$, has the general solution

$$\mathbf{f}(\mathbf{u}(t), t) = \mathbf{f}(\mathbf{u}_0, t_0) e^{-\frac{t-t_0}{\tau}} \rightarrow 0 \quad (45)$$

as an exponentially decaying function in t with $\mathbf{u}(t_0) = \mathbf{u}_0$. Note that the argument $\mathbf{u}$ of the function $\mathbf{u} - \mathbf{W}\bar{\mathbf{r}} - \bar{\mathbf{e}} - \beta\bar{\mathbf{e}}^*$ does apriori not depend on time. It only becomes a function of time, $\mathbf{u}(t)$, by requiring that $\mathbf{f} + \tau \dot{\mathbf{f}} = 0$. The voltage $\mathbf{u}(t)$ becomes a non-trivial function of time because the time-dependence of the function $\mathbf{f}(\mathbf{u}, t)$, that apriori enters only through the second argument, expresses the temporally varying input $\bar{\mathbf{r}}_{\text{in}}(t)$, including a possible target output $\mathbf{u}_o^*(t)$.

Implicit differential equation

To make the dependence of $\dot{\mathbf{u}}$ on the right-hand-side of the implicit differential equation (Eq. 22) more transparent, we rewrite this in the form

$$\dot{\mathbf{u}} = -\frac{\mathbf{f}}{\tau} - \frac{\partial \mathbf{f}}{\partial t} + \frac{\partial}{\partial \mathbf{u}}(\mathbf{W}\bar{\mathbf{r}} + \bar{\mathbf{e}} + \beta\bar{\mathbf{e}}^*) \dot{\mathbf{u}} = -\frac{\mathbf{f}}{\tau} - \frac{\partial \mathbf{f}}{\partial t} + \mathbf{M}\dot{\mathbf{u}} = \mathbf{K}(\bar{\mathbf{r}}_{\text{in}}, \mathbf{u}_o^*, \mathbf{u}, \dot{\mathbf{u}}). \quad (46)$$

The partial derivative of $\mathbf{f}(\mathbf{u}, t)$ with respect to time, $\frac{\partial \mathbf{f}}{\partial t}$, captures the external drive from the inputs and output targets that do not depend on $\mathbf{u}$, but may directly depend on time. In fact, instead of the argument t of $\mathbf{f}$, we could consider the two arguments $\bar{\mathbf{r}}_{\text{in}}$ and $\mathbf{u}_o^*$, so that the partial derivative of $\mathbf{f}(\mathbf{u}, t) = \mathbf{f}(\mathbf{u}, \bar{\mathbf{r}}_{\text{in}}, \mathbf{u}_o^*)$ with respect to t can be written as

$$\frac{\partial f_i}{\partial t} = \frac{\partial f_i}{\partial \bar{\mathbf{r}}_{\text{in}}} \dot{\bar{\mathbf{r}}}_{\text{in}} + \frac{\partial f_i}{\partial \mathbf{u}_o^*} \dot{\mathbf{u}}_o^* = W_{i,\text{in}} \dot{\bar{\mathbf{r}}}_{\text{in}} + \beta \delta_{io} \dot{\mathbf{u}}_o^*, \quad (47)$$

where δ_{io} is the Kronecker delta that is equal to 1 if i is an output neuron, and 0 else. The partial derivatives of $\mathbf{f}$ with respect to $\mathbf{u}$ represents the (symmetric) Hessian of the Lagrangian, $H_{ij} = \frac{\partial f_i}{\partial u_j} = \frac{\partial^2 L}{\partial u_i \partial u_j} = \delta_{ij} - \frac{\partial}{\partial u_j}(\mathbf{W}\bar{\mathbf{r}} + \bar{\mathbf{e}} + \beta\bar{\mathbf{e}}^*)_i$ with δ_{ij} being the Kronecker-delta, $\bar{\mathbf{e}}$ defined in Eq. 21 and $\bar{\mathbf{e}}^*$ defined above Eq. 16. Remember that $\mathbf{W} = (\mathbf{W}_{\text{in}}, \mathbf{W}_{\text{net}})$ and $\bar{\mathbf{r}} = (\bar{\mathbf{r}}_{\text{in}}, \bar{\mathbf{r}}_{\text{net}}) = (\bar{\mathbf{r}}_{\text{in}}, \boldsymbol{\rho}(\mathbf{u}))$. In vectorial notation the Hessian of the Lagrangian is

$$\mathbf{H}(\mathbf{u}) = \frac{\partial \mathbf{f}}{\partial \mathbf{u}} = \frac{\partial^2 L}{\partial \mathbf{u} \partial \mathbf{u}} = \mathbf{1} - \mathbf{W}_{\text{net}} \text{diag}(\boldsymbol{\rho}'(\mathbf{u})) - \frac{\partial}{\partial \mathbf{u}}(\bar{\mathbf{e}} + \beta\bar{\mathbf{e}}^*)(\mathbf{u}), \quad (48)$$

where $\mathbf{1}$ is the identity matrix. This Hessian is ‘typically’ positive definite, as we argue next. For a functionally feedforward network, the weight matrix $\mathbf{W}_{\text{net}}$ is lower triangular, and for small enough β (for which $\bar{\mathbf{e}}$ is small) the diagonal elements are all positive and hence the Hessian $\mathbf{H}$ is invertible. Since $\mathbf{H}$ is also symmetric, it is therefore positive definite. In the general case, we require that $\mathbf{W}_{\text{net}}$ has Eigenvalues smaller than 1, since otherwise the recurrent network dynamics may explode. If $\boldsymbol{\rho}'(\mathbf{u})$ is also smaller than 1, and β still small enough, we conclude again that the Hessian is invertible (and hence positive definite). Notice that the property of $\mathbf{H} = \frac{\partial^2 L}{\partial \mathbf{u} \partial \mathbf{u}}$ being symmetric is a general property of differentiable functions and ways weaker than the requirement that the recurrent weight matrix $\mathbf{W}_{\text{net}}$ is symmetric, as classically required for fixed point convergence in Hopfield-type networks.

Since in the implicit differential equation of Eq. 46 the $\dot{\mathbf{u}}$ also arises on the right-hand-side, we need to show that this Eq. 46, for fixed $(\mathbf{r}_{\text{in}}, \mathbf{u}_o^*, \mathbf{u})$, can be solved for $\dot{\mathbf{u}}$. To find the solution $\dot{\mathbf{u}}$ for given arguments $(\mathbf{r}_{\text{in}}, \mathbf{u}_o^*, \mathbf{u})$ in $\mathbf{K}$, we search for a fixed point of the mapping $\dot{\mathbf{u}}^{(i+1)} = \mathbf{K}(\dot{\mathbf{u}}^{(i)})$. In the argument $\dot{\mathbf{u}}$, the mapping is affine, $\dot{\mathbf{u}}^{(i+1)} = -\frac{\mathbf{f}}{\tau} - \frac{\partial \mathbf{f}}{\partial t} + \mathbf{M}\dot{\mathbf{u}}^{(i)}$, with matrix $\mathbf{M}(\mathbf{u}) = \frac{\partial}{\partial \mathbf{u}}(\mathbf{W}\bar{\mathbf{r}} + \bar{\mathbf{e}} + \beta\bar{\mathbf{e}}^*)$ appearing in Eq. 46. The Banach fixed point theorem asserts that if $\mathbf{K}(\dot{\mathbf{u}})$ is strictly contracting with k , i.e. if $\|\mathbf{K}(\dot{\mathbf{u}}^{(2)}) - \mathbf{K}(\dot{\mathbf{u}}^{(1)})\|^2 \leq k\|\dot{\mathbf{u}}^{(2)} - \dot{\mathbf{u}}^{(1)}\|^2$ for all pairs of inputs and $0 \leq k < 1$, then the iteration (here with iteration time step dt) locally converges to a fixed point $\dot{\mathbf{u}} = \mathbf{K}(\dot{\mathbf{u}})$. Because $\mathbf{K}(\dot{\mathbf{u}}^{(2)}) - \mathbf{K}(\dot{\mathbf{u}}^{(1)}) = \mathbf{M}(\dot{\mathbf{u}}^{(2)} - \dot{\mathbf{u}}^{(1)})$, the mapping is k -contractive if the eigenvalues of $\mathbf{M}$ have absolute value smaller than k . This is the case if the Hessian $\mathbf{H}(\mathbf{u}) = \mathbf{1} - \mathbf{M}(\mathbf{u}) = \mathbf{1} - \mathbf{W}_{\text{net}}\boldsymbol{\rho}'(\mathbf{u}) - \bar{\mathbf{e}}'(\mathbf{u})$, with $\bar{\mathbf{e}} = \bar{\mathbf{e}} - \beta\bar{\mathbf{e}}^*$, is strictly positive definite (which is ‘typically’ the case, see above).

Stable solution of the implicit differential equation

According to the above reasoning, the Banach fixed point theorem asserts (for the typically positive Hessian) that during convergence of the mapping $\dot{\mathbf{u}}^{(i+1)} = \mathbf{K}(\dot{\mathbf{u}}^{(i)})$ the distance to the fixed point is bounded by a constant times $e^{-i\frac{1-k}{dt}}$, with iteration index i and ‘virtual’ Euler step dt . Formally,

$$\|\mathbf{K}(\dot{\mathbf{u}}^{(i+1)}) - \mathbf{K}(\dot{\mathbf{u}}^{(i)})\| = \|\mathbf{M}(\dot{\mathbf{u}}^{(i+1)} - \dot{\mathbf{u}}^{(i)})\| \leq k\|\dot{\mathbf{u}}^{(i+1)} - \dot{\mathbf{u}}^{(i)}\| \leq c_0 e^{-i\frac{1-k}{dt}} \rightarrow 0. \quad (49)$$

Crucially, because the mapping is affine in $\dot{\mathbf{u}}$, the convergence is global in $\dot{\mathbf{u}}$ while fixing the other arguments $(\mathbf{r}_{\text{in}}, \mathbf{u}_o^*, \mathbf{u})$ of $\mathbf{K}$.

In an analogue physical device that implements exactly this feedback circuit in continuous time, the dt becomes truly infinitesimal and in this sense the convergence is instantaneous. If dt remains finite, $\dot{\mathbf{u}}^{(i)}$ converges to a moving target because the mapping $\dot{\mathbf{u}}^{(i+1)} = \mathbf{K}(\dot{\mathbf{u}}^{(i)})$ changes with time. The target should not move quicker than the time scale $\frac{dt}{1-k}$ of the $\dot{\mathbf{u}}^{(i)}$ convergence (but, fortunately, this convergence time-scale can be made arbitrarily quick by reducing dt). Given a

time course of the input rate $r_{\text{in}}(t)$ and target $u_o^*(t)$ that has bounded variation, the dt can be chosen so that convergence becomes arbitrary quick, and in the limit instantaneous. As a consequence, for small enough dt , the $\dot{u}$ becomes the same on the left- and right-hand side of the implicit differential equation 46.

If $r_{\text{in}}(t)$ contains well-separated delta-functions, while otherwise still having bounded variations, the reasoning still applies since at any point in time, except at the time points of the isolated delta-function, $f(u, t) = 0$. This statement is proven in Appendix D below.

In practical simulations, a caveat is in place when the learning rate becomes too big and $\dot{W}$ starts to change the neuronal dynamics. In this case, the Hessian becomes $H = 1 - (W \text{diag}(\rho') + \dot{W} \text{diag}(\rho)) - \bar{e}'$, and the Eigenvalues may become negative due to the $\dot{W}$ term. Simulations can in fact become unstable with a big learning rate, and this is more pronounced if also the Euler dt is large. The explicit differential equation avoids the fast iteration towards a moving target and hence allows for larger dt . This in particular pays out in the presence of a high learning rate (although the Cholesky decomposition also requires positive definiteness). By this reason, the large-scale simulations involving plasticity are performed with the explicit form described next.

Explicit differential equation

To isolate $\dot{u}$ in the implicit differential equation, we rewrite $(1 + \tau \frac{d}{dt})f = 0$ again as

$$\tau \dot{f} = \tau \left(\frac{\partial f}{\partial u} \dot{u} + \frac{\partial f}{\partial t} \right) = -f, \quad \text{or} \quad \tau \frac{df_i}{dt} = \tau \left(\sum_j \frac{\partial f_i}{\partial u_j} \dot{u}_j + \frac{\partial f_i}{\partial t} \right) = -f_i. \quad (50)$$

Plugging the Hessian $H(u) = \frac{\partial^2 f}{\partial u^2}$ from Eq. 48 into Eq. 50, we obtain the voltage dynamics from Eq. 22 in the equivalent form

$$\tau H(u) \dot{u} = -f - \tau \frac{\partial f}{\partial t}. \quad (51)$$

In our applications, the Hessian H appears to be invertible (although this may not be the case for arbitrary networks), and Eq. 51 can be solved for the unique $\dot{u}$ using the Cholesky decompositions. In fact, if H is invertible, the system implicit ordinary differential equations from Eq. 51 can be converted into a system of explicit ordinary differential equations (with $f(u, t) = \frac{\partial L}{\partial u} = u - W\bar{r} - \bar{e} - \beta \bar{e}^*$ and H given in Eq. 48),

$$\dot{u} = g(u, t) = -\frac{1}{\tau} H^{-1}(u) \left(f(u) + \tau \frac{\partial f}{\partial t} \right), \quad (52)$$

and as such it has a unique solution for any given initial condition. Eq. 52 represents the explicit solution of the iteration $\dot{u}^{(i+1)} = K(\dot{u}^{(i)})$ after the convergence to the fixed point $\dot{u} = K(\dot{u})$. The condition for the convergence is the same as the condition for solving for $\dot{u}$, namely an invertible (and hence positive definite) Hessian.

Because $\frac{\partial f_i}{\partial t} = W_{i,\text{in}} \dot{r}_{\text{in}} + \beta \delta_{i,o} \dot{u}_o^*$, see Eq. 47, the regularity requirement for $\tau \dot{u}$ to be integrable is satisfied even if $\bar{r}_{\text{in}}(t)$ and $u_o^*(t)$ contain step functions (and $r_{\text{in}}(t)$ delta-functions), see Sect. D for details.

Even in the presence of such step-functions, the Euler-Lagrange equations $(1 + \tau \frac{d}{dt}) \mathbf{f} = 0$ lead to an $\mathbf{f}$ that is a decaying exponential, $f_i(\mathbf{u}(t), t) = c_i e^{-\frac{t-t_0}{\tau}}$, see Eq. 45. For initialization at $t_0 = -\infty$ we have $\mathbf{f} = \frac{\partial L}{\partial \mathbf{u}} = 0$ at any time. In fact, Eq. 52 is equivalent to $(1 + \tau \frac{d}{dt}) \mathbf{f} = 0$ and hence any solution of Eq. 52, even if $\frac{\partial \mathbf{f}}{\partial t}$ contains a delta-function, is also a solution of $(1 + \tau \frac{d}{dt}) \mathbf{f} = 0$. Possible jumps in $\bar{\mathbf{r}}_{\text{in}}$ or $\mathbf{u}_o^*$ are compensated by the jumps they induce in $\mathbf{u}$ (see below for the full mathematical description with a simple example). To give an intuition, we assume that a recurrent network of our prospective neurons has separate fixed points for the two constant input currents $\mathbf{r}_{\text{in}}^{(1)}$ and $\mathbf{r}_{\text{in}}^{(2)}$. This network still shows an overall relaxation time of τ_{in} (but not longer!) when the input rates instantaneously switch from $\mathbf{r}_{\text{in}}^{(1)}$ to $\mathbf{r}_{\text{in}}^{(2)}$. Nevertheless, at any moment during this relaxation process, gradient learning of the mapping $\mathbf{u}_o(t) = \mathbf{F}_o(\bar{\mathbf{r}}_{\text{in}}(t))$ towards $\mathbf{u}_o^*(t) = \mathbf{F}_o^*(\bar{\mathbf{r}}_{\text{in}}(t))$ is still guaranteed (Theorem 1).

Link to time-varying optimal control

The explicit differential equation, Eq. 52, is a special case of the one in Simonetto, Dall'Anese, *et al.*, 2020, (Eq. 20), where the function to be minimized (their f) can take a general (Lipschitz continuous) form (hence their f is our Lagrangian, $f \equiv L$). To avoid inverting the Hessian, an iteration algorithm can be applied similar to our implicit form, although more involved to deal with the more general form of L (Simonetto & Dall'Anese, 2017). The idea of tracking the solution of a time-varying optimization problem with a linear look-ahead in time has been introduced in Zhao & Swamy, 1998.

The NLA keeps stability as compared to an infinitesimally fast ($\tau \rightarrow 0$) Equilibrium Propagation

What have we gained as compared to making the time constant in Equilibrium propagation very small? The answer lies in the stability. To make the comparison explicit, we remind us of the Equilibrium Propagation which we formulate in terms of a Lagrangian. In both cases, the NLA and the Equilibrium Propagation, the Lagrangians are identical. We have chosen the notation $\bar{\mathbf{r}} = (\bar{\mathbf{r}}_{\text{in}}, \rho(\mathbf{u}))$ to express that the quantities can be interpreted as a low-pass filtering of $\mathbf{r} + \tau \mathbf{r}$. In the Equilibrium propagation, they are called $\mathbf{r} = (\mathbf{r}_{\text{in}}, \rho(\mathbf{u}))$, but this only represents a renaming of the input rates (and a notation change from $\bar{\mathbf{r}}$ to $\mathbf{r}$). Hence, the Lagrangian $L = \frac{1}{2} \|\mathbf{u} - \mathbf{W} \bar{\mathbf{r}}\|^2 + \beta C$ in the NLA with $\bar{\mathbf{r}} = (\bar{\mathbf{r}}_{\text{in}}, \rho(\mathbf{u}))$ is rewritten by $L = \frac{1}{2} \|\mathbf{u} - \mathbf{W} \mathbf{r}\|^2 + \beta C$ with $\mathbf{r} = (\mathbf{r}_{\text{in}}, \rho(\mathbf{u}))$.

The only true difference is that in the NLA we apply the combined lookahead operator $(1 + \tau \frac{d}{dt}) \frac{\partial}{\partial \mathbf{u}}$ to the Lagrangian L , while in the Equilibrium Propagation one applies the reduced operator $\frac{\partial}{\partial \mathbf{u}}$ to L . We claim that the lookahead makes the difference for 'living' systems that learnt to optimize in-time with respect to future quantities. The Euler-Lagrange equations for the Equilibrium Propagation reduce to $\frac{\partial L}{\partial \mathbf{u}} = \mathbf{u} - \mathbf{W} \mathbf{r} - \rho'(\mathbf{u}) \cdot \mathbf{W}^T \mathbf{e} - \beta \mathbf{e}^* = 0$, where now $\mathbf{e} = \mathbf{u} - \mathbf{W} \mathbf{r}$. Since these are not differential equations (no $\dot{\mathbf{u}}$ in there), one introduces the gradient descent dynamics $\tau \dot{\mathbf{u}} = -\frac{\partial L}{\partial \mathbf{u}} = -\mathbf{u} + \mathbf{W} \mathbf{r} + \rho'(\mathbf{u}) \cdot \mathbf{W}^T \mathbf{e} + \beta \mathbf{e}^*$ and waits for convergence for fixed input and fixed target. If $\tau = 0$ in both the NLA and the Equilibrium propagation, we obtain the same implicit and stationary equations in $\mathbf{u}$.

Because the equation is implicit in $\mathbf{u}$, it has to be solved first for $\mathbf{u}$. While the Equilibrium propagation suggests to keep the inputs constant until a fixed point is reached, the NLA dynamically moves along the trajectory of fixed points while the inputs are changing. In the Equilibrium propagation we cannot simply take the limit $\tau \rightarrow 0$ since then the dynamics either disappears (when τ remains on the left, $\tau \Delta \mathbf{u} \rightarrow 0$) or explodes (when τ is moved to the right, $\frac{dt}{\tau} \rightarrow \infty$), leading to either too small or too big jumps.

D. Contraction analysis and delta-function inputs

Contraction property

We next show when the voltage dynamics obtained from $(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = 0$ is locally contracting, i.e. when neighboring trajectories of the explicit differential equations, for given inputs and targets, locally converge. This is a different stability property as compared to the (global) convergence properties for the implicit differential equation stated in Eqs 45 and 49.

For the contraction analysis we rewrite Eq. 51 in the form

$\mathbf{G}(\mathbf{y}, \dot{\mathbf{u}}) = (1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = \mathbf{f} + \tau \dot{\mathbf{f}} = 0$, where the explicit time dependence of E and $\mathbf{f}$ is a short-cut to express the dependence on $\mathbf{y} = (\mathbf{r}_{\text{in}}, \mathbf{u}_o^*, \mathbf{u})$. According to the implicit function theorem, at any point in time when $\frac{\partial \mathbf{G}}{\partial \dot{\mathbf{u}}}$ is invertible, we can locally write $\dot{\mathbf{u}} = \mathbf{g}(\mathbf{y})$. When absorbing the dependence of $\dot{\mathbf{u}}$ on $(\mathbf{r}_{\text{in}}, \mathbf{u}_o^*)$ into an explicit time dependence, we can rewrite this differential equation as $\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, t)$, see Eq. 52, when explicitly expressing the dependency of $\mathbf{g}$ on $\mathbf{u}$ only (instead of all the variables $\mathbf{y}$). This differential equation is contractive and thus stable if the Jacobian of $\mathbf{g}$ with respect to $\mathbf{u}$ is uniformly negative definite (Lohmiller & Slotine, 1998). The contraction analysis tells that locally, where $\frac{\partial \mathbf{G}}{\partial \dot{\mathbf{u}}}$ is invertible, we can express $\dot{\mathbf{u}}$ as a function $\dot{\mathbf{u}} = \mathbf{g}(\mathbf{y})$, that has derivative $\frac{\partial \mathbf{g}}{\partial \mathbf{y}} = -(\frac{\partial \mathbf{G}}{\partial \dot{\mathbf{u}}})^{-1} \frac{\partial \mathbf{G}}{\partial \mathbf{y}}$ according to the implicit function theorem. Restricted to the $\mathbf{u}$ -component of $\mathbf{y}$ we get the Jacobian

$$\frac{\partial \mathbf{g}}{\partial \mathbf{u}} = -\left(\frac{\partial \mathbf{G}}{\partial \dot{\mathbf{u}}}\right)^{-1} \frac{\partial \mathbf{G}}{\partial \mathbf{u}} = -\frac{1}{\tau} \mathbf{1} - \mathbf{H}^{-1} \frac{\partial \mathbf{H}}{\partial \mathbf{u}} \dot{\mathbf{u}} = -\frac{1}{\tau} \mathbf{1} - \frac{d \log \mathbf{H}}{dt}. \quad (53)$$

To prove Eq. 53 we note that according to Eq. 51 we have

$\mathbf{G}(\mathbf{y}, \dot{\mathbf{u}}) = \mathbf{f} + \tau \cdot \mathbf{H}(\mathbf{u}) \dot{\mathbf{u}} + \tau \frac{\partial \mathbf{f}}{\partial t} = 0$, and with this calculate $\frac{\partial \mathbf{G}}{\partial \dot{\mathbf{u}}} = \tau \mathbf{H}$, with $H_{ij} = \frac{\partial f_i}{\partial u_j}$ specified above. Since according to Eq. 47 the term $\frac{\partial \mathbf{f}}{\partial t}$ does not depend on $\mathbf{u}$, we also calculate $\frac{\partial \mathbf{G}}{\partial \mathbf{u}} = \mathbf{H} + \tau \frac{\partial \mathbf{H}}{\partial \mathbf{u}} \dot{\mathbf{u}} = \mathbf{H} + \tau \frac{d \mathbf{H}}{dt}$. Hence,

$$\frac{\partial \mathbf{g}}{\partial \mathbf{u}} = -\frac{1}{\tau} \mathbf{H}^{-1} \left(\mathbf{H} + \tau \frac{d \mathbf{H}}{dt} \right) = -\frac{1}{\tau} \mathbf{1} - \frac{d \log \mathbf{H}}{dt}.$$

If we had $\frac{\partial \mathbf{g}}{\partial \mathbf{u}} = -\frac{1}{\tau} \mathbf{1}$ alone, $\frac{\partial \mathbf{g}}{\partial \mathbf{u}}$ would be obviously negative definite, guaranteeing the local contraction property (Lohmiller & Slotine, 1998). To understand this statement, we consider the local representation of neighboring trajectories. The linear approximation of the voltage dynamics $\mathbf{u}(t)$, starting with $\mathbf{u}(t_o)$ in a dim- $\mathbf{u}$ -dimensional neighbourhood around some point $\mathbf{u}_o(t_o)$, is $\frac{d}{dt}(\mathbf{u} - \mathbf{u}_o) = \frac{\partial \mathbf{g}(\mathbf{u}_o, t)}{\partial \mathbf{u}}(\mathbf{u} - \mathbf{u}_o) = -\frac{1}{\tau}(\mathbf{u} - \mathbf{u}_o)$. While $\mathbf{u}(t)$ is the linear approximations of the original voltage dynamics starting at $\mathbf{u}(t_o)$, the trajectory $\mathbf{u}_o(t)$ is the true voltage dynamics starting at the ‘center’ $\mathbf{u}_o(t_o)$ of the neighbourhood. This shows the exponential local contraction of surrounding trajectories to $\mathbf{u}_o$ at times t around t_o , would we have $\frac{\partial \mathbf{g}}{\partial \mathbf{u}} = -\frac{1}{\tau} \mathbf{1}$.

The additional log-term in Eq. 53 [↗](#) may cause a local violation of this contraction property. However, the additional term in Eq. 53 [↗](#) becomes small for large or small voltages for which we assume that the curvature of the transfer function also becomes small, $\rho''(\mathbf{u}) \approx \mathbf{0}$. In fact, based on Eq. 48 [↗](#) we calculate $\frac{\partial \mathbf{H}}{\partial \mathbf{u}}$ being a function of $\rho''(\mathbf{u})$ that vanishes with vanishing $\rho''(\mathbf{u})$,

$$\frac{\partial \mathbf{H}(\mathbf{u})}{\partial \mathbf{u}} = -\mathbf{W}_{\text{net}} \text{diag}(\rho''(\mathbf{u})) - \frac{\partial^2}{\partial \mathbf{u}^2}(\bar{\epsilon} + \beta \bar{\epsilon}^*)(\mathbf{u}) = o(\rho''(\mathbf{u})). \quad (54)$$

Plugging Eq. 54 [↗](#) into Eq. 53 [↗](#) we conclude that $\frac{\partial \mathbf{g}}{\partial \mathbf{u}} \approx -\frac{1}{\tau} \mathbf{1}$ where the curvature of the transfer function vanishes, $\rho''(\mathbf{u}) \approx \mathbf{0}$. Yet, ρ'' may strongly deviate from 0 for intermediate values of $\mathbf{u}$. For a rectified linear unit (ReLU), $\rho(u) = u$ for $u \geq 0$ and 0 else, for instance, ρ'' is a delta-function at 0. Even if the deviation from $\rho''(\mathbf{u}) = \mathbf{0}$ in this case is on a set of measure 0, integrating across voltages $u = 0$ can cause a jump in the voltage dynamics, preventing a strict local contraction property everywhere. The contraction property, in the example of a ReLU, only holds almost everywhere. Because the ReLU is an important transfer function in practical applications, we elaborate on this example further.

Comparison of the NLA with the Latent Equilibrium

Here we point out that the appearance of a delta-function for ReLU's in the NLA may represent a technical challenge, that this specific challenge is tamed in the simpler version of the NLA principle formulated as Latent Equilibrium (Haider *et al.*, 2021), and that the Latent Equilibrium may represent a viable new approach for time-varying optimization in general.

The Euler-Lagrange equation for the presented NLA, $(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = 0$, Eq. 20 [↗](#), with Jacobian $\frac{\partial L}{\partial \mathbf{u}} = \mathbf{f}(\mathbf{u}, t) = \mathbf{u} - \mathbf{W} \bar{\mathbf{r}} - \bar{\epsilon} - \beta \bar{\epsilon}^*$, implies second-order derivatives with respect to the voltage $\mathbf{u}$. This is the case because the total derivative with respect to time, $\frac{d}{dt}$, applied to the error $\bar{\epsilon} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}}$ contained in $\mathbf{f}(\mathbf{u}, t)$, implies the expression $\frac{d}{dt} \bar{\mathbf{r}}_{\text{net}}' = \rho''(\mathbf{u}) \dot{\mathbf{u}}$, with ρ'' equivalent to $\frac{\partial^2}{\partial \mathbf{u} \partial \mathbf{u}}$. These second-order derivatives enter both in the implicit form (Eq. 46 [↗](#) and Eq. 51 [↗](#)) and the explicit differential equation (Eq. 52 [↗](#)). When integrating these equations across the components $u = 0$, a jump in u emerges that must be explicitly included in the simulations. It is the same jump that may transiently violate the contraction property as shown above (Eqs 53 [↗](#) and 54 [↗](#)), but does not violate the exponential convergence from Eq. 45 [↗](#), see below).

In the Latent Equilibrium, our Lagrangian $L = \frac{1}{2} \|\mathbf{u} - \mathbf{W} \bar{\mathbf{r}}\|^2 + \beta C$ with $\bar{\mathbf{r}} = (\bar{\mathbf{r}}_{\text{in}}, \rho(\mathbf{u}))$ is replaced by $L(\mathbf{u}, \dot{\mathbf{u}}) = \frac{1}{2} \|\dot{\mathbf{u}} - \mathbf{W} \mathbf{r}\|^2 + \beta C$ with $\mathbf{r} = (\mathbf{r}_{\text{in}}, \rho(\dot{\mathbf{u}}))$ and $\dot{\mathbf{u}} = \mathbf{u} + \tau \dot{\mathbf{u}}$. If we were asking for stationary trajectories $\mathbf{u}(t)$ of $\int L(\mathbf{u}, \dot{\mathbf{u}}) dt$ with respect to variations in $\mathbf{u}$, we would obtain the unstable Euler-Lagrange equations $\frac{\partial L}{\partial \mathbf{u}} - \frac{d}{dt} \frac{\partial L}{\partial \dot{\mathbf{u}}} = (1 - \tau \frac{d}{dt}) \frac{\partial L}{\partial \mathbf{u}} = 0$, solved by an exponentially growing function of time, $\frac{\partial L}{\partial \mathbf{u}} = c e^{\frac{t-t_0}{\tau}}$, unless $c = 0$. To force the trajectory moving along $\frac{\partial L}{\partial \mathbf{u}} = 0$, one again requires that variations of the action $L dt$ are stationary with respect to a prospective voltage, this time with respect to $\dot{\mathbf{u}} = \mathbf{u} + \tau \dot{\mathbf{u}}$. Because the Lagrangian for the Latent Equilibrium can be expressed as a function $L(\dot{\mathbf{u}})$ of $\dot{\mathbf{u}}$ only, not of $\dot{\mathbf{u}}$, the Euler-Lagrange equations becomes $\frac{\partial L}{\partial \dot{\mathbf{u}}} = \dot{\mathbf{u}} - \mathbf{W} \mathbf{r} - \rho'(\dot{\mathbf{u}}) \cdot \mathbf{W}^T \bar{\mathbf{e}} - \beta \bar{\epsilon}^* = 0$, where now $\bar{\mathbf{e}} = \dot{\mathbf{u}} - \mathbf{W} \mathbf{r}$.

The Latent Equilibrium is consistent with the central idea of the NLA to deal with future quantities. While the presented version of the NLA deals with the discounted future voltage $\tilde{\mathbf{u}}$ (leading to the operator $(1 + \tau \frac{d}{dt}) \frac{\partial}{\partial \mathbf{u}}$ applied to the Lagrangian), the Latent Equilibrium deals with the linear approximation $\tilde{\mathbf{u}}$ of the future voltage (leading to the operator $\frac{\partial}{\partial \tilde{\mathbf{u}}}$ applied to the Lagrangian). Both, the NLA and the Latent Equilibrium differ in this crucial aspect from the Equilibrium Propagation that deals with the current voltage $\mathbf{u}$ (leading to the operator $\frac{\partial}{\partial \mathbf{u}}$ applied to the Lagrangian $L(\mathbf{u})$, without appearance of $\dot{\mathbf{u}}$, and hence without intrinsic dynamics, see above).

For the Latent Equilibrium, it is not obvious to find global convergence properties as expressed in Eqs 45 and 49 for the presented version of the NLA. However, the Latent Equilibrium satisfies the strict local contraction property $\frac{\partial \mathbf{g}}{\partial \mathbf{u}} = -\frac{1}{\tau} \mathbf{1}$. In fact, writing the Euler-Lagrange equations as $\mathbf{G}(\mathbf{y}, \dot{\mathbf{u}}) = \frac{\partial L}{\partial \tilde{\mathbf{u}}} = 0$ with the abbreviation $\mathbf{y} = (\mathbf{r}_{\text{in}}, \dot{\mathbf{r}}_{\text{in}}, \mathbf{u}_o^*, \dot{\mathbf{u}}_o^*, \mathbf{u})$, leads to $\frac{\partial \mathbf{G}}{\partial \tilde{\mathbf{u}}} = \frac{\partial \mathbf{G}}{\partial \mathbf{u}} = \frac{1}{\tau} \frac{\partial \mathbf{G}}{\partial \tilde{\mathbf{u}}}$, and the linear approximation obtained from plugging these partial derivatives into Eq. 53 yields the claimed contraction property $\frac{\partial \mathbf{g}}{\partial \mathbf{u}} = -\frac{1}{\tau} \mathbf{1}$ for the Latent Equilibrium. The Jacobian $\frac{\partial \mathbf{G}}{\partial \tilde{\mathbf{u}}}$ is invertible if the Hessian $\mathbf{H} = \frac{\partial^2 L}{\partial \tilde{\mathbf{u}} \partial \tilde{\mathbf{u}}}$ is (that looks as in Eq. 48, but with $\mathbf{u}$ replaced by $\tilde{\mathbf{u}}$).

Crucially, for the Latent Equilibrium, the analogous theorems on real-time dendritic error propagation and learning (rt-DeEP and rt-DeEL) hold. The key property is that the trajectories of the Latent Equilibrium are stationary solution of the corresponding action $A = \int L dt$, so that the variation of A induced by $\mathbf{W}$ reduces to the partial derivative of L with respect to $\mathbf{W}$.

The above strict contraction property that for non-vanishing errors only holds for the Latent Equilibrium, but not for the NLA, may be a reason why the simulations of the error-based updates in our hands seem to be more stable for the Latent Equilibrium as compared to the NLA. This does not withstand the fact that the NLA equations are used in the context of time-varying optimal control (see above). It would be interesting to consider also the Latent Equilibrium as a tool for non-stationary, time-varying optimization (Simonetto, Dall'Anese, *et al.*, 2020).

Delta-function inputs keep $\frac{\partial L}{\partial \mathbf{u}} = 0$

We next explain in more details why delta-functions in the input rates $\mathbf{r}_{\text{in}}(t)$ for the NLA the stationarity ('equilibrium') condition $\frac{\partial L}{\partial \mathbf{u}} = 0$ is always satisfied (the delta-function in $\mathbf{r}_{\text{in}}$ for the NLA corresponding to step-function in $\mathbf{r}_{\text{in}}$ for the Latent Equilibrium). We reconsider the explicit differential equation $\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, t) = -\frac{1}{\tau} \mathbf{H}^{-1} \left(\mathbf{f} + \tau \frac{\partial \mathbf{f}}{\partial t} \right)$ given in Eq. 52, with $\frac{\partial \mathbf{f}}{\partial t} = \frac{\partial \mathbf{f}}{\partial \tilde{\mathbf{r}}_{\text{in}}} \dot{\tilde{\mathbf{r}}}_{\text{in}} + \frac{\partial \mathbf{f}}{\partial \tilde{\mathbf{r}}_{\text{in}}} \dot{\mathbf{u}}_o^* = \mathbf{W}_{\text{in}} \delta_{\text{in}}(t)$ and $\mathbf{H}$ given in Eq. 48.

To simplify matters, we consider a single delta-function at $t = 0$ as input in the absence of output nudging. In this case we get $\frac{\partial \mathbf{f}}{\partial t} = \frac{\partial \mathbf{f}}{\partial \tilde{\mathbf{r}}_{\text{in}}} \dot{\tilde{\mathbf{r}}}_{\text{in}} + \frac{\partial \mathbf{f}}{\partial \tilde{\mathbf{r}}_{\text{in}}} \dot{\mathbf{u}}_o^* = \mathbf{W}_{\text{in}} \delta_{\text{in}}(t)$, where the input matrix $\mathbf{W}_{\text{in}}$ is typically sparse (not all network neurons receive external input), and $\delta_{\text{in}}(t)$ is a vector of delta-functions restricted to the input neurons. Following Nedeljkov & Oberguggenberger, 2012, Proposition 2.1, we can then write the explicit differential equation, Eq. 51, in the form

$$\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, t) = \check{\mathbf{g}}(\mathbf{u}, t) + \mathbf{H}(\mathbf{u})^{-1} \mathbf{W}_{\text{in}} \delta_{\text{in}}(t), \quad (55)$$

where $\check{g}(\mathbf{u}, t)$ is globally Lipschitz continuous. Due to the Lipschitz continuity the change in $\mathbf{u}$ evoked by $\check{g}(\mathbf{u}, t)$ during a small time interval $[\varepsilon, -\varepsilon]$ around $t = 0$ vanishes when this interval shrinks, $\varepsilon \rightarrow 0$. To quantify the change in $\mathbf{u}$ during these intervals it is therefore enough to consider $\dot{\mathbf{u}} = \mathbf{H}(\mathbf{u})^{-1} \mathbf{W}_{\text{in}} \delta_{\text{in}}(t)$, or equivalently $\mathbf{H}(\mathbf{u}) \dot{\mathbf{u}} = \mathbf{W}_{\text{in}} \delta_{\text{in}}(t)$. To estimate the jump induced by the delta-functions, we consider some mollifier $\phi_\varepsilon(t) = \varepsilon^{-1} \varphi(t/\varepsilon)$, where $\varphi(t)$ is a smooth function on the interval $[1, -1]$ with integral 1. By $\phi_{\text{in},\varepsilon}(t)$ we denote the vector of mollifiers centered at the delta-functions of the input neurons. We now consider the two differential equations, with the second approximating the first on the interval $[-\varepsilon, \varepsilon]$, but without regular term $\check{g}(\mathbf{u}, t)$,

$$\tau \mathbf{H}(\mathbf{u}_\varepsilon) \dot{\mathbf{u}}_\varepsilon = \tau \mathbf{H}(\mathbf{u}_\varepsilon) \check{g}(\mathbf{u}, t) + \mathbf{W}_{\text{in}} \phi_{\text{in},\varepsilon}(t) \quad (56a)$$

$$\tau \mathbf{H}(\check{\mathbf{u}}_\varepsilon) \dot{\check{\mathbf{u}}}_\varepsilon = \mathbf{W}_{\text{in}} \phi_{\text{in},\varepsilon}(t), \quad \check{\mathbf{u}}_\varepsilon(-\varepsilon) = \mathbf{u}_\varepsilon(-\varepsilon). \quad (56b)$$

We assume that for all $t \in [-\varepsilon, \varepsilon]$ the matrices $\mathbf{H}(\mathbf{u}_\varepsilon(t))$ and $\mathbf{H}(\check{\mathbf{u}}_\varepsilon(t))$ are invertible, so that the two Eqs 56a, 56b can be turned into an explicit differential equations. Analogously to the 1-dimensional case (Nedeljkov & Oberguggenberger, 2012), we conclude that the solution of Eqs 56a, 56b on the interval $[-\varepsilon, \varepsilon]$ converge to each other, $\sup_{t \in [-\varepsilon, \varepsilon]} |\mathbf{u}_\varepsilon(t) - \check{\mathbf{u}}_\varepsilon(t)| \rightarrow 0$ for $\varepsilon \rightarrow 0$. As a consequence, the jump of $\check{\mathbf{u}}_\varepsilon$ at $t = 0$ converges to the corresponding jump of $\mathbf{u}_\varepsilon$ in the various dimensions.

To calculate the jump of $\check{\mathbf{u}}_\varepsilon$ we have to integrate $\tau \mathbf{H}(\check{\mathbf{u}}_\varepsilon) \dot{\check{\mathbf{u}}}_\varepsilon$ across the time interval $[-\varepsilon, \varepsilon]$. Instead of integrating $\dot{\check{\mathbf{u}}}_\varepsilon$, we first integrate $\tau \mathbf{H}(\check{\mathbf{u}}_\varepsilon) \dot{\check{\mathbf{u}}}_\varepsilon$ given in Eq. 56b. Moving from right to left yields

$$\mathbf{W}_{\text{in}} \mathbf{I}_{\text{in}} = \int_{-\varepsilon}^{\varepsilon} \mathbf{W}_{\text{in}} \phi_{\text{in},\varepsilon}(t) dt = \tau \int_{-\varepsilon}^{\varepsilon} \mathbf{H}(\check{\mathbf{u}}_\varepsilon(t)) \dot{\check{\mathbf{u}}}_\varepsilon(t) dt = \tau \int_{\check{\mathbf{u}}_\varepsilon(-\varepsilon)}^{\check{\mathbf{u}}_\varepsilon(\varepsilon)} \mathbf{H}(\check{\mathbf{u}}_\varepsilon) d\check{\mathbf{u}}_\varepsilon, \quad (57)$$

where $\mathbf{I}_{\text{in}}$ is the index vector of the input neurons having a delta-function, i.e. $I_{\text{in},j} = 1$ if there is a delta-input, and else 0. Note that $\dot{\check{\mathbf{u}}}_{\varepsilon,j} dt = dv_{\varepsilon,j}$. Because $\mathbf{H}$ is itself a derivative, $\mathbf{H} = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}$, we can explicitly calculate the latter integral (also for $\beta > 0$, but for clarity here only done for $\beta = 0$). The last integral in Eq. 57 is defined as a vector with i -th component being

$$\left(\int_{\check{\mathbf{u}}_\varepsilon(-\varepsilon)}^{\check{\mathbf{u}}_\varepsilon(\varepsilon)} \mathbf{H}(\check{\mathbf{u}}_\varepsilon) d\check{\mathbf{u}}_\varepsilon \right)_i = \sum_{j=1}^N \int_{\check{u}_{\varepsilon,j}(-\varepsilon)}^{\check{u}_{\varepsilon,j}(\varepsilon)} H_{ij}(u_j) du_j = \sum_{j=1}^N (\delta_{ij} u_i - W_{ij} \rho(u_j)) \Big|_{\check{u}_{\varepsilon,j}(-\varepsilon)}^{\check{u}_{\varepsilon,j}(\varepsilon)} = (v_\varepsilon(\varepsilon) - v_\varepsilon(-\varepsilon))_i, \quad (58)$$

where in the second last equality we used that $H_{ij}(\mathbf{u}) = \delta_{ij} - W_{ij} \rho'(u_j)$ does only depend on the component u_j , see Eq. 48. In the last equality we introduced the ‘network voltage error’ $\check{\mathbf{v}}_\varepsilon = \check{\mathbf{u}}_\varepsilon - \mathbf{W}_{\text{net}} \rho(\check{\mathbf{u}}_\varepsilon)$. Following the 1-dimensional case treated in Nedeljkov & Oberguggenberger, 2012, Proposition 1.2 we introduce the ‘jump function’ (called $G(y)$ in the cited work)

$$\mathbf{J}(\check{\mathbf{u}}_\varepsilon) = \tau (\check{\mathbf{v}}_\varepsilon - \check{\mathbf{v}}_0), \quad (59)$$

with $\check{\mathbf{v}}_0$ thought to represent $\check{\mathbf{v}}_\varepsilon(t_0)$ at some time t_0 before the delta-kick sets in. With this setting, Eq. 57 turns into

$$\mathbf{W}_{\text{in}} \mathbf{I}_{\text{in}} = \mathbf{J}(\check{\mathbf{u}}_\varepsilon(\varepsilon)) - \mathbf{J}(\check{\mathbf{u}}_\varepsilon(-\varepsilon)). \quad (60)$$

In the limit $\varepsilon \rightarrow 0$ we get a relation between $\mathbf{u}(t)$ immediately before and after the jump, $\mathbf{u}(-0)$ and $\mathbf{u}(+0)$, using that in this limit the boundary points of the trajectories also converge, $\tilde{\mathbf{u}}_\varepsilon(\pm\varepsilon) \rightarrow \mathbf{u}(\pm 0)$,

$$\mathbf{J}(\mathbf{u}(+0)) = \mathbf{J}(\mathbf{u}(-0)) + \mathbf{W}_{\text{in}} \mathbf{I}_{\text{in}}. \quad (61)$$

We now assume that the function $\mathbf{J}(\mathbf{u}) = \tau(\mathbf{v} - \mathbf{v}_0)$ is invertible around the jump. This is the case if the Jacobian $\frac{\partial \mathbf{J}(\mathbf{u})}{\partial \mathbf{u}}$ is invertible, and because $\mathbf{v} = \mathbf{u} - \mathbf{W}_{\text{net}} \rho(\mathbf{u})$, we require invertability of $\frac{\partial \mathbf{J}(\mathbf{u})}{\partial \mathbf{u}} = \tau \mathbf{H}(\mathbf{u})$, with Hessian defined in Eq. 48.

In the case of invertability we get the voltage after the jump as

$$\mathbf{u}(+0) = \mathbf{J}^{-1}(\mathbf{J}(\mathbf{u}(-0)) + \mathbf{W}_{\text{in}} \mathbf{I}_{\text{in}}). \quad (62)$$

We next calculate the jump in $\mathbf{v}$. This is easy since $\mathbf{v} = \mathbf{u} - \mathbf{W}_{\text{net}} \rho(\mathbf{u})$ linearly enters in the function $\mathbf{J}(\mathbf{u}) = \tau(\mathbf{v} - \mathbf{v}_0)$. Plugging the explicit expression for $\mathbf{J}$ into Eq. 61 we get $\tau(\mathbf{v}(+0) - \mathbf{v}_0) = \tau(\mathbf{v}(-0) - \mathbf{v}_0) + \mathbf{W}_{\text{in}} \mathbf{I}_{\text{in}}$, or

$$\mathbf{v}(+0) = \mathbf{v}(-0) + \frac{1}{\tau} \mathbf{W}_{\text{in}} \mathbf{I}_{\text{in}}. \quad (63)$$

Knowing the jump in $\mathbf{v}$ helps to show that the equilibrium condition $\frac{\partial L}{\partial \mathbf{u}} = 0$ is always satisfied, even immediately after the delta-in put, provided the initialization is at $t_0 = -\infty$. To show this, remember that in the absence of nudging we have

$\frac{\partial L}{\partial \mathbf{u}} = \mathbf{f}(\mathbf{u}, t) = \mathbf{u} - \mathbf{W} \bar{\mathbf{r}} = \mathbf{u} - \mathbf{W}_{\text{net}} \rho(\mathbf{u}) - \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}} = \mathbf{v} - \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}}$. The jump size of $\bar{\mathbf{r}}_{\text{in}}$ for a delta-function at $t = 0$, $\mathbf{r}_{\text{in}}(t) = \delta_{\text{in}}(t)$ is $\frac{1}{\tau}$. This is because $\bar{\mathbf{r}}_{\text{in}}$ satisfies the differential equations $\dot{\bar{\mathbf{r}}}_{\text{in}}(t) = -\frac{1}{\tau} \bar{\mathbf{r}}_{\text{in}}(t) + \frac{1}{\tau} \delta_{\text{in}}(t)$, provided that $t_0 = -\infty$. Hence,

$$\bar{\mathbf{r}}_{\text{in}}(+0) = \bar{\mathbf{r}}_{\text{in}}(-0) + \frac{1}{\tau} \mathbf{I}_{\text{in}}. \quad (64)$$

With Eqs 63 and 64 we conclude that $\frac{\partial L}{\partial \mathbf{u}} = \mathbf{v} - \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}} = 0$ throughout.

E. Example of a single recurrently connected neuron

To get an intuition for the instantaneity in a the recurrent case we consider the example of a single, recurrently connected neuron. We also put this into the context of the Latent Equilibrium (Haider *et al.*, 2021). Consider the weight vector $\mathbf{W} = (\mathbf{W}_{\text{in}}, \mathbf{W}_{\text{net}})$ with an input an input rate $r_{\text{in}}(t)$ driving the postsynaptic voltage u . The postsynaptic rate is $r = \rho(u) + \tau \dot{\rho}(u)$, and its low-pass filter with respect to τ is $\bar{r} = \rho(u)$. As always, the low-pass filtering reaches back to an initialization at $t_0 = -\infty$, see Eq. 15. The Lagrangian has the form

$$L = \frac{1}{2} \bar{e}^2 + \frac{\beta}{2} \bar{e}^{*2} = \frac{1}{2} (u - (\mathbf{W}_{\text{net}} \rho(u) + \mathbf{W}_{\text{in}} \bar{\mathbf{r}}_{\text{in}}))^2 + \frac{\beta}{2} (u^* - u)^2. \quad (65)$$

The Euler-Lagrange equations $(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial u} = 0$ are derived from $\frac{\partial L}{\partial u} = \bar{e} - \rho'(u)W_{\text{net}}\bar{e} - \beta\bar{e}^*$. Applying the look-ahead operator $(1 + \tau \frac{d}{dt})$ (Eq. 14 [↗](#)), and abbreviating $\bar{e} = \rho'(u)W_{\text{net}}\bar{e}$, the Euler-Lagrange equations deliver the voltage dynamics,

$$\tau \dot{u} = -u + W_{\text{net}}(\rho(u) + \tau \dot{\rho}(u)) + W_{\text{in}}r_{\text{in}} + \epsilon + \beta e^* . \quad (66)$$

To simplify matters, we consider the nudging-free case, $\beta = 0$. This implies that $\bar{e} = \bar{e} = 0$. With $\dot{\rho}(u) = \rho'(u)\dot{u}$, we obtain the differential equation

$$\tau (1 - W_{\text{net}}\rho'(u)) \dot{u} = -u + W_{\text{net}}\rho(u) + W_{\text{in}}r_{\text{in}} . \quad (67)$$

Abbreviating $v = u - W_{\text{net}}\rho(u)$ as ‘network voltage error’, the above differential equation reads as

$$\tau \dot{v} = -v + W_{\text{in}}r_{\text{in}} . \quad (68)$$

Integrating the effective voltage dynamics (Eq. 68 [↗](#)), assuming initialization infinitely far in the past, is equal to $v = W_{\text{in}}\bar{r}_{\text{in}}$. This equation is equivalent to the Euler-Lagrange equation $(1 + \tau \frac{d}{dt}) \frac{\partial L}{\partial u} = 0$ being integrate, and because the solution of the Euler Lagrange equation is $\frac{\partial L}{\partial u} = c e^{-\frac{t-t_0}{\tau}}$, we have (using $t = -\infty$)

$$\frac{\partial L}{\partial u} = v - W_{\text{in}}\bar{r}_{\text{in}} = 0 . \quad (69)$$

Voltage dynamics for a delta-function input

We next apply a delta-function in the input rate, say $r_{\text{in}}(t) = \delta(t)$ and consider the dynamics at the level of the voltage, Eq. 67 [↗](#). As in [Nedeljkov & Oberguggenberger, 2012 \[↗\]\(#\)](#), Proposition 1.2 we introduce the ‘jump function’ (called $G(y)$ in the cited work)

$$J(u) = \tau \int_{u_0}^u (1 - W_{\text{net}}\rho'(y)) dy = \tau \left((u - W_{\text{net}}\rho(u)) - v_0 \right) = \tau(v - v_0) . \quad (70)$$

As in [Nedeljkov & Oberguggenberger, 2012 \[↗\]\(#\)](#), Proposition 1.2, we show that the voltage u makes a unique jump at the moment of the delta function that $J(u)$ is invertible around the jump.

We set $v_0 = u_0 - W_{\text{net}}\rho(u_0)$. Here, u_0 is some voltage before the jump, say $u_0 = u(-1)$ evaluated at time $t = -1$, when the jump is at $t = 0$. When $u_0^- = u(-0)$ is the voltage immediately before the jump, the voltage immediately after the jump is $u_0^+ = u(+0)$ specified by

$$J(u_0^+) = J(u_0^-) + W_{\text{in}} . \quad (71)$$

The reason is that the W_{in} -scaled delta-function triggers a step of size W_{in} when integrating over it as done in Eq. 70 [↗](#). The new value u_0^+ is unique if J is invertible, and looking at the defining integral in Eq. 70 [↗](#), this is the case if $1 - W_{\text{net}}\rho'(u_0^+) \neq 0$.

The jump in u translates into a jump in $v = u - W_{\text{net}}\rho(u)$ from v_0^- to $v_0^+ = u_0^+ - W_{\text{net}}\rho(u_0^+)$. This endpoint can also be expressed as

$$v_0^+ = v_0^- + \frac{W_{\text{in}}}{\tau} . \quad (72)$$

To check this, we assume without loss of generality that $v_0 = u_0 - W_{\text{net}}\rho(u_0) = 0$. Then $J(u) = \tau(u - W_{\text{net}}\rho(u)) = \tau v$ and $J(u_0^+) = J(u_0^-) + W_{\text{in}} = \tau v_0^- + W_{\text{in}}$ according to Eq. 71. Since also $J(u_0^+) = \tau v_0^+$, we conclude that $\tau v_0^+ = \tau v_0^- + W_{\text{in}}$, as claimed above.

We finally show that even far away from the initialization, the stationarity condition $\frac{\partial L}{\partial u} = 0$ holds before and immediately after the jump. In fact, for $t_0 = -\infty$, the evolution of the ‘network voltage error’ becomes

$$v(t) = 0 \text{ for } t \leq 0, \quad \text{and} \quad v(t) = \frac{W_{\text{in}}}{\tau} e^{-\frac{t}{\tau}} \text{ for } t > 0. \quad (73)$$

Here we used that $v_0^- = 0$ and according to Eq. 72 the v jumps to $v_0^+ = \frac{W_{\text{in}}}{\tau}$. Remember that for initialization far in the past, $\frac{\partial L}{\partial u} = 0$ is equivalent to $v = W_{\text{in}}\bar{r}_{\text{in}}$, see Eq. 69. We therefore have to calculate the jump in $\bar{r}_{\text{in}}$ induced by the delta-input. Since $\bar{r}_{\text{in}}(t)$ is the solution of $\tau \dot{\bar{r}}_{\text{in}}(t) = -\bar{r}_{\text{in}}(t) + \delta(t)$ for $t_0 = -\infty$, we find that

$$\bar{r}_{\text{in}}(t) = 0 \text{ for } t \leq 0, \quad \text{and} \quad \bar{r}_{\text{in}}(t) = \frac{1}{\tau} e^{-\frac{t}{\tau}} \text{ for } t > 0. \quad (74)$$

Combining the two Eqs 73 and 74 proves that $\frac{\partial L}{\partial u} = v - W_{\text{in}}\bar{r}_{\text{in}} = 0$ holds true any moment in time, provided the initialization is far in the past.

One may ask why the delta-kink is different from resetting v at a new initialization off from 0. The reason is that at $t = 0$ there is a cause for the jump in $r(0)$, while at t_0 there is no cause in $r(t_0)$. In fact, there is no jump initially, just the start of v at some initial condition. Differently from the initialization at t_0 , where $v(t_0) > 0$ implies $\frac{\partial L}{\partial u}(t) > 0$ for finite $t - t_0 > 0$, the jump of $v(0)$ at $t = 0$ to a positive value does leave $\frac{\partial L}{\partial u}(t) = 0$ for all $t > 0$, provided $t_0 = -\infty$.

Linear transfer function

We first consider the case of a linear transfer-function $\rho(u) = 0$ (or threshold linear, being in the linear regime). Then the differential equation becomes

$$\tau \dot{u} = -u + \frac{W_{\text{in}}}{1 - W_{\text{net}}} r_{\text{in}}. \quad (75)$$

With initialization at $t = -\infty$ and low-pass filtering $\bar{r}_{\text{in}}^\tau$ defined in Eq. 15 the solution is

$$u(t) = \frac{W_{\text{in}}}{1 - W_{\text{net}}} \bar{r}_{\text{in}}^\tau(t). \quad (76)$$

The point is that the time constant is τ and is not $\frac{\tau}{1 - W_{\text{net}}}$, as this would be the case without prospective firing rate. In fact, for the ‘classical’ differential equation,

$$\tau \dot{u} = -u + W_{\text{net}}\rho(u) + W_{\text{in}}r_{\text{in}}, \quad (77)$$

and ρ the identity, we obtain the differential equation $\tau_{\text{eff}} \dot{u} = -u + \frac{W_{\text{in}}}{1 - W_{\text{net}}} r_{\text{in}}$ with

$\tau_{\text{eff}} = \frac{\tau}{1 - W_{\text{net}}}$ and solution $u = \frac{W_{\text{in}}}{1 - W_{\text{net}}} \bar{r}_{\text{in}}^{\tau_{\text{eff}}}$ that is now the low-pass filtering with respect to the effective time constant.

Sigmoidal transfer function

For a sigmoidal transfer function $\rho(u) = \frac{\bar{r}_{\max}}{1 + e^{\frac{u}{\theta}}}$, a positive feedback weight $W_{\text{net}} > 0$, and a constant external input r_{in} , say, the solutions $u(t)$ of Eq. 66 either converge to a fixed point or diverge. When converging, the voltage satisfies the fixed point condition

$$u = W_{\text{net}}\rho(u) + W_{\text{in}}r_{\text{in}}. \quad (78)$$

This fixed point equation can be numerically solved by time-discrete iteration process. But it can also be solved by a time-continuous process that underlies a neural or neuromorphic implementation. The prospective firing rate introduced in the NLA can be seen as a method to quickly find the fixed point in continuous time. When directly solving the implicit differential equation (as opposed to convert this into an explicit differential equation using e.g. the Cholesky decomposition), the fixed point is potentially found with a fewer number of steps.

To estimate the speed of convergence, we look at the initial speed when taking off at initial condition $u(0)$ between the unstable and stable fixed point. The initial speed for the classical differential equation, Eq. 77, and the NLA version, Eq. 66, are, respectively,

$$\dot{u}(0) = \frac{\Delta u(0)}{\tau}, \quad (79a)$$

$$\dot{u}(0) = \frac{\Delta u(0)}{\tau(1 - W_{\text{net}}\rho'(u(0)))}, \quad (79b)$$

where we set $\Delta u(0) = u(0) + W_{\text{net}}\rho(u(0)) + W_{\text{in}}r_{\text{in}}(0)$. As $W_{\text{net}}\rho' > 0$, the initial convergence speed of the NLA solution is larger. The scheme has some resemblance to the Newton algorithm of finding zero's of a function by using its derivative.

F. NLA for conductance-based neurons and least-action principle in physics

The mismatch energies and costs can be generalized in different ways. Here we focus on a biophysical version of the mismatch energy that includes conductance-based neurons. This also relates to the least-action principle in physics. But the NLA can also be generalized to include other dynamical variables such as adaptive thresholds or synaptic short-term plasticity.

Equivalent somato-dendritic circuit

For conductance based synapses, the excitatory and inhibitory conductances, g_E and g_I , are driven by the presynaptic firing rates and have the form $g_E(t) = W_E r(t)$, and analogously $g_I(t) = W_I r(t)$. The dynamics of a somatic voltage u and a dendritic voltage v reads as

$$c \dot{u} = g_L(E_L - u) + g_{\text{sd}}(v - u) \quad (80)$$

$$c_d \dot{v} = g_L(E_L - v) + g_E(E_E - v) + g_I(E_I - v) + g_{\text{ds}}(u - v), \quad (81)$$

where c and c_d are the somatic and dendritic capacitances, $E_{L/E/I}$ the reversal potentials for the leak, the excitatory and inhibitory currents, g_{sd} the transfer conductance from the dendrite to the soma, and g_{ds} in the other direction.

We consider the case when the dendritic capacitance c_d is small as compared to the sum of conductances g_d on the right-hand-side of Eq. 81, yielding a fast dendritic time constant. In this case we can solve this equation in the steady state for v , plug this into Eq. 80, and get

$$c \dot{u} = g(V - u), \quad (82)$$

with effective reversal potential $V = (g_L E_L + g_{sd} \frac{g_{ff}}{g_{ff} + g_{ds}} v_{ff})/g$, total conductance $g = g_L + g_{sd} \frac{g_{ff}}{g_{ff} + g_{ds}}$, feedforward dendritic voltage $v_{ff} = (g_L E_L + g_E E_E + g_I E_I)/g_{ff}$ and feedforward dendritic conductance $g_{ff} = g_L + g_E + g_I$. Because $g_{E/I} = W_{E/I} r(u_{pre}, \dot{u}_{pre})$, the conductance depends on the presynaptic voltage and its derivative. Equation 82 describes the effective circuit that has the identical voltage time course as Eqs 80 and 81 with $\dot{v} = 0$, but with a single time-dependent ‘battery voltage’ V and Ohmic resistance $R = 1/g$.

Somato-dendritic mismatch power and action

The synaptic inputs $g_E(t)$ and $g_I(t)$ are continuously driving $V(t)$, and the best what one can hope for the dynamics of u is that it traces V with some integration delay determined by the time constant $\tau = c/g$. In fact, if u follows the dynamics of Eq. 82, then u becomes the low-pass filtered target potential, $u = \bar{V}$, where $V(t)$ is filtered with the dynamic time constant $\tau(t)$. The defining equations for the low-pass filtering is

$$\bar{u} + \tau \dot{\bar{u}} = u, \quad (83)$$

and this self-consistency equation is equivalent to the explicit form

$$\bar{u}(t) = \int_{-\infty}^t dt' \frac{u(t')}{\tau(t')} e^{-\int_{t'}^t dt'' \frac{1}{\tau(t'')}}. \quad (84)$$

To capture the voltage dynamics with our NLA principle we recall that the somatic voltage u can be nudged by an ‘apical voltage’ $\bar{e}$ that causes a somatic voltage error $\bar{e} = u - \bar{V}$. The voltage error drives a current $I = g\bar{e}$ through the conductance g . The electrical power of this current I driven by the voltage $\bar{e}$ is $P = I\bar{e}^2$. This motivates the definition of the mismatch power in a network of N neurons by

$$\mathcal{P} = \sum_i^N \frac{g_i}{2} (u_i - \bar{V}_i)^2 = \frac{1}{2} \mathbf{g}^T (\mathbf{u} - \bar{\mathbf{V}})^2. \quad (85)$$

$\mathcal{P}$ is a virtual power that, nevertheless, is related to some physical flow of ions. Assume we could measure all the ions flowing in the original circuit of Eqs 80 and 81 (in the limit of small ratio C_d/g_d). From this flow, delete the ion flow that cancels at the level of electrical charge exchange due to the counter directed flow. The remaining effective ion flow defines an effective current flowing through the conductance g with driving force $(V - u)$, Eq. 82. If it were only this effective current in the reduced circuit, the voltage $u(t)$, starting at $u(0)$, would converge with time-constant τ to the low-pass filtering $\bar{V}$. Without additional ‘hidden’ current, the voltage u would then instantaneously follow $\bar{V}$ that is itself given by the forward dendritic input conductances. The deviation of u from $\bar{V}$, caused by some initial conditions in u or by a feedback current from the network affecting u , builds the mismatch power $\mathcal{P}$. The feedback may originate from a target imposed downstream, and the neuron is ‘free’ in how to dynamically match u and $\bar{V}$. It is therefore tempting to see $\mathcal{P}$ as a ‘free power’, and the NLA principle as minimizing the

corresponding ‘free energy’. In fact, the free-energy principle says that any self-organizing system that is in a dynamic equilibrium with its environment (here $u = \bar{V}$ in the absence of output nudging) must minimize its free energy (that here builds up by imposing a target; Friston, 2010).

The NLA principle states that the time-integral of $\mathcal{P}$ is minimized with respect to the look-ahead voltage $\tilde{u}$. We therefore define the physical mismatch energy as

$$A = \int_{t_1}^{t_2} \mathcal{P} dt, \quad (86)$$

that has the units of energy. ϵ_M takes the role of our neural action (A in the main text) for conductance-based neurons.

Euler-Lagrange equations for conductance-based neurons

The NLA for conductance-based neurons seeks to minimize $A = \int \mathcal{A}(u) dt$ with respect to variations of u , such that $\frac{\delta A}{\delta u} = 0$. In the simplest example of the main text we considered prospective rates, $r = \rho(u) + \tau \dot{\rho}(u)$, so that the low-pass filtered rates become a function of the instantaneous voltage, $\bar{r} = \rho(u)$. These low-pass filtered presynaptic rates, $\bar{r}$, determine the postsynaptic voltage u . Analogously, the low-pass filtered reversal potential, $\bar{V}$, determines the postsynaptic voltage u , and we again postulate that V is an instantaneous function of the presynaptic voltage, $\bar{V} = \phi(u)$. Here we argue that active dendritic mechanisms advance to postsynaptic reversal potential V , so that the delayed $\bar{V}$ again becomes instantaneous, similarly to the advancement of the apical dendritic potential observed in cortical pyramidal neurons (Ulrich, 2002), see also Fig. 2b. With this instantaneity, the stationarity of the action with respect to generalized (compact and non-compact) variations, $\frac{\delta A}{\delta u} = 0$, translates to the condition $\frac{\partial \mathcal{P}}{\partial u} = 0$.

Calculating $\frac{\partial \mathcal{P}}{\partial u} = 0$ with $\mathcal{P}$ given in Eq. 85 and $\tau = c/g$ for the total conductance $g(u)$ specified after Eq. 82 is a bit more demanding. For a probabilistic version, where $\mathcal{P}$ is derived from the negative log-likelihood of a Gaussian density of the voltage, $\mathcal{P} = -\log p(u|\bar{V}) = \frac{1}{2} g^T (u - \bar{V})^2 - \frac{1}{2} \log g + \text{const}$, the calculation is done in Jordan *et al.*, 2022. In this probabilistic version, there is an additional normalization term that enters as $\log g$. In the deterministic version considered here, this log term is not present and we calculate

$$\frac{\partial \mathcal{P}}{\partial u} = g \cdot (u - \bar{V}) - \bar{\epsilon} \quad \text{with} \quad \bar{\epsilon} = \bar{r}' \cdot W_{\text{net}}^T \left((u - \bar{V}) \cdot (E^{E/I} - \bar{V}) - \frac{1}{2} (u - \bar{V})^2 \right). \quad (87)$$

Notice that the transpose W_{net}^T selects the downstream network neuron to backpropagate from there the first- and second-order errors. From $\frac{\partial \mathcal{P}}{\partial u} = 0$ and $\tau = c/g$ we conclude that

$$\frac{c}{\tau} \cdot (u - \bar{V}) - \bar{\epsilon} = 0 \quad \text{or} \quad u = \bar{V} + \frac{\tau}{c} \cdot \bar{\epsilon}. \quad (88)$$

We next apply the lookahead operator to the expression in this Eq. 88. Assuming an initialization at $t_0 = -\infty$, the condition $\frac{\partial \mathcal{P}}{\partial u} = 0$ becomes equivalent to $(1 + \tau \frac{d}{dt}) \frac{\partial \mathcal{P}}{\partial u} = 0$, and hence Eq. 88 becomes equivalent to

$g(u - \bar{V}) - \bar{\epsilon} + \tau g(\dot{u} - \dot{\bar{V}}) - \tau \dot{\bar{\epsilon}} + \tau \dot{g}(u - \bar{V}) = c \dot{u} - g \cdot (V - u) - \epsilon + \tau \dot{g}(u - \bar{V}) = 0$, and with the total conductance $g(u)$ specified after Eq. 82 [this is calculated to become](#) (see [Jordan et al., 2022 \[this\]\(#\)](#))

$$c \dot{u} = g \cdot (V - u) + \epsilon + \dot{\tau} \cdot \bar{\epsilon}. \quad (89)$$

A learning rule of the form $\dot{W} \propto \frac{\partial \mathcal{P}}{\partial W} = \bar{e}_{\text{post}} \bar{r}^T$ with an appropriate postsynaptic error $\bar{e}_{\text{post}}$ can again be derived (see [Jordan et al., 2022 \[this\]\(#\)](#) for a single neuron in the probabilistic framework).

Generalizations: long memories, reinforcement learning

One could also extend the NLA principle by adding e.g. threshold adaptation that endows the dynamics with additional and longer time constants. For this, the rate function is parametrized by an additional threshold, $\bar{r} = \rho(u - \vartheta)$, an the Lagrangian is added by an error term on the threshold. Such an error addition can take the form $L = \dots + \frac{1}{2} \|\vartheta - \bar{r}^{\tau \vartheta}\|^2$, where $\bar{r}^{\tau \vartheta}$ now represents a low-pass filtering of the rate with a long threshold adaptation time constant τ_{ϑ} . Neurons that show an additional threshold adaptation will still be able to instantaneously transmit a voltage jump through a prospective firing rate, but this will now also depend on the neuron's history. Short-term plasticity may be included in the same way. Due to the history dependence, the stationarity of the action $\delta A = 0$ cannot anymore be reduced to the stationarity of the Lagrangian at any moment in time. As a consequence, the errors will look-ahead into to future more than only based on the local derivatives.

Generalizations are further possible for the cost function that favor voltage regions with high cost, say corresponding to punishment, or negative cost, corresponding to punishment to reward. These extensions will be considered in future work.

Voltage dynamics from the least-action principle in physics

We have shown that through the look-ahead in Hamilton's least-action principle, the notion of friction enters through the backdoor. In the least-action formalism in physics, friction is directly introduced by extending the Hamiltonian principle to a generalized D'Alembert principle, where at the level of the Euler-Lagrange equations the generalized force is equated to the dissipation force ([Flannery, 2005 \[this\]\(#\)](#)). For an introduction to the least-action principle in physics see e.g. [Feynman et al., 2011 \[this\]\(#\)](#), and for an introduction to the calculus of variation in general with a derivation of the Euler-Lagrange equation see e.g. [Chachuat, 2007 \[this\]\(#\)](#).

The electro-chemical properties of a membrane can be captured by an equivalent circuit consisting of a battery voltage V , a capacitance C and a resistance R , arranged in parallel. The voltage dynamics is derived from the Euler-Lagrange equations that are added by a dissipative force. Formally, in the absence of an inductance defining the kinetic energy, the Lagrangian $\mathcal{L}$ becomes identical to the potential energy $\mathcal{U}$ that itself is

$$\mathcal{L} = \mathcal{U} = QV - Q^2/(2C), \quad (90)$$

where Q represents the charge across the membrane. Looking for stationary trajectory of the action (time integral of the Lagrangian) with only the potential energy in the Lagrangian would not give any dynamics. In fact, the Euler-Lagrange equation $\partial_Q \mathcal{L} - d_t \partial_{\dot{Q}} \mathcal{L} = 0$ yields $Q = C V$. The potential energy is therefore extended by the dissipative Rayleigh energy $\mathcal{F}$ that introduces friction,

$$\mathcal{F} = R\dot{Q}^2/2. \quad (91)$$

According to D'Alembert's principle, the Rayleigh energy enters as a gradient at the level of the Euler-Lagrange equation. At this point, the theory steps out from the original principle by adding the Rayleigh gradient 'by hand' to the Euler-Lagrange equation (different from the NLA, see below). Otherwise, no first order differential equation with friction in the desired form would be obtained from the least-action principle in physics (if we were adding $\mathcal{U}$ to $\mathcal{L}$, for instance, the $d_t \partial_{\dot{Q}} \mathcal{L}$ -term in the Euler-Lagrange equations would yield the term $R\ddot{Q}$ instead of $R\dot{Q}$). Hence, with D'Alembert's patch the dynamics is characterized by the Euler-Lagrange equation added by a dissipative force,

$$\partial_Q \mathcal{L} - d_t \partial_{\dot{Q}} \mathcal{L} + \partial_{\dot{Q}} \mathcal{F} = 0,$$

and this equation reduces to $-V + Q/C + R\dot{Q} = 0$. Identifying the charge by means of voltage across the capacitance, $Q = Cu$, this equation can also be written as $-V + u + RC\dot{u} = 0$, or

$$\tau \dot{u} = -u + V, \quad (92)$$

with $\tau = RC$, similarly as derived in [Eq. 89](#).

The link to our NLA principle is most easily made by assuming that the 'membrane' conductance $g = 1/R$ does not depend on the voltage u (of the post- or presynaptic neuron, as this is the case in general, see explanation after [Eq. 82](#)). In this simplified case, the Lagrangian from [Eq. 86](#) becomes the power

$$\mathcal{P} = (u - \bar{V})^2/(2R). \quad (93)$$

The Euler-Lagrange equation with respect to $\tilde{u}$, applied to the action $A = \int \mathcal{P} dt$ is then calculated by

$$\frac{\partial \mathcal{P}}{\partial \tilde{u}} - \frac{d}{dt} \frac{\partial \mathcal{P}}{\partial \dot{\tilde{u}}} = \left(1 + \tau \frac{d}{dt}\right) \frac{\partial \mathcal{P}}{\partial u} = \frac{1}{R} (u + \tau \dot{u} - (\bar{V} + \tau \dot{\bar{V}})) = 0, \text{ or} \quad (94)$$

$$\tau \dot{u} = -u + V. \quad (95)$$

We used that $\tau = RC$ while $u = \tilde{u} - \tau \dot{\tilde{u}}$ and $V = \bar{V} + \tau \dot{\bar{V}}$.

The dynamics derived from the NLA ([Eq. 95](#)) is identical to the dynamics derived from the least-action principle with friction in physics ([Eq. 92](#)). Hence, the minimization of the energy (i.e. the time-integral of the power, $A = \int \mathcal{P} dt$) by looking ahead in time is equivalent to the stationarity of the physical action without looking ahead, but by taking account of the Rayleigh dissipation. The trick of looking ahead in time generates a dynamics out of an action A that encompasses only a potential energy, no kinetic energy. Without looking ahead, no dynamics would emerge from such an action. Notice, though, that the NLA action has units of energy (the time-integral of a power), while the physical action has units of energy times time (the time-integral of energies).

G. Table of mathematical symbols

Mathematical expression	Naming	Comment
u_i	Instantaneous (somatic) voltage	only for network neurons
$r_i = \rho(u_i) + \tau \dot{\rho}(u_i)$	Instantaneous firing rate of neuron i	that looks linearly ahead in time
$\bar{r}(t) = \frac{1}{\tau} \int_{-\infty}^t r(t') e^{-\frac{t-t'}{\tau}} dt'$	Definition of low-pass filtering	See Eq. 15
$\bar{r}_i = \rho(u_i) = \bar{r}_i + \tau \dot{\bar{r}}_i$	Low-pass filtered firing rate	postulated to be a function of u_i
$\mathbf{r} = \bar{\mathbf{r}} + \tau \dot{\bar{\mathbf{r}}}$	Self-consistency eq.	for low-pass filtered rate
$\mathbf{r}_{\text{in}}$	Input rate vector, column	projects to selected neurons
$\bar{\mathbf{r}}_{\text{in}}$	Low-pass filter input rates	instantaneously propagates
$e_i = (u_i + \tau \dot{u}_i) - \sum_j W_{ij} r_j$	Prospective error of neuron i	in apical dendrite
$\bar{e}_i = u_i - \sum_j W_{ij} \bar{r}_j$	Error of neuron i	in soma
$E_i^{\text{M}} = \frac{1}{2} \bar{e}_i^2 = \frac{1}{2} (u_i - \sum_j W_{ij} \bar{r}_j)^2$	Mismatch energy in neuron i	between soma and basal dendrite
u_o^*	Target voltage for output neuron o	could impose target on r_o or $\bar{r}_o$
$\bar{e}_o^* = u_o^* - u_o$	Error of output neuron o	also called target error

$C_o = \frac{1}{2}(\bar{e}_o^*)^2$	Cost contribution of output neuron o	between soma and basal dendrite
$L = \sum_{i \in \mathcal{N}} E_i^M + \beta \sum_{o \in \mathcal{O}} C_o$	Lagrangian	output $\mathcal{O} \subset$ network $\mathcal{N}$
$\tilde{u}(t) = \frac{1}{\tau} \int_t^\infty u(t') e^{(t-t')/\tau} dt'$	Discounted future voltage	prospective coordinates for NLA
$\mathbf{u} = \tilde{\mathbf{u}} - \tau \dot{\tilde{\mathbf{u}}}$	Self-consistency eq.	for discounted future voltage
$A = \int_{t_1}^{t_2} L[\tilde{\mathbf{u}}(t), \dot{\tilde{\mathbf{u}}}(t)] dt$	Neuronal Least Action (NLA)	expressed in prospect. coordinates
$\frac{\partial L}{\partial \tilde{u}_i} - \frac{d}{dt} \frac{\partial L}{\partial \dot{\tilde{u}}_i} = \left(1 + \frac{d}{dt}\right) \frac{\partial}{\partial u_i} L = 0$	Euler-Lagrange equations	turned into lookahead operator
$\mathbf{W}_{\text{in}}$	weights from input neurons $\mathbf{r}_{\text{in}}$	$\dim(\mathcal{N}) \times \dim(\mathbf{r}_{\text{in}})$, most 0
$\mathbf{W}_{\text{net}}$	weights between network neurons	$\dim(\mathcal{N}) \times \dim(\mathcal{N})$
$\mathbf{W} = (\mathbf{W}_{\text{in}}, \mathbf{W}_{\text{net}})$	total weight matrix	$\dim(\mathcal{N}) \times (\dim(\mathbf{r}_{\text{in}}) + \dim(\mathcal{N}))$
$\mathbf{r} = (\mathbf{r}_{\text{in}}, \mathbf{r}_{\text{net}})^T$	instantaneous firing rate vector	column (indicated by transpose)
$\dot{\mathbf{W}} \propto \bar{\mathbf{e}} \bar{\mathbf{r}}^T$	Plasticity of $\mathbf{W}$	$\bar{\mathbf{e}}$ is a column, $\bar{\mathbf{r}}^T$ a row vector
$\mathbf{u}_o^*(t) = \mathbf{F}^*(\bar{\mathbf{r}}_{\text{in}}(t))$	Target function formulated for $\bar{\mathbf{r}}_{\text{in}}(t)$	a functional of $\mathbf{r}_{\text{in}}(t)$
$\mathbf{u}_o(t) = \mathbf{F}_W(\bar{\mathbf{r}}_{\text{in}}(t), \bar{\mathbf{e}}_o^*(t))$	Func. implemented by forward network	instant. func. of $\bar{\mathbf{r}}_{\text{in}}(t)$, not $\mathbf{r}_{\text{in}}(t)$
N	Layers in forward network, w/o $\mathbf{r}_{\text{in}}$	Last-layer voltages: $\mathbf{u}_N = \mathbf{u}_o$
$\mathbf{W}_l^{\text{IP}}$	Weights from pyr to interneurons	lateral, within layer l
$\mathbf{W}_l^{\text{PI}}$	Weights from inter- to pyr'neurons	lateral, within layer l
$\mathbf{W}_l$	Bottom-up weights from layer $l-1$ to l	between pyramidal neurons
$\mathbf{B}_l$	Top-down weights from layer $l+1$ to l	between pyramidal neurons
$\bar{\mathbf{e}}_l^A = \mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{\text{PI}} \mathbf{u}_l^{\text{I}}$	Low-pass filtered apical error in layer l	top-down minus lateral feedback
$\bar{\mathbf{e}}_l = \bar{\mathbf{r}}_l' \cdot \bar{\mathbf{e}}_l^A = \bar{\mathbf{r}}_l' \cdot \mathbf{B}_l \bar{\mathbf{e}}_{l+1}$	Somato-basal prediction error	is correct error for learning
$E_l^{\text{IP}} = \frac{1}{2} \ \mathbf{u}_l^{\text{I}} - \mathbf{W}_l^{\text{IP}} \bar{\mathbf{r}}_l\ ^2$	Interneuron mismatch energy	minimized to learn $\mathbf{W}_l^{\text{IP}}$
$E_l^{\text{PI}} = \frac{1}{2} \ \mathbf{B}_l \mathbf{u}_{l+1} - \mathbf{W}_l^{\text{PI}} \mathbf{u}_l^{\text{I}}\ ^2$	Apical mismatch energy	minimized to learn $\mathbf{W}_l^{\text{PI}}$
$\eta, \eta^{\text{IP}}, \eta^{\text{PI}}$	Learning rates for plasticity of...	... $\mathbf{W}_l, \mathbf{W}_l^{\text{IP}}, \mathbf{W}_l^{\text{PI}}$
$\mathbf{H} = \frac{\partial^2 L}{\partial \mathbf{u}^2} = \mathbf{1} - \mathbf{W}_{\text{net}} \boldsymbol{\rho}' - \bar{\mathbf{e}}'$	Hessian, $\frac{\partial^2 L}{\partial \mathbf{u}^2} = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}$. If pos. definite	$\Rightarrow$ stable dynamics
$\mathbf{f}(\mathbf{u}, t) = \frac{\partial L}{\partial \mathbf{u}} = \mathbf{u} - \mathbf{W} \bar{\mathbf{r}}(\mathbf{u}) - \bar{\mathbf{e}}(\mathbf{u})$	Corrected error	becomes 0 with τ
$\mathbf{f}(\mathbf{u}, t) + \tau \dot{\mathbf{f}}(\mathbf{u}, t) = 0$	Euler-Lagrange equations	satisfy $\mathbf{f}(\mathbf{u}, t) = \mathbf{f}_0 e^{-(t-t_0)/\tau}$
$\mathbf{f}(\mathbf{u}, t) = 0$	Always the case after transient,	exponentially decaying with τ
$\dot{\mathbf{u}} = -\frac{1}{\tau} \mathbf{H}^{-1}(\mathbf{u}) \left(\mathbf{f}(\mathbf{u}) + \tau \frac{\partial \mathbf{f}}{\partial t} \right)$	Explicit diff. eq.	obtained by solving for $\dot{\mathbf{u}}$
$\mathbf{g}(\mathbf{u}, t) = -\frac{1}{\tau} \mathbf{H}^{-1}(\mathbf{u}) \left(\mathbf{f}(\mathbf{u}) + \tau \frac{\partial \mathbf{f}}{\partial t} \right)$	Used to write the explicit diff. eq.	$\dot{\mathbf{u}} = \mathbf{g}(\mathbf{u}, t)$
$\mathbf{G}(\mathbf{y}, \dot{\mathbf{u}}) = \left(1 + \tau \frac{d}{dt}\right) \frac{\partial L}{\partial \mathbf{u}} = \mathbf{f} + \tau \dot{\mathbf{f}}$	Used for contraction analysis, Eq. 53	$\mathbf{y} = (\mathbf{r}_{\text{in}}, \mathbf{u}_o^*, \mathbf{u})$
$\mathbf{M}, \mathbf{K}$	Used to iteratively converge to $\dot{\mathbf{u}}$	see Eq. 46
$\check{\mathbf{u}} = \mathbf{u} + \tau \dot{\mathbf{u}}$	Linear lookahead voltage	in Latent Equilibrium, Sect. D

H. A tutorial on total and partial derivatives as used in the paper

The proof of Theorem 1 given in the Methods (Sect. D) makes use of partial and total derivatives and follows the notation of Scellier & Bengio, 2017 [\[17\]](#) and Meulemans *et al.*, 2022 [\[18\]](#). As there is some variability (and community-dependent sloppiness) in the notation of partial and total

derivatives here and in general, we provide some explanations on how this notation is interpreted, and why, for instance, total derivatives commute with each other, and also partial with each other (although not total with partial). For a standard basic textbook introducing the concepts of partial derivatives, total differentials, the implicit function theorem etc., see [Stewart, 2010](#), being historically based on the classics of [Courant, 1934](#).

1. In a differential geometric setting, the derivative of real-valued function $E(\mathbf{u})$ on a point $\mathbf{u}$ of a manifold (like the flat Euclidean space) is considered as a mapping of tangent vectors at $\mathbf{u}$ to the real numbers. When interpreting the derivative as mapping, the $\frac{\partial E}{\partial \mathbf{u}}$ is living in the dual space of $\mathbf{u}$ and is therefore a row vector if $\mathbf{u}$ is a column vector. If a function $\mathbf{f}(\mathbf{u})$ of $\mathbf{u}$ is a vector valued, then its derivative is a matrix with entries $\left(\frac{\partial \mathbf{f}}{\partial \mathbf{u}}\right)_{ij} = \frac{\partial f_i}{\partial u_j}$, where i is indexing the rows (i.e. running down) and j is indexing the columns (i.e. running right). When $\mathbf{r} = \rho(\mathbf{u})$ is a column vector with ρ applied to each component of $\mathbf{u}$, we consider the (partial) derivative $\mathbf{r}' = \rho'(\mathbf{u})$ for convenience as column vector with components $\rho'(u_i)$. To strictly follow the formalism, it should be a diagonal matrix.
2. Because we introduced the error vector $\bar{\mathbf{e}}$ as column vector, it is easier to write $\frac{\partial L}{\partial \mathbf{u}} = \bar{\mathbf{e}} + \dots$, where now $\frac{\partial L}{\partial \mathbf{u}}$ is also considered as column vector. To be consistent with the above, we should have written $\frac{\partial L}{\partial \mathbf{u}}^T = \bar{\mathbf{e}} + \dots$. The T appeared as too heavy so that we neglected it, where it did not have further mathematical consequences (hoping it does not cause confusions). Sticking here to a column vector also renders the backpropagation error to be a column vector in Eq. 21. This error then gets the classical form with the weight transpose, $\bar{\mathbf{e}} = \bar{\mathbf{r}}_{\text{net}}' \cdot \mathbf{W}_{\text{net}}^T \bar{\mathbf{e}}$.
3. We typically have real-valued functions of the form $E(\mathbf{u}_\theta, \theta)$, with $\theta = (\mathbf{W}, \beta)$ being a vector of parameters, and $\mathbf{u}$ being a function of θ . To get the total derivative of E with respect to θ we consider the values E as a function of θ . This can be done by introducing a new function $\mathcal{E}(\theta)$ defined $\mathcal{E}(\theta) = E(\mathbf{u}_\theta, \theta)$, where the components θ_i are considered as independent variables. The total derivative of E with respect to θ is then defined as vector-valued function $\frac{dE}{d\theta} = \left(\frac{dE}{d\theta_1}, \dots, \frac{dE}{d\theta_n}\right) = \left(\frac{d\mathcal{E}}{d\theta_1}, \dots, \frac{d\mathcal{E}}{d\theta_n}\right) = \frac{\partial \mathcal{E}}{\partial \theta}$ for a n -dimensional θ . It can be helpful to think of the components of this total derivative as a (total) directional derivatives in the unit directions. For the last (n -th) unit direction $\Delta\theta^{(n)} = (0, \dots, 0, 1)$, for instance, we have $\frac{dE}{d\theta_n} = \lim_{\varepsilon \rightarrow 0} \frac{1}{\varepsilon} (E(\mathbf{u}_{\theta + \varepsilon \Delta\theta^{(n)}}, \theta + \varepsilon \Delta\theta^{(n)}) - E(\mathbf{u}_\theta, \theta))$.
4. The cost gradient, $\frac{dC}{d\mathbf{W}} \propto (\mathbf{u} - \mathbf{W}\bar{\mathbf{r}}) \bar{\mathbf{r}}^T$ has the same dimension as $\mathbf{W}$. Recall that by the cost gradient we mean $\frac{dC}{d\mathbf{W}} = \frac{\partial \mathcal{C}}{\partial \mathbf{W}}$, where $\mathcal{C}$ is defined as $\mathcal{C}(\mathbf{W}) = C(\mathbf{u}_o(\mathbf{W}), \mathbf{u}_o^*)$, with the voltage $\mathbf{u}_o$ of the output neurons being itself a function of $\mathbf{W}$.
5. To calculate the partial derivative $\frac{\partial}{\partial \theta} E(\mathbf{u}, \theta)$ with respect to θ , we fix the first argument $\mathbf{u}_\theta$, even if for $\mathbf{u}$ we often plugged in the components of the trajectory $\mathbf{u} = \mathbf{u}_\theta(t)$ that now does depend on θ . In contrast, the total derivative is $\frac{d}{d\theta} E(\mathbf{u}, \theta) = \frac{\partial E(\mathbf{u}, \theta)}{\partial \mathbf{u}} \frac{d\mathbf{u}}{d\theta} + \frac{\partial E(\mathbf{u}, \theta)}{\partial \theta}$. Here, $\frac{\partial E(\mathbf{u}, \theta)}{\partial \mathbf{u}}$ is a row vector, as also $\frac{\partial E(\mathbf{u}, \theta)}{\partial \theta}$, consistent with the convention (that we have broken in Eq. 28 to keep vectors as columns). When $\mathbf{u}$ is considered as trajectory, $\frac{d\mathbf{u}}{d\theta}$ does not vanish in general, but it does when $\mathbf{u}$ is simply considered as independent variable.

6. When replacing the argument $\mathbf{u}$ in $\mathbf{u} = \tilde{\mathbf{u}} - \tau \dot{\tilde{\mathbf{u}}}$ we get the ‘Lagrangian’

$L(\tilde{\mathbf{u}}, \dot{\tilde{\mathbf{u}}}, \boldsymbol{\theta}) = E(\tilde{\mathbf{u}} - \tau \dot{\tilde{\mathbf{u}}}, \boldsymbol{\theta})$. The partial derivative of L with respect to $\tilde{\mathbf{u}}$, for instance, is then $\frac{\partial}{\partial \tilde{\mathbf{u}}} L(\tilde{\mathbf{u}}, \dot{\tilde{\mathbf{u}}}, \boldsymbol{\theta}) = \frac{\partial}{\partial \mathbf{u}} E(\mathbf{u}, \boldsymbol{\theta}) \frac{\partial \mathbf{u}}{\partial \tilde{\mathbf{u}}} = \frac{\partial}{\partial \mathbf{u}} E(\mathbf{u}, \boldsymbol{\theta})$. The partial derivative $\frac{\partial}{\partial \tilde{\mathbf{u}}} L$ considers L as a function of independent arguments $\tilde{\mathbf{u}}$, $\dot{\tilde{\mathbf{u}}}$ and $\boldsymbol{\theta}$.

7. We also used that the total derivatives commute, in the current example

$\frac{d}{d\mathbf{W}} \frac{d}{d\beta} E = \frac{d}{d\beta} \frac{d}{d\mathbf{W}} E$. This is generally true for derivatives of Lipschitz continuous functions, for which derivatives exist almost everywhere. The total derivatives (where they exist) then commute because the difference quotients in $\mathbf{W}$ and β are uniformly bounded. The Moore-Osgood theorem tells that two limits, of which at least one is uniform in the other, can be commuted. This also applies to the double difference quotients involved in the definition of $\frac{d}{d\mathbf{W}} \frac{d}{d\beta}$. Remember that the total derivative, for instance with respect to the n -th parameter, can be written as $\frac{dE}{d\theta_n} = \lim_{\varepsilon \rightarrow 0} \frac{1}{\varepsilon} (E(\mathbf{u}_{\boldsymbol{\theta} + \varepsilon \Delta \boldsymbol{\theta}^{(n)}}, \boldsymbol{\theta} + \varepsilon \Delta \boldsymbol{\theta}^{(n)}) - E(\mathbf{u}_{\boldsymbol{\theta}}, \boldsymbol{\theta}))$. But note again that, while for Lipschitz functions partial derivatives commute among themselves, and total derivatives commute among themselves, partial and total derivatives in general do not commute (not even with additional smoothness conditions)!

I. Proof of Theorem 1, part (ii), using only partial derivatives

This Section proves Eqs 29–31 in terms with only partial derivatives, banning nested functions that require to deal with a combination of total partial derivatives. While this represents a conceptual simplification, it requires many more additional (unnested) functions to be defined (and in fact being the reason why in mathematics the concept of a total differential / total derivative is introduced, see e.g. Courant, 1934). – The cited 3 equations are also the core for the proof for Equilibrium Propagation Scellier & Bengio, 2017, although there only applied in the steady state after the network converged to a constant activity.

We assume a network of d neurons whose membrane potential is given by $\mathbf{u} \in \mathbb{R}^d$ and which are connected via weights $\mathbf{W} \in \mathbb{R}^{d \times d}$. By $\nabla_{\mathbf{u}}$ we denote the gradient with respect to the membrane potentials, i.e., $\nabla_{\mathbf{u}} = (\frac{\partial}{\partial u_1}, \dots, \frac{\partial}{\partial u_d})$. Similarly, $\nabla_{\mathbf{W}}$ is a matrix containing the derivatives with respect to the weights, $(\nabla_{\mathbf{W}})_{ij} = \frac{\partial}{\partial W_{ij}}$.

To prove the rt-DeEP theorem (theorem 1), we first have to make a few definitions and observations:

1. For a given $(\mathbf{W}, \beta)$, the dynamics yield certain membrane potentials $f_{\mathbf{u}} \in \mathbb{R}^d$. Formally, we define this as

$$f_{\mathbf{u}} : \mathbb{R}^{d \times d} \times \mathbb{R} \rightarrow \mathbb{R}^d, \quad (\mathbf{W}, \beta) \mapsto f_{\mathbf{u}}(\mathbf{W}, \beta). \quad (96)$$

The i th element of $f_{\mathbf{u}}$ is denoted by f_{u_i} and hence $\nabla_{f_{\mathbf{u}}} = (\frac{\partial}{\partial f_{u_1}}, \dots, \frac{\partial}{\partial f_{u_d}})$.

2. We define the following functions:

- the mismatch energy E^M :

$$\mathbb{R}^d \times \mathbb{R}^{d \times d} \rightarrow \mathbb{R}, (\mathbf{u}, \mathbf{W}) \mapsto E^M(\mathbf{u}, \mathbf{W}) = \frac{1}{2} \sum_{i=1}^d \left[u_i - \sum_j W_{ij} \bar{r}_j \right]^2$$

- the cost function $C: \mathbb{R}^d \rightarrow \mathbb{R}, \mathbf{u} \mapsto C(\mathbf{u}) = \frac{1}{2} \sum_{k \in \mathcal{O}} \|u_k^* - u_k\|^2$
- the Lagrangian $L: \mathbb{R}^d \times \mathbb{R}^{d \times d} \times \mathbb{R} \rightarrow \mathbb{R}, (\mathbf{u}, \mathbf{W}, \beta) \mapsto L(\mathbf{u}, \mathbf{W}, \beta) = E^M(\mathbf{u}, \mathbf{W}) + \beta C(\mathbf{u})$.

To make the dependency of the cost and energies on β and $\mathbf{W}$ explicit, we further introduce three auxiliary functions F_M, F_C and F_L :

- for the mismatch energy $F_M: \mathbb{R}^{d \times d} \times \mathbb{R} \rightarrow \mathbb{R}, (\mathbf{W}, \beta) \mapsto F_M(\mathbf{W}, \beta) = E^M(f_{\mathbf{u}}(\mathbf{W}, \beta), \mathbf{W})$,
- for the cost function $F_C: \mathbb{R}^{d \times d} \times \mathbb{R} \rightarrow \mathbb{R}, (\mathbf{W}, \beta) \mapsto F_C(\mathbf{W}, \beta) = C(f_{\mathbf{u}}(\mathbf{W}, \beta))$,
- for the Lagrangian $F_L: \mathbb{R}^{d \times d} \times \mathbb{R} \rightarrow \mathbb{R}, (\mathbf{W}, \beta) \mapsto F_L(\mathbf{W}, \beta) = F_M(\mathbf{W}, \beta) + \beta F_C(\mathbf{W}, \beta)$

3. The Euler-Lagrange equations can be written as $\tau \frac{d}{dt} \nabla_{\mathbf{u}} L = -\nabla_{\mathbf{u}} L$. Hence, far enough away from initialization and for smooth enough input (and targets) we have $\nabla_{\mathbf{u}} L = 0$ at all times, even when changing the network input continuously. Note that both the cost and the mismatch energies are defined on low-pass-filtered signals, and it is with respect to the low-pass filtered external input that the low-pass-filtered output error is minimized.

4. Without output nudging (i.e., $\beta = 0$), the output error vanishes and consequently all other prediction errors vanish as well, $\bar{\mathbf{e}} = \mathbf{u} - \mathbf{W} \bar{\mathbf{r}} = 0$. This can be easily shown for layered network architectures and holds true for arbitrary connections (e.g., recurrent networks) as long as $f_{\mathbf{u}}$ uniquely exists, i.e., $\mathbf{u} = \mathbf{W} \bar{\mathbf{r}}$ has a unique solution for $\mathbf{u}$. From the form of the mismatch energy, we then get

$$\nabla_{\mathbf{W}} E^M|_{\beta=0} = (\mathbf{W} \bar{\mathbf{r}} - \mathbf{u}) \bar{\mathbf{r}}^T|_{\beta=0} = 0. \quad (97)$$

Since we are assuming smooth functions, this also implies that

$$\lim_{\epsilon \rightarrow 0} \nabla_{\mathbf{W}} E^M|_{\beta=\epsilon \beta} = 0. \quad (98)$$

5. From the assumption of well-behaved (smooth) functions, it also follows that partial derivatives commute $\nabla_{\mathbf{W}} \frac{\partial}{\partial \beta} = \frac{\partial}{\partial \beta} \nabla_{\mathbf{W}}$.

Our goal is to find a plasticity rule that minimizes the cost C , which we do by calculating $\nabla_{\mathbf{W}} F_C|_{\beta=0}$. Similar to Scellier & Bengio (2017) [\[2\]](#), to achieve this, we first calculate the partial derivatives of F_L with respect to the nudging strength β

$$\frac{\partial F_L}{\partial \beta} = \frac{\partial F_M}{\partial \beta} + \beta \frac{\partial F_C}{\partial \beta} + F_C \quad (99a)$$

$$= \sum_{i=1}^d \left(\frac{\partial E^M}{\partial f_{u_i}} \frac{\partial f_{u_i}}{\partial \beta} + \beta \frac{\partial C}{\partial f_{u_i}} \frac{\partial f_{u_i}}{\partial \beta} \right) + C \quad (99b)$$

$$= \sum_{i=1}^d \underbrace{\frac{\partial (E^M + \beta C)}{\partial f_{u_i}}}_{=0} \frac{\partial f_{u_i}}{\partial \beta} + C \quad (99c)$$

$$= C, \quad (99d)$$

and the weights $\mathbf{W}$

$$\frac{\partial F_L}{\partial W_{ij}} = \frac{\partial F_M}{\partial W_{ij}} + \beta \frac{\partial F_C}{\partial W_{ij}} \quad (100a)$$

$$= \sum_{k=1}^d \left(\frac{\partial E^M}{\partial f_{u_k}} \frac{\partial f_{u_k}}{\partial W_{ij}} + \beta \frac{\partial C}{\partial f_{u_k}} \frac{\partial f_{u_k}}{\partial W_{ij}} \right) + \frac{\partial E^M}{\partial W_{ij}} \quad (100b)$$

$$= \sum_{k=1}^d \underbrace{\frac{\partial (E^M + \beta C)}{\partial f_{u_k}}}_{=0} \frac{\partial f_{u_k}}{\partial W_{ij}} + \frac{\partial E^M}{\partial W_{ij}} \quad (100c)$$

$$= \frac{\partial E^M}{\partial W_{ij}}, \quad (100d)$$

$$(100e)$$

or in vectorized form

$$\nabla_{\mathbf{W}} F_L = \nabla_{\mathbf{W}} E^M. \quad (101)$$

With these identities in place, we can calculate the plasticity rule:

$$- \lim_{\epsilon_\beta \rightarrow 0} \nabla_{\mathbf{W}} F_C = \lim_{\epsilon_\beta \rightarrow 0} \nabla_{\mathbf{W}} \frac{\partial F_L}{\partial \beta} \quad (102a)$$

$$= \lim_{\epsilon_\beta \rightarrow 0} \frac{\partial}{\partial \beta} \nabla_{\mathbf{W}} F_L \quad (102b)$$

$$= \lim_{\epsilon_\beta \rightarrow 0} \lim_{\delta \rightarrow 0} \frac{1}{\delta} \left(\nabla_{\mathbf{W}} F_L|_{\beta=\epsilon_\beta+\delta} - \nabla_{\mathbf{W}} F_L|_{\beta=\epsilon_\beta} \right) \quad (102c)$$

$$= \lim_{\delta \rightarrow 0} \frac{1}{\delta} \lim_{\epsilon_\beta \rightarrow 0} \left(\nabla_{\mathbf{W}} E^M|_{\beta=\epsilon_\beta+\delta} - \nabla_{\mathbf{W}} E^M|_{\beta=\epsilon_\beta} \right) \quad (102d)$$

$$= \lim_{\delta \rightarrow 0} \frac{1}{\delta} \left(\lim_{\epsilon_\beta \rightarrow 0} \nabla_{\mathbf{W}} E^M|_{\beta=\epsilon_\beta+\delta} - \underbrace{\lim_{\epsilon_\beta \rightarrow 0} \nabla_{\mathbf{W}} E^M|_{\beta=\epsilon_\beta}}_{=0 \text{ from Eq. 98}} \right) \quad (102e)$$

$$= \lim_{\delta \rightarrow 0} \frac{1}{\delta} \nabla_{\mathbf{W}} E^M|_{\beta=\delta} \quad (102f)$$

$$\approx \frac{1}{\delta} \nabla_{\mathbf{W}} E^M|_{\beta=\delta} \text{ for } \delta \ll 1, \quad (102g)$$

where we used that limits can be exchanged for smooth functions. Using the definition of E^M , we obtain a plasticity rule that minimizes the cost function

$$-\lim_{\epsilon_\beta \rightarrow 0} \nabla_{\mathbf{W}} C|_{\beta=\epsilon_\beta} \approx -\frac{1}{\epsilon_\beta} \nabla_{\mathbf{W}} E^M|_{\beta=\epsilon_\beta} = \frac{1}{\epsilon_\beta} (\mathbf{u} - \mathbf{W} \hat{\mathbf{r}}) \hat{\mathbf{r}}^T|_{\beta=\epsilon_\beta} \quad \text{for } \epsilon_\beta \ll 1. \quad (103)$$

In practice, the prefactor $\frac{1}{\epsilon_\beta}$ is absorbed into the learning rate.

References

1. Abbott L. F., Varela J. a., Sen K., Nelson S. B. (1997) **Synaptic depression and cortical gain control** *Science* **275**:220–224
2. Ackley D. H., Hinton G. E., Sejnowski T. J. (1985) **A learning algorithm for Boltzmann machines** *Cognitive Science* **9**:147–169
3. Akrouf M., Wilson C., Humphreys P. C., Lillicrap T., Tweed D. (2019) **Deep learning without weight transport** *arXiv*
4. Alonso E., Fairbank M., Mondragón E. (2012) **Conditioning for least action** *Proceedings of the 11th International Conference on Cognitive Modeling, ICCM* :234–239
5. Amirikian B. R., Lukashin A. V. (1992) **A neural network learns trajectory of motion from the least action principle** *Biological Cybernetics* **66**:261–264
6. Bannon N. M., Chistiakova M., Volgushev M. (2020) **Synaptic Plasticity in Cortical Inhibitory Neurons: What Mechanisms May Help to Balance Synaptic Weight Changes?** *Frontiers in Cellular Neuroscience* **14**
7. Bartolozzi C., Indiveri G., Donati E. (2022) **Embodied neuromorphic intelligence** *Nature Communications* **13**:1–14
8. Bastos A. M., et al. (2012) **Canonical Microcircuits for Predictive Coding** :695–711
9. Bellec G., et al. (2020) **A solution to the learning dilemma for recurrent networks of spiking neurons** *Nature Communications* **11**:1–15
10. Betti A., Gori M. (2016) **The principle of least cognitive action** *Theoretical Computer Science* **633**:83–99 <https://doi.org/10.1016/j.tcs.2015.06.042>
11. Blom T., Feuerriegel D., Johnson P., Bode S., Hogendoorn H. (2020) **Predictions drive neural representations of visual events ahead of incoming sensory information** *Proceedings of the National Academy of Sciences of the United States of America* **117**:7510–7515
12. Borovik A. (2021) **A mathematician's view of the unreasonable ineffectiveness of mathematics in biology** *Biosystems* **205**
13. Brendel W., Bourdoukan R., Vertechi P., Machens C. K., Denève S. (2020) **Learning to represent signals spike by spike** *PLoS Computational Biology* **16**:1–23
14. Burrello A., Cavigelli L., Schindler K., Benini L., Rahimi A. (2019) **Laelaps: An Energy-Efficient Seizure Detection Algorithm from Long-term Human iEEG Recordings without False Alarms in Proceedings of the 2019 Design Automation & Test in Europe Conference & Exhibition (DATE)** :752–757
15. Chavlis S., Poirazi P. (2021) **Drawing inspiration from biological dendrites to empower artificial neural networks** *Current Opinion in Neurobiology* **70**:1–10 <https://doi.org/10.1016/j.conb.2021.04.007>

16. Coopersmith J. (2017) **The Lazy Universe: An Introduction to the Principle of Least Action**
17. Cramer B., et al. (2022) **Surrogate gradients for analog neuromorphic computing** *Proceedings of the National Academy of Sciences of the United States of America* **119**
18. Deperrois N., Petrovici M. A., Senn W., Jordan J. (2023) **Learning beyond sensations: how dreams organize neuronal representations** *Neuroscience and Biobehavioral Reviews* **157**
19. Deperrois N., Petrovici M. A., Senn W., Jordan J. (2022) **Learning cortical representations through perturbed and adversarial dreaming** *eLife* :1–34
20. Dimitriou M. (2016) **Enhanced Muscle Afferent Signals during Motor Learning in Humans** *Current Biology* **26**:1062–1068 <https://doi.org/10.1016/j.cub.2016.02.030>
21. Dimitriou M. (2022) **Human muscle spindles are wired to function as controllable signal-processing devices** *eLife* **11**:1–14
22. Dimitriou M., Edin B. B. (2010) **Human muscle spindles act as forward sensory models** *Current Biology* **20**:1763–1767 <https://doi.org/10.1016/j.cub.2010.08.049>
23. Feldman A. G., Levin M. F., Sternad D. (2009) **Progress in Motor Control** :699–726
24. Feynman R., Leighton R., Sands M. (2011) **The Feynman Lectures on Physics**
25. Fox S., Kotelba A. (2018) **Principle of least psychomotor action: Modelling situated entropy in optimization of psychomotor work involving human, cyborg and robot workers** *Entropy* **20**
26. Friedrich J., Senn W. (2012) **Spike-based Decision Learning of Nash Equilibria in Two-Player Games** *PLoS Computational Biology* **8**
27. Friedrich J., Urbanczik R., Senn W. (2011) **Spatio-Temporal Credit Assignment in Neuronal Population Learning** *PLoS Computational Biology* **7** <https://doi.org/10.1371/journal.pcbi.1002092>
28. Friston K. (2010) **The free-energy principle: a unified brain theory?** *Nature Reviews Neuroscience* **11**:127–138
29. Friston K., et al. (2022) **A free energy principle made simpler but not too simple** :1–42
30. Gerstner W., Kistler W. (2002) **Spiking neuron models: Single neurons, populations, plasticity**
31. Göltz J., et al. (2021) **Fast and energy-efficient neuromorphic deep learning with first-spike times** *Nature Machine Intelligence* **3**:823–835
32. Granier A., Petrovici M. A., Senn W., Wilmes K. A. (2023) **Granier, A., Petrovici, M. A., Senn, W. & Wilmes, K. A. Precision estimation and second-order prediction errors in cortical circuits, 1–22.** <http://arxiv.org/abs/2309.16046> (2023). :1–22
33. Guerguiev J., Lillicrap T. P., Richards B. A. (2017) **Towards deep learning with segregated dendrites** *eLife* **6**:1–37

34. Haider P., et al. (2021) **Latent Equilibrium: Arbitrarily fast computation with arbitrarily slow neurons** *Advances in Neural Information Processing Systems* **34**
35. Hassabis D., Kumaran D., Summerfield C., Botvinick M. (2017) **Neuroscience-Inspired Artificial Intelligence** *Neuron* **95**:245–258 <https://doi.org/10.1016/j.neuron.2017.06.011>
36. Herzfeld D. J., Vaswani P. A., Marko M. K., Shadmehr R. (2014) **A memory of errors in sensorimotor learning** :1349–1353
37. Hodgkin A. L., Huxley A. F. (1952) **A quantitative description of membrane current and its application to conduction and excitation in nerve** *Bulletin of Mathematical Biology* **117**:500–544
38. Hopfield J. J. (1982) **Neural networks and physical systems with emergent collective computational abilities** *Proc. Nat. Acad. Sci. USA* **79**:2554–2558
39. Karkar S., Ayed I., de Bézenac E., Gallinari P. (2021) **A Principle of Least Action for the Training of Neural Networks** *Lecture Notes in Computer Science 12458 LNAI* :101–117
40. Kawato M. (1999) **Internal models for motor control and trajectory planning** *Current Opinion in Neurobiology* **9**:718–727
41. Keller G. B., Mrosovsky N. (2018) **Predictive Processing: A Canonical Cortical Computation** *Neuron* **100**:424–435
42. Kiebel S. J., Friston K. (2011) **Free energy and dendritic self-organization** *Frontiers in Systems Neuroscience* **5**:1–13
43. Köndgen H., et al. (2008) **The dynamical response properties of neocortical neurons to temporally modulated noisy inputs in vitro** *Cerebral cortex* **18**:2086–2097
44. Kunin D., et al. (2020) **Two routes to scalable credit assignment without weight symmetry** *International Conference on Machine Learning* :5511–5521
45. La Camera G., et al. (2006) **Multiple time scales of temporal response in pyramidal and fast spiking cortical neurons** *Journal of Neurophysiology* **96**:3448–3464
46. Larkum M., Senn W., Lüscher H. (2004) **Top-down Dendritic Input Increases the Gain of Layer 5 Pyramidal Neurons** *Cerebral Cortex* **14**:1059–1070
47. Latash M. L. (2010) **Motor synergies and the equilibrium-point hypothesis** *Motor Control* **14**:294–322
48. Latash M. L. (2018) **Muscle coactivation: Definitions, mechanisms, and functions** *Journal of Neurophysiology* **120**:88–104
49. LeCun Y. (1998) **LeCun, Y. The MNIST database of handwritten digits.** <http://yann.lecun.com/exdb/mnist/> (1998).
50. Li S., et al. (2015) **Coordinated alpha and gamma control of muscles and spindles in movement and posture** *Frontiers in Computational Neuroscience* **9**:1–15
51. Lillicrap T. P., Cownden D., Tweed D. B., Akerman C. J. (2016) **Random synaptic feedback weights support error backpropagation for deep learning** *Nature communications* **7**:1–10

52. Lillicrap T. P., Santoro A., Marris L., Akerman C. J., Hinton G. (2020) **Backpropagation and the brain** *Nature Reviews Neuroscience* **21**:335–346
53. Max K., et al. (2022) **Learning efficient backprojections across cortical hierarchies in real time** :1–31
54. Mesnard T., Vignoud G., Sacramento J., Senn W., Bengio Y. (2019) **Ghost Units Yield Biologically Plausible Backprop in Deep Neural Networks** *arXiv*
55. Meulemans A., Farinha M. T., et al. (2021) **Credit Assignment in Neural Networks through Deep Feedback Control** *Advances in Neural Information Processing Systems* **34**
56. Meulemans A., Zucchet N., Kobayashi S., von Oswald J., Sacramento J., et al. (2022) **The least-control principle for local learning at equilibrium** *Advances in Neural Information Processing Systems* **35**
57. Mikulasch F. A., Rudelt L., Priesemann V. (2021) **Local dendritic balance enables learning of efficient representations in networks of spiking neurons** *Proceedings of the National Academy of Sciences of the United States of America* **118**
58. Mikulasch F. A., Rudelt L., Wibral M., Priesemann V. (2023) **Where is the error? Hierarchical predictive coding through dendritic error computation** *Trends in Neurosciences* **46**:45–59 <https://doi.org/10.1016/j.tins.2022.09.007>
59. Ostojic S., et al. (2015) **Neuronal morphology generates high-frequency firing resonance** *Journal of Neuroscience* **35**:7056–7068
60. Palmer S. E., Marre O., Berry M. J., Bialek W. (2015) **Predictive information in a sensory population** *Proceedings of the National Academy of Sciences* **112**:6908–6913
61. Papaioannou S., Dimitriou M. (2021) **Goal-dependent tuning of muscle spindle receptors during movement preparation** *Science Advances* **7**:1–14
62. Pezzulo G., Parr T., Friston K. (2022) **The evolution of brain architectures for predictive coding and active inference** *Philosophical Transactions of the Royal Society B: Biological Sciences* **377**
63. Pfister J.-P., Dayan P., Lengyel M. (2010) **Synapses with short-term plasticity are optimal estimators of presynaptic membrane potentials** *Nature Neuroscience* **13**:1271–1275
64. Poirazi P., Papoutsis A. (2020) **Illuminating dendritic function with computational models** *Nature Reviews Neuroscience* **21**:303–321
65. Rao R. P. N., Ballard D. H. (1999) **Predictive coding in the visual cortex: a functional interpretation of some extraclassical receptive-field effects** :79–87
66. Richards B. A., et al. (2019) **A deep learning framework for neuroscience** *neuroscience* **22**:1761–1770
67. Rumelhart D. E., Hinton G. E., Williams R. J. (1986) **Learning Representations by Back-propagating Errors** *Nature* **323**:533–536

68. Sacramento J., Ponte Costa R., Bengio Y., Senn W. (2018) **Dendritic cortical microcircuits approximate the back-propagation algorithm** *Advances in Neural Information Processing Systems* **31**:8721–8732
69. Scellier B., Bengio Y. (2017) **Equilibrium propagation: Bridging the gap between energy-based models and backpropagation** *Frontiers in computational neuroscience* **11**
70. Schiess M., Urbanczik R., Senn W. (2016) **Somato-dendritic Synaptic Plasticity and Error-backpropagation in Active Dendrites** *PLoS Computational Biology* **12**:1–18
71. Senn W., Wyler K., Clamann H., Kleinle J., Larkum M., et al. (1995) **Recruitment by size and principle of least action tech. rep. July** *Internal Report IAM* :2–6
72. Senn W., Wyler K., Clamann H., Kleinle J., Lüscher H.-R., et al. (1997) **Size principle and information theory** :11–22
73. Simonetto A., Dall'Anese E., Paternain S., Leus G., Giannakis G. B. (2020) **Time-Varying Convex Optimization: Time-Structured Algorithms and Applications** *Proceedings of the IEEE* **108**:2032–2048
74. Sjöström P. J., Turrigiano G. G., Nelson S. B. (2001) **Rate, timing, and cooperativity jointly determine cortical synaptic plasticity** *Neuron* **32**:1149–1164
75. Song Y., et al. (2024) **Inferring neural activity before plasticity as a foundation for learning beyond backpropagation** *Nature Neuroscience*
76. Spicher D., Clopath C., Senn W. (2017) **Predictive plasticity in dendrites: from a computational principle to experimental data** :1–2
77. Summers R. L. (2021) **An Action Principle for Biological Systems** *Journal of Physics: Conference Series* **2090**
78. Talairach J., Tournoux P. (1988) **Co-planar stereotaxic atlas of the human brain: 3-Dimensional proportional system: An approach to cerebral imaging**
79. Theis A. K., Rózsa B., Katona G., Schmitz D., Jochenning F. W. (2018) **Voltage gated calcium channel activation by backpropagating action potentials downregulates NMDAR function** *Frontiers in Cellular Neuroscience* **12**:1–14
80. Thorpe S., Fize D., Marlot C. (1996) **Speed of processing in the human visual system** *Nature* **381**:520–522
81. Todorov E., Doya K., Ishii S., Pouget A., Rao R. P. (2006) **The Bayesian Brain** :1–28
82. Todorov E. (2004) **Optimality principles in sensorimotor control** *Nature Neuroscience* **7**:907–915
83. Todorov E., Jordan M. I. (2002) **Optimal feedback control as a theory of motor coordination** *Nature Neuroscience* **5**:1226–1235
84. Ulrich D. (2002) **Dendritic resonance in rat neocortical pyramidal cells** *Journal of Neurophysiology* **87**:2753–2759

85. Urbanczik R., Senn W. (2014) **Learning by the Dendritic Prediction of Somatic Spiking** *Neuron* **81**:521–528
86. Vargas A. M., et al. (2023) **Task-driven neural network models predict neural dynamics of proprioception**
87. Vaswani A., et al. (2017) **Vaswani, A. et al. Attention Is All You Need in NIPS (2017), 1–11.** <http://arxiv.org/abs/1706.03762>. :1–11
88. Vogels T. P., Sprekeler H., Zenke F., Clopath C., Gerstner W. (2012) **Inhibitory Plasticity Balances Excitation and Inhibition in Sensory Pathways and Memory Networks** *Science* **334**:1569–1573
89. Wang X. D., Chen C., Zhang D., Yao H. (2014) **Cumulative latency advance underlies fast visual processing in desynchronized brain state** *Proceedings of the National Academy of Sciences of the United States of America* **111**:515–520
90. Whittington J. C., Bogacz R. (2017) **An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity** *Neural computation* **29**:1229–1262
91. Whittington J. C., Bogacz R. (2019) **Theories of error back-propagation in the brain** *Trends in cognitive sciences* **23**:235–250
92. Wigner E. (1959) **The Unreasonable Effectiveness of Mathematics in the Natural Sciences. Richard Courant lecture in mathematical sciences** *Communications in pure and applied mathematics* **13**:1–14
93. Wolpert D. M., Ghahramani Z. (2000) **Computational principles of motor neuroscience** *Nature Neuroscience* **3**:1212–1217
94. Xie X., Seung H. S. (2003) **Equivalence of backpropagation and contrastive Hebbian learning in a layered network** *Neural computation* **15**:441–454
95. Zenke F., Ganguli S. (2018) **SuperSpike: Supervised Learning in Multilayer Spiking Neural Networks** *Neural Computation* **30**:1514–1541
1. Abbott L. F., Varela J. a., Sen K., Nelson S. B. (1997) **Synaptic depression and cortical gain control** *Science* **275**:220–224
2. Anderson J. S., Lampl I., Gillespie D. C., Ferster D. (2000) **The contribution of noise to contrast invariance of orientation tuning in cat visual cortex** *Science* **290**:1968–1972
3. Campagnola L., et al. (2022) **Local connectivity and synaptic dynamics in mouse and human neocortex** *Science* **375**
4. Chachuat B. (2007) **Nonlinear and Dynamic Optimization: From Theory to Practice** :1–187
5. Courant R. (1934) **Differential and Integral Calculus**
6. Feynman R., Leighton R., Sands M. (2011) **The Feynman Lectures on Physics** *Mainly Electromagnetism and Matter, Chapt. 19*

7. Flannery M. R. (2005) **The enigma of nonholonomic constraints** *American Journal of Physics* **73**:265–272
8. Friston K. (2010) **The free-energy principle: a unified brain theory?** *Nature Reviews Neuroscience* **11**:127–138
9. Haider P., et al. (2021) **Latent Equilibrium: Arbitrarily fast computation with arbitrarily slow neurons** *Advances in Neural Information Processing Systems* **34**
10. Jordan J., Sacramento J., Wybo W. A. M., Petrovici M. A., Senn W. (2022) **Learning Bayes-optimal dendritic opinion pooling**
11. Lohmiller W., Slotine J.-J. E. (1998) **On Contraction Analysis for Non-linear Systems** *Automatica* **34**:683–696
12. Meulemans A., Zucchet N., Kobayashi S., Oswald J., Sacramento J. (2022) **The least-control principle for local learning at equilibrium** *Advances in Neural Information Processing Systems* **35**
13. Nedeljkov M., Oberguggenberger M. (2012) **Ordinary differential equations with delta function terms** *Publications de l'Institut Mathématique* **91**:125–135
14. Rauch A., La Camera G., Lüscher H.-R., Senn W., Fusi S. (2003) **Neocortical Pyramidal Cells Respond as Integrate- and-Fire Neurons to In Vivo-Like Input Currents** *Journal of Neurophysiology* **90**:1598–1612
15. Scellier B., Bengio Y. (2017) **Equilibrium propagation: Bridging the gap between energy-based models and backpropagation** *Frontiers in computational neuroscience* **11**
16. Simonetto A., Dall'Anese E. (2017) **Prediction-Correction Algorithms for Time-Varying Constrained Optimization** *IEEE Transactions on Signal Processing* **65**:5481–5494
17. Simonetto A., Dall'Anese E., Paternain S., Leus G., Giannakis G. B. (2020) **Time-Varying Convex Optimization: Time-Structured Algorithms and Applications** *Proceedings of the IEEE* **108**:2032–2048
18. Stewart J. (2010) **Multivariable Calculus**
19. Tsodyks M., Markram H. (1997) **The neural code between neocortical pyramidal neurons depends on neurotransmitter release probability** *Proc. Nat. Acad. Sci. USA* **94**:719–723
20. Ulrich D. (2002) **Dendritic resonance in rat neocortical pyramidal cells** *Journal of Neurophysiology* **87**:2753–2759
21. Varela J. A., et al. (1997) **A quantitative description of short-term plasticity at excitatory synapses in layer 2/3 of rat primary visual cortex** *Journal of Neuroscience* **17**:7926–7940
22. Zhao Y., Swamy M. N. (1998) **Novel technique for tracking time-varying minimum and its applications** *Canadian Conference on Electrical and Computer Engineering* **2**:910–913

Article and author information

Walter Senn

Department of Physiology, University of Bern

For correspondence: senn@pyl.unibe.ch

ORCID iD: [0000-0003-3622-0497](https://orcid.org/0000-0003-3622-0497)

Dominik Dold

Department of Physiology, University of Bern, Kirchhoff-Institute for Physics, Heidelberg University, European Space Research and Technology Centre, European Space Agency

Akos F. Kungl

Department of Physiology, University of Bern, Kirchhoff-Institute for Physics, Heidelberg University

Benjamin Ellenberger

Department of Physiology, University of Bern, Insel Data Science Center, University Hospital Bern

Jakob Jordan

Department of Physiology, University of Bern

Yoshua Bengio

MILA, University of Montreal

João Sacramento

Department of Computer Science, ETH Zurich

Mihai A. Petrovici

Department of Physiology, University of Bern, Kirchhoff-Institute for Physics, Heidelberg University

Copyright

© 2023, Senn et al.

This article is distributed under the terms of the [Creative Commons Attribution License](https://creativecommons.org/licenses/by/4.0/), which permits unrestricted use and redistribution provided that the original author and source are credited.

Editors

Reviewing Editor

Srdjan Ostojic

École Normale Supérieure - PSL, Paris, France

Senior Editor

Panayiota Poirazi

FORTH Institute of Molecular Biology and Biotechnology, Heraklion, Greece

Reviewer #1 (Public Review):

The manuscript considers a hierarchical network of neurons, of the type that can be found in sensory cortex, and assumes that they aim to constantly predict sensory inputs that may change in time. The paper describes the dynamics of neurons and rules of synaptic plasticity that minimize the integral of prediction errors over time.

The manuscript describes and analyses the model in great detail, and presents multiple and diverse simulations illustrating the model's functioning. However, the manuscript could be made more accessible and easier to read. The paper may help to understand the organization of cortical neurons, their properties, as well as the function of its particular components (such as apical dendrites).

<https://doi.org/10.7554/eLife.89674.2.sa1>

Reviewer #2 (Public Review):

Neuroscientists often state that we have no theory of the brain. The example of theoretical physics is often cited, where numerous and quite complex phenomena are explained by a compact mathematical description. Lagrangian and Hamiltonian pictures provide such powerful 'single equation'. These frameworks are referred to as 'energy', an elegant way to turn numerous differential equations into a single compact relationship between observable quantities (state variables like position and speed) and scaling constants (like the gravity constant or the Planck constant). Such energy pictures have been used in theoretical neuroscience since the 1980s.

The manuscript "neuronal least-action principle for real-time learning in cortical circuits" by Walter Senn and collaborators describes a theoretical framework to link predictive coding, error-based learning, and neuronal dynamics. The central concept is that an energy function combining self-supervised and supervised objectives is optimized by realistic neuronal dynamics and learning rules when considering the state of a neuron as a mixture of the current membrane potential and its rate of change. As compared with previous energy functions in theoretical neuroscience, this theory captures a more extensive range of observations while satisfying normative constraints. Particularly, no theory had to my knowledge related adaptive dynamics widely observed in the brain (referred to as prospective coding in the text, but is sometimes referred to as adaptive coding or redundancy reduction) with the dynamics of learning rules.

The manuscript first exposes the theory of two previously published papers by the same group on somato-dendritic error with apical and basal dendrites. These dynamics are then related to an energy function, whose optimum recovers the dynamics. The rest of the manuscript illustrates how features of this model fits either normative or observational constraints. Learning follows a combination of self-supervised learning (learning to predict the next step) and supervised learning (learning to predict an external signal). The credit assignment problem is solved by an apical-compartment projecting set of interneurons with learning rules whose role is to align many weight matrices to avoid having to do multiplexing. An extensive method section and supplementary material expand on mathematical proofs and make more explicit the mathematical relationship between different frameworks.

Experts would say that much of the article agglomerates previous theoretical papers by the same authors that have been published recently either in archival servers or in conference proceedings. A number of adaptations to previous theoretical results were necessary, so the present article is not easily reduced to a compendium of previous pre-prints. However, the

manuscript is by no means easy to read. Also, there remain a few thorny assumptions (unobserved details of the learning rules or soma-dendrites interactions), but the theory is likely going to be regarded as an important step towards a comprehensive theory of the brain.

<https://doi.org/10.7554/eLife.89674.2.sa0>

Author response:

The following is the authors' response to the original reviews.

Reviewer #1 (Recommendations For The Authors):

Major comments:

(1) It is nice that the authors compared their model to the one "without lookahead" in Figure 4, but this comparison requires more evidence in my opinion, as I explain in this comment. The model without lookahead is closely related or possibly equivalent to the standard predictive coding. In predictive coding, one can make the network follow the stimulus rapidly by reducing the time constant τ . However, as the time constant decreases, the network would become unstable both in simulations (due to limited integration time step) and physical implementation (due to noise). Therefore I wonder if the proposed model has an advantage over standard predictive coding with an optimized time constant. Hence I suggest to also add a comparison between the proposed model, and the predictive coding with parameters (such as τ) optimized independently for each model. Of course, we know that the time-constant of biological neurons is fixed, but biological neurons might have had different time constants (by changing leak conductance) and such analysis could shed light on the question of why the neurons are organized the way they are.

The comparison with a predictive network for which the neuronal time constants shrink towards 0 is in fact helpful. We added two new subsections in the SI that formally compares the NLA with other approaches, Equilibrium propagation and the Latent Equilibrium, with a version of Equilibrium Propagation also covering the standard predictive coding you describe (SI, Sect. C and D). The Subsection C concludes: "In the Equilibrium propagation we cannot simply take the limit $\tau \rightarrow 0$ since then the dynamics either disappears (when τ remains on the left, $\tau \rightarrow 0$) or explodes (when τ is moved to the right, $d\tau/dt \rightarrow \infty$), leading to either too small or too big jumps."

We have also expanded the passage on the predictive coding in the main text, comparing our instantaneous network processing (up to a remaining time constant τ_{in}) with experimental data from humans (see page 10 of the revised ms). The new paragraph ends with:

"Notice that, from a technical perspective, making the time constants of individual cortical neurons arbitrarily short leads to network instabilities and is unlikely the option chosen by the brain (see SI Sect. C, Comparison to the Equilibrium Propagation)."

A new formal definition of the moving equilibrium in the Methods (Sect. F) helps to understand this notion of being in a balanced equilibrium state during the dynamics. This formal definition directly leads to the contraction analysis in the SI, Sect. D, showing why the Latent Equilibrium is always contractive, while the current form of the NLA may show jumps at the corner of a ReLu (since a second order derivative of the transfer function enters in the error propagation).

The reviewer perhaps has additional simulations in mind that compare the robustness of the different models. However, as this paper is more about presenting a novel concept with a comprehensive theory (summing up to 45 pages), we prefer to not add more than the simulations necessary to check the statements of the theorems.

(2) I found this paper difficult to follow, because the Results sections went straight into details, and various elements of the model were introduced without explaining why they are necessary. Furthermore, the neural implementation was introduced after the model simulations. I suggest reorganizing the manuscript, to describe the model following Marr's levels of description and then presenting the results of simulations. In particular, I suggest starting the Results section by explaining what computation the network is trying to achieve (describe the setup, function L , define its integral over time, and explain that the goal is to find a model minimizing this integral). Then, I suggest presenting the algorithm the neurons need to employ to minimize this integral, i.e. their dynamics and plasticity (I wonder if $r = \rho(u) + \tau \rho(u)'$ is a consequence of action minimization or a necessary assumption - please clarify it). Next please explain how the algorithms could be implemented in biological neurons. Afterward please present the results of the simulation.

We are sorry to realize that we could not convey the main message clearly enough. After rewriting the paper and straightening the narrative, we hope it is simpler to understand now.

The paper does not suggest a new model to solve a task, and writing down the function to be minimized is not enough. The point of the NLA is that the time integral of our Lagrangian is minimized with respect to the prospective coordinates, i.e. the discounted future voltage. It is about the question how dynamic equations in biology are derived. Of course, we also solve these equations, prove theorems and perform simulations. But the main point that biology seems to deal with time differently than physics deals with time. Biology “thinks” in terms of future quantities, physics “thinks” in terms of current quantities. We tried to explain this better now in the Introduction, the Results (e.g. after Eq. 5) and the Methods.

(3) Understanding the paper requires background knowledge that most readers of eLife are unlikely to have, even if they are mathematically minded. For example, I am from the field of computational neuroscience, and I have never heard about Least Action principle from physics or the EulerLagrange equation. I felt lost after reading this paper, and to be able to write this review I needed to watch videos on the Euler-Lagrange equation. To help other readers, I have two suggestions: First, I feel that Eq 4-6 could be moved to the methods, because I found the concept of $u \sim$ difficult to understand, and it does not appear in the algorithm. Second, I advise to write in the Introduction, what knowledge is required to follow this paper, and point the readers to resources where they can find the required information. The authors may specify what background is required to follow the main text, and what is required to understand the methods.

We hope that after explaining the rationale better, it becomes clear that we cannot skip the equations for the prospective coordinates. Likewise, the Euler-Lagrange equations need to be presented in the abstract form, since these are the equations that are eventually transformed into the “model”. We tried to give the basic intuition for this in the main text. As we explained above, the equations asked to be skipped represent the essence of the proposal. It is about how to derive a model equations.

Moreover, we give more explanations in the Methods to understand the derivations, and we refer to the specifically sections in the SI for further details. We are aware that a full understanding of the theory requires some basic knowledge of the calculus of variation.

We are hesitating to write in the Introduction what type of knowledge is required to understand the paper. An understanding can be on various levels. Moreover, the materials that are considered to be helpful depend on the background. While for some it is a Youtube, for some Wikipedia, and for others it is a textbook where specific ingredients can be extracted. But we do cite two textbooks in the Results and more in the SI, Sect. F, when referring to the principle of least action in physics and the mathematics, including weblinks.

Minor comments

Eq.3: The Authors refer to this equation as a Lagrangian. Could you please clarify why? Is the logic to minimize the energy subject to a constraint that Cost = 0?

Thanks for asking. The cost is not really a constraint, it is globally minimized, in parallel steps. We are explaining this right after Eq. 3. “We prospectively’ minimize L locally across a voltage trajectory, so that, as a consequence, the local synaptic plasticity for W will globally reduce the cost along the trajectory (Theorem 1 below).”

We were adding two sentence that explain why this function in Eq. 3 is called a Lagrangian: “While in classical energy-based approaches L is called the total energy, we call it the Lagrangian’ because it will be integrated along real and virtual voltage trajectories as done in variational calculus (leading to the Euler-Lagrange equations, see below and SI, Sect. F)”

p.4, below Eq. 5 - Please explain the rationale behind NLA, i.e. why is it beneficial that “the trajectory $u(t)$ keeps the action A stationary with respect to small variations δu ”? I guess you wish to minimize L integrated over time, but this is not evident from the text.

Hmm, yes and no. We wish to minimize the cost, and on the way there minimize the action. Since the global minimization of C is technically difficult, one looks for stationary trajectory as defined in the cited sentence, while minimizing L with respect to W , to eventually minimize the cost.

In the text we now explain after Eq. 5:

“The motivation to search for a trajectory that keeps the action stationary is borrowed from physics. The motivation to search for a stationary trajectory by varying the near-future voltages $\tilde{u}$ instead of u is assigned to the evolutionary pressure in biology to ‘think ahead of time’. To not react too late, internal delays involved in the integration of external feedback need to be considered and eventually need to be overcome. In fact, only for the ‘prospective coordinates’ defined by looking ahead into the future, even when only virtually, will a real-time learning from feedback errors become possible (as expressed by our Theorems below).”

Bottom of page 8. The authors say that in the case of single equilibrium and strong nudging the model reduced to the Least Control Principle. Does it also reduce to Predictive coding for supervised learning? If so, it would be helpful to state so.

Yes, in this case the prediction error in the apical dendrite becomes the one of predictive coding. We are stating this now right at the end of the cited sentence:

“In the case of strong nudging and a single steady-state equilibrium, the NLA principle reduces to the Least-Control Principle (Meulemans et al., 2022) that minimizes the mismatch energy E^M for a constant input and a constant target, with the apical prediction error becoming the prediction error from standard predictive coding (Rao & Ballard, 1999).”

In the Discussion we also added a further point (iv) to compare the NLA principle with predictive coding. Both “improve” the sensory representation, but the NLA does in favor of

an output, and the predictive coding in favor of the sensory prediction itself (see Discussion).

Whenever you refer to supplementary materials, please specify the section, so it is easier for the reader to find it.

Done. Sorry to not have done it earlier. We are now also indicate specific sections when referring to the Methods.

Reviewer #2 (Recommendations For The Authors):

There are no major issues with this article, but I have several considerations that I think would greatly improve the impact, clarity, and validity of the claims.

(1) Unifying the narrative. There are many many ideas put forward in what feels like a deluge. While I appreciate the enthusiasm, as a reader I found it hard to understand what it was that the authors thought was the main breakthrough. For instance, the abstract, results, introduction, and discussion all seem to provide different answers to that question. The abstract seems to focus on the motor error idea. The introduction seems to focus on the novel prospective+predictive setup of the energy function. The discussion lists the different perks of the theory (delay compensation, moving equilibrium, microcircuit) without referring to the prospective+predictive setup of the energy function.

Thanks much for these helpful hints. Yes, the paper became an agglomerate of many ideas, also own to the fact that we wish to show how the NLA principle can be applied to explain various phenomenology in neuroscience. We now simplified the narrative to this one point of providing a novel theoretical framework for neuroscience, and explaining why this is novel and why it “suddenly works” (the prospective minimization of the energy).

As you can see from the dominating red in the revised pdf, we did fully rewrite Abstract, Introduction and Discussion under the narrative of the NLA and prospective coding.

(2) Laying out the organization of the notation clearly. There are quite a few subtle distinctions of what is meant by the different weight matrices (omnibus matrix then input vs recurrent then layered architecture), different temporal horizon formalisms (bar, not bar, tilde), different operators (L , curly L , derivative version, integral version). These different levels are introduced on the fly, which makes it harder to grasp. The fact that there are many duplicate notations for the same quantities does not help the reader. For instance u_0 becomes equal to u_N at one point (above Eq 25). Another example is the constant flipping between integrated and 'current input' pictures. So laying out the multiple layers early, making a table or a figure for the notation, or sticking with one level would help convey the idea to a wide readership.

Thanks for the hints. We included the table you suggested, but put it to the SI as it became a full page itself. We banned the curly L abbreviating the look-ahead operator.

The “change of notation” you are alluding to is tricky, though. In a recurrent layer, the index of the output neuron is called o . In a forward network with N layer, the index of the output neurons becomes the last layer N . One has to introduce the layer index l anyway for the deeper layers $l < N$, and we found it more consistent to explain that, while switching from the recurrent to the forward network, the voltage of the output layer becomes now $u_o = u_N$. There are more of these examples, like the weight matrix W splitting into a intrinsic network part W_{net} across which errors backpropagate, and a part conveying the input, W_{in} , that has to be excluded when writing the backpropagation formula for general networks. Again, in the case of the feedforward networks, the notation reduces to W_l , with index l coding for

the layer. Presenting the general approach and a specific example may appear as we would duplicate notations – we haven't found a solution here.

(3) Separate the algorithm from the implementation level. I particularly struggled with separating the ideas that belonged to the algorithm level (cost function, optimization objectives) and the biophysics. The two are interwoven in a way that does not have to be. Particularly, some of the normative elements may be implemented by other types of biophysics than the authors have in mind. It is for this reason that I think that separating more clearly what belongs to the implementation and algorithm levels would help make the ideas more widely understood. On this point, a trigger point for me was the definition of the 'prospective input rates' e_i , which comes in the second paragraph.

We are very sorry to have made you thinking that the 'prospective input rates' would be e_i . The prospective input rates are r_i . The misunderstanding likely appeared by an unclear formulation from our side that is now corrected (see first and second paragraph of the Results where we introduce r_i and e_i).

From a biophysical perspective, it is quite arbitrary to define the input to be the difference between the basal input and the somatic (prospective) potential. It sounds like it comes from some unclear normative picture at this point. But the authors seem to have in mind to use the fact that the somatic potential is the sum of apical and basal input, that's the biophysical picture.

We hope to have disentangled the normative and biophysical view in the 2nd and 3rd paragraph of the Results, respectively. We introduce the prospective error e_i as abstract notion in the first paragraph, while explaining that it will be interpreted as somato-dendritic mismatch error in neuron i in the next paragraph. The second paragraph contains the biophysical details with the apical and basal morphology.

(4) Experts and non-expert would appreciate an explanation of why/how the choice of state variables matters in the NLA. The prospective coding state variables cannot be said to be the naïve guess. Why does the simple $u, \dot{u}$ not work as state variables applied on the same energy function, as would be a naïve application of the Lagrangian ideas?

We are very glad for this hint to present an intuition behind the variation of the action with respect to a prospective state, instead of the state itself. The simple $L(u, \dot{u})$ does not work because one does not obtain the first-order voltage dynamics compatible with the biophysics. We made an effort to explain the intuition to non-experts and experts in an additional paragraph right after presenting the voltage and error dynamics (Eq. 7 on page 4).

Here is how the paragraph starts (not displaying the formulas here):

“From the point of view of theoretical physics, where the laws of motion derived from the least-action principle contain an acceleration term (as in Newton's law of motion, like ... for a harmonic oscillator), one may wonder why no second-order time derivative appears in the NLA dynamics. As an intuitive example, consider driving into a bend. Looking ahead in time helps us to reduce the lateral acceleration by braking early enough, as opposed to braking only when the lateral acceleration is already present. This intuition is captured by minimizing the neuronal action A with respect to the discounted future voltages $\tilde{u}_i$ instead of the instantaneous voltages u_i .

Keeping up an internal equilibrium in the presence of a changing environment requires to look ahead and compensate early for the predicted perturbations.

Technically, ...”

More details are given in the Methods after Eq. 20. Moreover, in the last part of the SI, Sect. F, we have made the link to the least-action principle in physics more explicitly. There we show how the voltage dynamics can be derived from the physical least-action principle by including the Rayleigh dissipation (Eq. 92 and 95).

(5) Specify that the learning rules have not been observed. Though the learning rules are Hebbian, the details of the rules have not to my knowledge been observed. Would be worth mentioning as this is a sticking point of most related theories.

We agree, and we do now explicitly write in the Discussion that the learning rule still awaits to be experimentally tested.

1. Some relevant literature. Chalk et al. PNAS (2018) have explored the relationship between temporal predictive coding and Rao & Ballard predictive coding based on the parameters of the cost function. Harkin et al. eLife (2023) have shown that 'prospective coding' also takes place in the serotonergic system, while Kim ... Ma (2021) have put forward similar ideas for dopamine, both may participate in setting the cost function. Instantaneous voltage propagation is also a focus of Greedy et al. (2023). The authors cite Zenke et al. for spiking error propagation, but there are biological references to that end.

Thanks much for these hints. We do now cite the book of Gerstner & Kistler on spiking neurons, and more specifically the spike-based approach for learning to represent signals (Brendel, ..., Machens, Denève, PLoS CB, 2020). Otherwise, we had difficulties to incorporate the other literature that seems to us not directly related to our approach, even when related notions come up (like predictive coding and temporal processing in Chalk et al. (2018), where various temporal coding schemes coding efficiency is studied as a function of the signal-to-noise ratio), or the apical activities in Greedy et al. (2022), where bursting, multiplexing and synaptic facilitation arises). We found it would confuse more than it would help if we would cite these papers too (we do already cite 95 papers).

(7) In the main text, theorem two is presented as proof without assumptions on the level of nudging, but the actual proof uses strong assumptions in that respect, relying on numerical ad hoc observations for the general case.

Thanks for pointing this out. We agree it is a better style to state all the critical assumptions in Theorem itself, rather than deferring them to the Methods. We now state: “Then, for suitable top-down nudging, learning rates, and initial conditions, theweights ...evolve such that...”.

(8) In the discussion regarding error-backpropagation, it seems to me that it could be clarified that the current algorithm asks for a weight alignment between FF and FB matrices as well as between FB and interneuron circuit matrices. Whether all of these matrices can be learned together remains to be shown; neither Akrou, Kunin nor Max et al. have shown this explicitly. Particularly when there are other inputs to the apical dendrites from other areas.

Yes, it is difficult to learn to align all in parallel. Nevertheless, our simulations in fact do align the lateral and vertical circuits, at is also claimed in Theorem 2. Yet, as specified in the theorem, “for suitable learning rates” (that were all the same, but were commonly reduced after some training time, as previously explained in the Methods, Details for Fig. 5).

In the Discussion we now emphasis that, in general, simulating all the circuitries jointly from scratch in a single phase is tricky. We write:

“A fundamental difficulty arises when the neuronal implementation of the Euler-Lagrange equations requires an additional microcircuit with its own dynamics. This is the case for the suggested microcircuit extracting the local errors. Formally, the representation of the apical feedback errors first needs to be learned before the errors can teach the feedforward synapses on the basal dendrites. We showed that this error learning can itself be formulated as minimizing an apical mismatch energy. What the lateral feedback through interneurons cannot explain away from the top-down feedback remains as apical prediction error.

Ideally, while the network synapses targetting the basal tree are performing gradient descent on the global cost, the microcircuit synapses involved in the lateral feedback are performing gradient descent on local error functions, both at any moment in time.

The simulations show that this intertwined system can in fact learn simultaneously with a common learning rate that is properly tuned. The cortical model network of inter- and pyramidal neurons learned to classify handwritten digits on the fly, with 10 digit samples presented per second. Yet, the overall learning is more robust if the error learning in the apical dendrites operates in phases without output teaching but with corresponding sensory activity, as may arise during sleep (see e.g. Deperrois et al., 2022 and 2023).”

(9) *The short-term depression model is assuming a slow type of short-term depression, not the fast types that are the focus of much recent experimental literature (like Campagnola et al. Science 2022).*

This assumption should be specified.

Thanks for hinting to this literature that we were not aware of. We are now citing the releaseindependent plasticity (Campagnola et al. 2022) in the context of our synaptic depression model.

(10) *There seems to be a small notation issue: Eq 21 combines vectors of the size of the full network ($\bar{e}$) and the size of the readout network ($\bar{e}^*$).*

Well, for notational convenience we set the target error to $e^*=0$ for non-output neurons. This way we can write the total error for an arbitrary network neuron as the sum of the backpropagated error plus the putative target error (if the neuron is an output neuron). Otherwise we would always have to distinguish between network neuron that may be output neurons, and those that are not. We did say this in the main text, but are repeating it now again right after Eq. 21. -- Notations are often the result of a tradoff.